BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỎ - ĐỊA CHẤT



ĐÀM MẠNH DŨNG

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ THÔNG TIN

TÊN ĐỀ TÀI:

XÂY DỰNG NỀN TẢNG AI HỖ TRỢ NGHIÊN CỨU VÀ VIẾT BÀI BÁO KHOA HỌC VINUNI DỰA TRÊN AGENTIC RAG, TRUY XUẤT TRI THỨC HỌC THUẬT VÀ KIỂM CHỨNG TRÍCH DẪN

Hà Nội – Năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỎ - ĐỊA CHẤT

ĐỒ ÁN TỐT NGHIỆP

CHUYÊN NGÀNH: HỆ THỐNG THÔNG TIN

TÊN ĐỀ TÀI:

XÂY DỰNG NỀN TẢNG AI HỖ TRỢ NGHIÊN CỨU VÀ VIẾT BÀI BÁO KHOA HỌC VINUNI DỰA TRÊN AGENTIC RAG, TRUY XUẤT TRI THỨC HỌC THUẬT VÀ KIỂM CHỨNG TRÍCH DẪN

GIÁO VIÊN HƯỚNG DẪN	SINH VIÊN THỰC HIỆN
(Chữ ký)	*(Chữ ký)*
ThS. Vũ Lan Phương	**Đàm Mạnh Dũng**

Hà Nội – Năm 2026

PHIẾU GIAO ĐỀ TÀI TỐT NGHIỆP

1. Thông tin chung

Tên đề tài: Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn

Sinh viên: Đàm Mạnh Dũng — Mã sinh viên: 2221050359

Điện thoại: 0978461865 — Email: dammanhdungvn@gmail.com

Lớp: DCCTCT67_09B — Hệ đào tạo: Đại học chính quy

Ngành: Công nghệ thông tin — Chuyên ngành: Hệ thống thông tin

Nơi thực hiện: Trường ĐH Mỏ – Địa chất — Thời gian làm ĐATN: Năm 2026

2. Mục tiêu của ĐATN

Mục tiêu tổng quát:

- Xây dựng nền tảng Trợ lý AI nghiên cứu khoa học VeriScholar cho học giả VinUni.
- Triệt tiêu ảo giác qua Hybrid RAG và kiểm chứng trích dẫn Bounding Box hai tầng.
- Tự động hóa biên dịch LaTeX trong sandbox E2B và bảo mật khóa riêng BYOK.

Mục tiêu cụ thể:

– **Về sản phẩm:**
  – Phân hệ Khám phá: Tìm kiếm học thuật đa nguồn (arXiv, Semantic Scholar, CrossRef) và lưu trữ CAS SHA-256.
  – Phân hệ Đọc sâu: Đọc PDF Canvas hai cột kết hợp kiểm chứng tọa độ Bounding Box.
  – Phân hệ GraphRAG: Đồ thị tri thức 4 thực thể (Paper, Method, Dataset, MetricResult).
  – Phân hệ Soạn thảo: Soạn thảo LaTeX trực tuyến, tự sửa lỗi trong sandbox E2B.
  – Phân hệ Bảo mật: Quản lý kho khóa API cá nhân BYOK mã hóa AES-256-GCM.
– **Về kiến thức và kỹ năng:**
  – Vận dụng kiến thức: Kỹ nghệ phần mềm, CSDL nâng cao, Kiến trúc hệ thống, NLP.
  – Nắm vững quy trình phát triển phần mềm theo phương pháp kiểm thử TDD.
  – Làm chủ kỹ thuật Hybrid RAG, đồ thị tri thức GraphRAG và tác nhân ReAct tự chủ.
  – Rèn luyện năng lực nghiên cứu độc lập và tuân thủ liêm chính học thuật.

3. Các nhiệm vụ cụ thể của ĐATN

Để hoàn thành các mục tiêu đề ra, sinh viên thực hiện các nhiệm vụ cụ thể sau:

- Khảo sát quy trình nghiên cứu, viết bài báo tại VinUni; xác định bài toán liêm chính học thuật.
- Đặc tả yêu cầu chức năng và phi chức năng hệ thống (BYOK, trễ RAG $\leq 1{,}5\,\mathrm{s}$, IoU $\geq 0{,}85$).
- Thiết kế kiến trúc Lục giác, mô hình CSDL quan hệ PostgreSQL kết hợp véc-tơ pgvector.
- Cài đặt hệ thống Backend (FastAPI, pgvector, LangGraph) và Frontend (TypeScript, React 19).
- Tích hợp sandbox E2B MicroVM biên dịch LaTeX cô lập an toàn, ngăn chặn mã độc hại.
- Triển khai toàn bộ quy trình kiểm thử TDD với 580 kịch bản kiểm thử tự động (340 Backend pytest và 240 Frontend vitest, 100% pass).
- Đánh giá thực nghiệm hệ thống trên các bài báo thực tế, đo độ trễ và độ chính xác tọa độ.
- Viết báo cáo tổng kết đồ án tốt nghiệp, chuẩn bị slide thuyết trình và đóng gói sản phẩm.

4. Lời cam đoan của sinh viên:

Tôi – Đàm Mạnh Dũng – cam đoan ĐATN là công trình nghiên cứu độc lập của bản thân, được thực hiện dưới sự hướng dẫn của ThS. Vũ Lan Phương. Mọi kết quả, số liệu và mã nguồn trình bày trong đồ án hoàn toàn trung thực, không sao chép nguyên văn bất kỳ công trình nào khác.

Hà Nội, ngày……tháng……năm 2026

TRƯỞNG BỘ MÔN	**GIẢNG VIÊN HƯỚNG DẪN**	**SINH VIÊN THỰC HIỆN**
(Ký và ghi rõ họ tên)	*(Ký và ghi rõ họ tên)*	*(Ký và ghi rõ họ tên)*
	ThS. Vũ Lan Phương	**Đàm Mạnh Dũng**

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN HỆ THỐNG THÔNG TIN VÀ TRI THỨC

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

ĐỀ CƯƠNG ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Sinh viên thực hiện:

TT	Mã sinh viên	Họ và tên	Lớp	Điện thoại	E-mail
1	2221050359	Đàm Mạnh Dũng	DCCTCT67_09B	0978461865	dammanhdungvn@gmail.com

Giảng viên hướng dẫn:

TT	Mã cán bộ	Họ và tên	Đơn vị	Điện thoại	E-mail
1	0802-09	ThS. Vũ Lan Phương	Khoa CNTT	0984223379	vulanphuong@humg.edu.vn

1. Tên đề tài

Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn

2. Lý do chọn đề tài

Trong bối cảnh bùng nổ nghiên cứu khoa học, giảng viên và học giả tại Trường Đại học VinUni đối mặt với khối lượng lớn tài liệu quốc tế đa nguồn cần đọc hiểu, tổng hợp và thẩm định. Việc tiếp cận nhanh chóng các phương pháp tiên tiến đồng thời duy trì năng suất công bố khoa học chuẩn IEEE/ACM là nhu cầu thường trực của các nhóm nghiên cứu.

Tuy nhiên, các mô hình ngôn ngữ lớn hiện nay thường mắc lỗi ảo giác tri thức (*Hallucination*), tự bịa đặt trích dẫn hoặc dẫn chiếu sai lệch số liệu. Bên cạnh đó, các công cụ phổ biến như ChatPDF làm mất cấu trúc hai cột và không có tọa độ minh chứng, trong khi Overleaf chưa tích hợp AI tự sửa lỗi biên dịch LaTeX. Vì vậy, việc nghiên cứu và xây dựng nền tảng VeriScholar là hết sức cấp thiết, cung cấp giải pháp toàn diện hỗ trợ học giả VinUni từ khâu khám phá tài liệu, đọc sâu kiểm chứng tọa độ, đến biên dịch bản thảo khoa học chuẩn mực.

3. Mục tiêu của đề tài

– Mục tiêu tổng quát:

- Xây dựng Trợ lý AI nghiên cứu khoa học VeriScholar cho học giả VinUni.
- Triệt tiêu ảo giác qua kiểm chứng trích dẫn tọa độ Bounding Box hai tầng.
- Tự động hóa biên dịch LaTeX và sửa lỗi cú pháp trong sandbox E2B.

3. Mục tiêu của đề tài (tiếp theo)

– Về sản phẩm:

- Hệ thống Backend: REST API và 5 luồng SSE theo Kiến trúc Lục giác.
- Phân hệ Khám phá tài liệu: Tìm kiếm đa nguồn (Exa AI, arXiv, Semantic Scholar, CrossRef) kết hợp truy xuất lai pgvector Dense và BM25 Sparse.
- Phân hệ Đọc sâu PDF: Phân tích 2 cột, hỏi đáp và đối chiếu tọa độ Bounding Box.
- Phân hệ GraphRAG: Trích xuất 4 thực thể (Paper, Method, Dataset, MetricResult) và phân cụm Louvain.
- Phân hệ Soạn thảo và Bảo mật: Tích hợp sandbox E2B, tác nhân LangGraph ReAct tự sửa cú pháp LaTeX và mã hóa khóa BYOK bằng AES-256-GCM.
- Giao diện Web: Phát triển trên React 19, TypeScript, Tailwind và Monaco.

– Về kiến thức và kỹ năng:

- Vận dụng kiến thức: Kỹ nghệ phần mềm, CSDL nâng cao, Học máy và NLP.
- Làm chủ quy trình TDD, bảo đảm 100% các cổng xác minh chất lượng.
- Thành thạo công nghệ: FastAPI, SQLAlchemy, PostgreSQL, React 19, E2B.
- Nâng cao năng lực nghiên cứu độc lập và tư duy liêm chính học thuật.

4. Các nhiệm vụ cụ thể

Các nhiệm vụ cụ thể của đồ án bao gồm:

– Khảo sát bài toán nghiên cứu tại VinUni và lập đề cương chi tiết.
– Đặc tả yêu cầu chức năng, phi chức năng hệ thống (BYOK, IoU $\geq 0{,}85$).
– Thiết kế kiến trúc Lục giác, mô hình quan hệ và CSDL pgvector.
– Thiết kế giao diện (UI/UX) và các biểu đồ tuần tự nghiệp vụ cốt lõi.
– Xây dựng module Backend: Xác thực, phân quyền, thư viện và lưu trữ CAS.
– Xây dựng phân hệ Hybrid RAG (pgvector + BM25) và kiểm chứng Bbox.
– Xây dựng GraphRAG và tác nhân AI sửa lỗi LaTeX trong sandbox E2B.
– Xây dựng Frontend: Monaco Editor, PDF Canvas và streaming SSE.
– Kiểm thử tự động TDD, đánh giá thực nghiệm và đóng gói sản phẩm.

5. Kế hoạch thực hiện

Tuần	Thời gian	Nội dung công việc
1	24/08/2026 – 30/08/2026	Khảo sát bài toán nghiên cứu tại VinUni, thu thập tài liệu tham khảo liên quan, lập đề cương chi tiết và kế hoạch thực hiện đồ án.
2	31/08/2026 – 06/09/2026	Phân tích yêu cầu nghiệp vụ; xác định các tác nhân (Actor), biểu đồ Use Case tổng thể và đặc tả chi tiết các Use Case chính.
3	07/09/2026 – 13/09/2026	Thiết kế kiến trúc Lục giác (Hexagonal Architecture), thiết kế CSDL quan hệ PostgreSQL và cơ sở dữ liệu véc-tơ pgvector.
4	14/09/2026 – 20/09/2026	Thiết kế giao diện người dùng khoa học tối giản (UI/UX); thiết kế biểu đồ tuần tự cho các luồng nghiệp vụ RAG và kiểm chứng Bounding Box.
5	21/09/2026 – 27/09/2026	Xây dựng Backend cốt lõi: Cấu hình môi trường, triển khai tầng Core nghiệp vụ, xác thực JWT, lưu trữ CAS và mã hóa BYOK.
6	28/09/2026 – 04/10/2026	Báo cáo tiến độ giữa kỳ với Bộ môn và GVHD. Hiện thực hóa phân hệ Hybrid RAG: Tìm kiếm kết hợp HNSW + BM25, dung hợp RRF, tái xếp hạng Cross-Encoder và trích xuất Bounding Box PDF.
7	05/10/2026 – 11/10/2026	Hiện thực hóa phân hệ Đồ thị tri thức (GraphRAG) và Tác nhân AI ReAct tự động phục hồi lỗi biên dịch LaTeX trong sandbox E2B.
8	12/10/2026 – 18/10/2026	Xây dựng giao diện Frontend (React 19): Monaco Editor, trình hiển thị PDF Canvas thời gian thực, bảng dẫn chứng Bbox và luồng SSE.
9	19/10/2026 – 25/10/2026	Tích hợp toàn diện hệ thống Frontend – Backend; thực hiện kiểm thử tự động TDD (Unit Test, Boundary Test, E2E); tối ưu hóa hiệu năng và bảo mật.
10	26/10/2026 – 02/11/2026	Đánh giá thực nghiệm trên các ấn phẩm khoa học thực tế; hoàn thiện báo cáo tốt nghiệp, slide thuyết trình, đóng gói sản phẩm và nộp báo cáo về Bộ môn Hệ thống thông tin và Tri thức ngày 02/11/2026.

6. Nơi thực hiện đồ án: Trường Đại học Mỏ – Địa chất

Hà Nội, ngày……tháng……năm 2026

Giảng viên hướng dẫn	**Sinh viên thực hiện**
(ký, họ tên)	*(ký, họ tên)*
ThS. Vũ Lan Phương	**Đàm Mạnh Dũng**

LỜI CẢM ƠN

Để hoàn thành bản đồ án tốt nghiệp đại học này, trước hết em xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất tới ThS. Vũ Lan Phương – Giảng viên Khoa Công nghệ thông tin, Trường Đại học Mỏ - Địa chất. Cô đã trực tiếp định hướng đề tài, tận tình chỉ bảo, đóng góp nhiều ý kiến học thuật và chuyên môn quý báu, cũng như luôn tạo mọi điều kiện thuận lợi nhất để em nghiên cứu và hiện thực hóa các công nghệ trí tuệ nhân tạo hiện đại vào nền tảng VeriScholar.

Em xin trân trọng gửi lời cảm ơn chân thành tới các thầy, cô giáo trong Bộ môn Hệ thống thông tin và Tri thức, Bộ môn Công nghệ thông tin cùng toàn thể các thầy, cô trong Khoa Công nghệ thông tin – Trường Đại học Mỏ - Địa chất. Những tri thức nền tảng vững chắc, phương pháp luận nghiên cứu khoa học cùng sự dạy dỗ tâm huyết của các thầy, cô trong suốt những năm tháng học tập và rèn luyện trên giảng đường đại học chính là hành trang quý giá giúp em hoàn thành công trình tốt nghiệp này.

Cuối cùng, em xin gửi lời tri ân sâu sắc tới bố mẹ và gia đình, đồng thời xin cảm ơn bạn bè đã luôn ở bên động viên, sẻ chia và là điểm tựa tinh thần vững chắc cho em trong suốt quá trình học tập và hoàn thiện đồ án.

Dù đã nỗ lực hết mình để nghiên cứu, xây dựng hệ thống và biên soạn báo cáo một cách khoa học, cẩn trọng nhất, song do giới hạn về thời gian và kinh nghiệm thực tiễn, đồ án chắc chắn khó tránh khỏi những thiếu sót nhất định. Em kính mong nhận được những ý kiến đóng góp, chỉ dẫn quý báu từ quý Thầy, Cô trong Hội đồng chấm đồ án tốt nghiệp để công trình được hoàn thiện hơn nữa.

Hà Nội, ngày 02 tháng 11 năm 2026

Sinh viên thực hiện

Đàm Mạnh Dũng

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI TỐT NGHIỆP iii
ĐỀ CƯƠNG ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC v
DANH MỤC HÌNH ẢNH xii
DANH MỤC BẢNG BIỂU xiv
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT xv
CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI 1
1.1 Giới thiệu đề tài 1
1.2 Tính cấp thiết của đề tài 1
1.3 Mục tiêu nghiên cứu 3
1.3.1 Mục tiêu tổng quát 3
1.3.2 Mục tiêu kỹ thuật cụ thể 3
1.4 Nội dung nghiên cứu và phương pháp tiếp cận 4
1.4.1 Nội dung nghiên cứu 4
1.4.2 Phương pháp tiếp cận 4
1.5 Phạm vi và đối tượng nghiên cứu 5
1.6 Bố cục báo cáo đồ án 5
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ 6
2.1 Cơ sở dữ liệu quan hệ và mở rộng Vector 6
2.1.1 Hệ quản trị cơ sở dữ liệu quan hệ đối tượng PostgreSQL 6
2.1.2 Tiện ích mở rộng pgvector và cấu trúc chỉ mục HNSW 6
2.1.3 Hệ thống đệm dữ liệu bộ nhớ truy cập nhanh Redis 7
2.1.4 Luận giải lựa chọn cơ sở dữ liệu Vector tích hợp 7
2.2 Kỹ nghệ truy xuất thông tin hỗn hợp 8
2.2.1 Phân tích bố cục tài liệu PDF hai cột và chuẩn hóa tọa độ Bounding Box 8
2.2.2 Không gian truy xuất Vector đặc và từ khóa thưa hai pha 9
2.2.3 So sánh đặc tính kỹ thuật giữa hai không gian truy xuất 9
2.2.4 Thuật toán hợp nhất xếp hạng đảo ngược và cơ chế tái xếp hạng. 10
2.3 Mô hình ngôn ngữ lớn và tạo sinh tăng cường truy xuất nâng cao 11
2.3.1 Hạn chế của mô hình ngôn ngữ lớn và nguyên lý RAG 11
2.3.2 Kiến trúc RAG nâng cao đa giai đoạn 12

2.3.3 Cơ chế phản xạ tự đánh giá Self-RAG 12
2.3.4 Cơ chế hiệu chỉnh truy xuất và kiểm soát ảo giác (Corrective RAG) 13
2.4 Đồ thị tri thức học thuật và phân cụm cộng đồng 13
2.4.1 Mô hình biểu diễn mạng lưới tri thức học thuật 4 thực thể 13
2.4.2 Thuật toán phân cụm Louvain và hàm mục tiêu Modularity Q ... 14
2.4.3 Cơ chế biên dịch trước trang bách khoa tri thức (Graph Wiki).... 15
2.5 Kiến trúc tác nhân tự chủ và môi trường Sandbox an toàn 15
2.5.1 Mô hình suy luận và hành động ReAct phối hợp gọi công cụ..... 15
2.5.2 Khung điều phối tác nhân có trạng thái LangGraph 16
2.5.3 Môi trường máy ảo MicroVM E2B cô lập cấp hệ điều hành và vòng lặp tự sửa lỗi LaTeX 16
2.5.4 Mô hình quản lý khóa người dùng tự cấp Bring Your Own Key (BYOK) và chuẩn mã hóa AES-256-GCM 17
2.5.5 Nền tảng quan sát vận hành và truy vết suy luận Langfuse..... 18
CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG..... 19
3.1 Khảo sát yêu cầu người dùng học thuật 19
3.2 Khảo sát nghiệp vụ và quy trình xử lý tài liệu nghiên cứu 19
3.2.1 Khảo sát quy trình tìm kiếm và thu thập tài liệu nghiên cứu 19
3.2.2 Khảo sát quy trình đọc sâu và thẩm định nguồn trích dẫn 20
3.2.3 Khảo sát quy trình quản lý và liên kết đồ thị tri thức học thuật ... 20
3.2.4 Khảo sát quy trình biên soạn bản thảo và sửa lỗi biên dịch LaTeX 20
3.2.5 Khảo sát quy trình quản lý hồ sơ học giả và bảo mật khóa cá nhân 21
3.3 Thiết kế kiến trúc hệ thống và phân tầng Lục giác 21
3.3.1 Nguyên lý phân tầng Kiến trúc Lục giác 22
3.3.2 Mô hình 4 Không gian nghiên cứu (4 Research Spaces)..... 22
3.3.3 Thiết kế các đường ống thuật toán xử lý cốt lõi 23
3.4 Phân tích các ca sử dụng (Use Case) của hệ thống 25
3.4.1 Biểu đồ Use Case tổng quan của hệ thống 25
3.4.2 Các tác nhân và danh mục ca sử dụng của hệ thống 25
3.4.3 Phân tích các Use Case phía Học giả..... 27
3.4.4 Cơ chế bảo mật và vận hành hệ thống phía Quản trị viên..... 31
3.5 Phân tích một số biểu đồ tuần tự chính của hệ thống..... 31
3.5.1 Biểu đồ tuần tự Use Case "Tra cứu và tải tài liệu nghiên cứu".... 31

3.5.2 Biểu đồ tuần tự Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ" 32
3.5.3 Biểu đồ tuần tự Use Case "Khai phá Đồ thị tri thức và truy vấn liên tài liệu" 32
3.5.4 Biểu đồ tuần tự Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX" 32
3.6 Thiết kế cơ sở dữ liệu cho hệ thống 33
3.6.1 Xây dựng biểu đồ lớp (Class Diagram) của hệ thống 33
3.6.2 Cấu trúc các phân hệ dữ liệu của hệ thống 33
3.6.3 Đặc tả chi tiết các bảng cơ sở dữ liệu hạt nhân 35
3.6.4 Thiết kế hạ tầng lưu trữ vector và đồ thị tri thức phụ trợ 36
CHƯƠNG 4 CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM 39
4.1 Môi trường cài đặt và tổ chức Monorepo 39
4.1.1 Môi trường phát triển tích hợp và quản lý mã nguồn 39
4.1.2 Cài đặt môi trường hạ tầng và cơ sở dữ liệu 39
4.1.3 Triển khai thực nghiệm hệ thống 40
4.2 Giao diện các phân hệ chính của hệ thống 41
4.2.1 Giao diện người dùng các phân hệ chức năng 42
4.3 Kết quả kiểm thử tự động qua bốn cổng xác minh 47
4.3.1 Xây dựng kịch bản kiểm thử tự động hệ thống qua 4 cổng xác minh 47
4.4 Đánh giá thực nghiệm định lượng và hiệu năng hệ thống 48
4.4.1 Đánh giá số liệu thực nghiệm định lượng và đối chiếu Langfuse Cloud 48
4.4.2 Hệ thống giám sát vận hành và sử dụng tài nguyên 51
4.4.3 Đánh giá tổng kết kết quả thực nghiệm 53
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 55
TÀI LIỆU THAM KHẢO 57

DANH MỤC HÌNH ẢNH

Hình 2-1 Cơ chế tìm kiếm lai (Hybrid Search) kết hợp thuật toán hợp nhất xếp hạng đảo ngược (RRF) 11
Hình 2-2 Mô hình kiến trúc tạo sinh tăng cường truy xuất nâng cao (Advanced RAG) 12
Hình 2-3 Cấu trúc đồ thị tri thức học thuật (Academic Knowledge Graph) và cơ chế biên dịch trang bách khoa (Graph Wiki)..... 14
Hình 2-4 Cơ chế tác nhân ReAct điều phối môi trường E2B MicroVM Sandbox tự động phục hồi lỗi cú pháp LaTeX 17

Hình 3-1 Mô hình đối chiếu chuyển đổi quy trình nghiên cứu khoa học AS-IS và TO-BE 21
Hình 3-2 Kiến trúc Lục giác phân tầng và mô hình 4 Không gian nghiên cứu của hệ thống VeriScholar..... 23
Hình 3-3 Biểu đồ Use Case tổng quan của hệ thống VeriScholar..... 26
Hình 3-4 Biểu đồ ca sử dụng con "Tra cứu và tải tài liệu nghiên cứu" 27
Hình 3-5 Biểu đồ ca sử dụng con "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"..... 28
Hình 3-6 Biểu đồ ca sử dụng con "Khai phá Đồ thị tri thức và biên soạn tri thức học thuật"..... 29
Hình 3-7 Biểu đồ ca sử dụng con "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"..... 30
Hình 3-8 Biểu đồ tuần tự Use Case Tra cứu và tải tài liệu nghiên cứu..... 31
Hình 3-9 Biểu đồ tuần tự Use Case Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ..... 32
Hình 3-10 Biểu đồ tuần tự Use Case Khai phá Đồ thị tri thức và truy vấn liên tài liệu 32
Hình 3-11 Biểu đồ tuần tự Use Case Biên soạn bản thảo và tự phục hồi lỗi LaTeX 32
Hình 3-12 Biểu đồ lớp (Class Diagram) mô tả cấu trúc các thực thể miền và quan hệ nghiệp vụ 33

Hình 4-1 Mô hình kiến trúc không gian tương tác người–máy bốn phân hệ VeriScholar 42

Hình 4-2 Giao diện tìm kiếm và khám phá tài liệu học thuật..... 43
Hình 4-3 Giao diện quản lý thư viện nghiên cứu cá nhân..... 44
Hình 4-4 Giao diện đọc sâu tài liệu PDF hai cột và Bounding Box viền vàng 45
Hình 4-5 Giao diện mạng đồ thị tri thức học thuật 4 thực thể GraphRAG 45
Hình 4-6 Giao diện kho khóa API cá nhân (BYOK) và hồ sơ học giả..... 46
Hình 4-7 Giao diện soạn thảo bản thảo bài báo LaTeX và tự sửa lỗi biên dịch 47
Hình 4-8 Nhật ký kiểm chứng và giám sát vết thực thi mô hình trên Langfuse Cloud..... 51
Hình 4-9 Báo cáo giám sát lượng token luân chuyển và chi phí API OpenAI (lũy kế chu kỳ tháng 10/2026)..... 52

DANH MỤC BẢNG BIỂU

Bảng 1-1 Đối chiếu năng lực kỹ thuật giữa các công cụ hỗ trợ nghiên cứu học thuật hiện có và hệ thống VeriScholar 3
Bảng 2-1 So sánh đa tiêu chí giữa PostgreSQL pgvector và cơ sở dữ liệu Vector chuyên dụng 8
Bảng 2-2 So sánh đặc tính kỹ thuật giữa không gian Dense Vector và Sparse Search (BM25) 10
Bảng 3-1 Các Use Case chính đối với mỗi Actor trong hệ thống VeriScholar 26
Bảng 3-2 Đặc tả Use Case "Tra cứu và tải tài liệu nghiên cứu" 27
Bảng 3-3 Đặc tả Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ" 28
Bảng 3-4 Đặc tả Use Case "Khai phá Đồ thị tri thức và biên soạn tri thức học thuật" 29
Bảng 3-5 Đặc tả Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX". 30
Bảng 3-6 Phân tích các lớp thực thể miền và mối quan hệ cấu trúc trong hệ thống 34
Bảng 3-7 Đặc tả bảng documents 35
Bảng 3-8 Đặc tả bảng document_chunks 36
Bảng 3-9 Đặc tả bảng graph_nodes 37
Bảng 3-10 Đặc tả bảng drafts 37
Bảng 4-1 Cấu trúc phân vùng kho mã nguồn Monorepo và quy tắc ranh giới kiến trúc 40
Bảng 4-2 Kết quả kiểm thử tự động hệ thống qua 4 cổng xác minh 48
Bảng 4-3 Kết quả thực nghiệm đối chứng (Ablation Study) giữa các phương pháp truy xuất trên 36 truy vấn chuẩn 49
Bảng 4-4 Kết quả thực nghiệm đối chứng (Ablation Study) các cơ chế tự sửa lỗi biên dịch LaTeX trên 25 kịch bản lỗi 50

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

STT	Ký hiệu / viết tắt	Ý nghĩa
1	ACID	Atomicity, Consistency, Isolation, Durability – Bốn thuộc tính toàn vẹn giao dịch cơ sở dữ liệu
2	AES-GCM	Advanced Encryption Standard – Galois/Counter Mode – Chuẩn mã hóa khối nâng cao xác thực khóa API BYOK
3	API	Application Programming Interface – Giao diện lập trình ứng dụng
4	ASGI	Asynchronous Server Gateway Interface – Giao diện cổng máy chủ bất đồng bộ Python
5	BBox	Bounding Box – Khung hộp chữ nhật định vị tọa độ
6	BM25	Best Matching 25 – Thuật toán xếp hạng tìm kiếm từ khóa toàn văn
7	BYOK	Bring Your Own Key – Mô hình quản lý khóa bảo mật do người dùng tự cung cấp
8	CAS	Content-Addressable Storage – Hệ thống lưu trữ định danh theo nội dung băm
9	CI/CD	Continuous Integration / Continuous Deployment – Tích hợp và Triển khai liên tục
10	CRAG	Corrective Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất có cơ chế hiệu chỉnh
11	CSDL	Cơ sở dữ liệu – Hệ thống lưu trữ và quản trị dữ liệu có cấu trúc
12	CTE	Common Table Expression – Bảng biểu thức chung trong truy vấn SQL đệ quy
13	DOI	Digital Object Identifier – Định danh đối tượng kỹ thuật số bài báo khoa học
14	E2B	E2B Sandbox / MicroVM – Môi trường máy ảo đám mây thực thi và biên dịch cô lập an toàn
15	FTS	Full-Text Search – Tìm kiếm toàn văn cơ sở dữ liệu
16	GIN	Generalized Inverted Index – Chỉ mục đảo tổng quát trong PostgreSQL

STT	Ký hiệu / viết tắt	Ý nghĩa
17	GraphRAG	Graph Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất qua đồ thị
18	HNSW	Hierarchical Navigable Small World – Cấu trúc đồ thị chỉ mục tìm kiếm vector gần đúng
19	IoU	Intersection over Union – Chỉ số trùng khớp diện tích hộp tọa độ
20	JWT	JSON Web Token – Chuỗi mã định danh phiên người dùng an toàn
21	LLM	Large Language Model – Mô hình ngôn ngữ lớn
22	ORCID	Open Researcher and Contributor ID – Mã định danh học giả quốc tế 16 số
23	RAG	Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất thông tin
24	ReAct	Reason + Act – Chu trình AI: Suy luận và Thực thi hành động
25	REST	Representational State Transfer – Kiến trúc dịch vụ web truyền trạng thái đại diện
26	RRF	Reciprocal Rank Fusion – Thuật toán hợp nhất xếp hạng đảo ngược
27	SHA-256	Secure Hash Algorithm 256-bit – Thuật toán băm mật mã an toàn 256 bit
28	SQL	Structured Query Language – Ngôn ngữ truy vấn cơ sở dữ liệu quan hệ
29	SSE	Server-Sent Events – Giao thức truyền sự kiện thời gian thực từ máy chủ
30	TDD	Test-Driven Development – Phương pháp phát triển hướng kiểm thử
31	TTFT	Time To First Token – Thời gian đáp ứng từ lúc gửi câu hỏi đến token đầu
32	UI/UX	User Interface / User Experience – Giao diện và trải nghiệm người dùng

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

Chương 1 trình bày tổng quan về bối cảnh xuất phát, tính cấp thiết học thuật và mục tiêu nghiên cứu của đề tài xây dựng nền tảng trợ lý AI hỗ trợ nghiên cứu khoa học VeriScholar. Nội dung chương tập trung phân tích các rào cản kỹ thuật của các công cụ hỗ trợ nghiên cứu hiện nay, làm rõ khoảng trống công nghệ thông qua bảng đối chuẩn tính năng, từ đó xác lập hệ thống mục tiêu kỹ thuật định lượng gắn liền với bốn không gian làm việc học thuật khép kín (Khám phá, Đọc sâu, Đồ thị tri thức và Soạn thảo bản thảo). Đồng thời, chương cũng xác định rõ đối tượng, phạm vi nghiên cứu và bố cục tổng thể của báo cáo đồ án tốt nghiệp.

1.1 Giới thiệu đề tài

Đề tài tốt nghiệp *"Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn"* được nghiên cứu và hiện thực hóa thông qua hệ thống phần mềm VeriScholar.

Mục tiêu trọng tâm của đề tài là xây dựng một nền tảng trợ lý trí tuệ nhân tạo hỗ trợ quy trình nghiên cứu khoa học lấy môi trường đại học định hướng nghiên cứu quốc tế như VinUniversity làm bối cảnh đối chuẩn điển hình, đồng thời phục vụ rộng rãi đội ngũ giảng viên, nghiên cứu sinh, học viên cao học và sinh viên đại học. Hệ thống liên kết xuyên suốt bốn không gian làm việc trong vòng đời của một công trình khoa học: từ khâu tìm kiếm khám phá ấn phẩm đa nguồn, phân tích đọc sâu tài liệu chuyên ngành hai cột, tổng hợp mạng lưới tri thức học thuật (GraphRAG [7]), cho đến khâu soạn thảo và biên dịch bản thảo chuẩn quốc tế bằng LaTeX.

Hệ thống VeriScholar tập trung giải quyết bài toán giảm thiểu hiện tượng ảo giác trích dẫn trong nghiên cứu học thuật bằng cách kết hợp kiến trúc tác nhân tạo sinh tăng cường truy xuất (Agentic RAG – *Retrieval-Augmented Generation*) với hai tầng bảo chứng bổ trợ lẫn nhau: thứ nhất là đối chiếu tọa độ trực quan đoạn văn nguồn (*visual citation grounding*) thông qua việc ánh xạ từng thẻ trích dẫn về đúng hộp tọa độ chuẩn hóa trên trang PDF gốc để học giả trực tiếp thẩm định bằng mắt thường; thứ hai là hậu kiểm tính bám sát bằng chứng (*semantic and lexical claim verification*) nhằm từ chối sinh câu trả lời khi ngữ cảnh thu hồi không đủ độ tin cậy, qua đó hạn chế rủi ro dẫn chiếu sai lệch trong các ấn phẩm khoa học.

1.2 Tính cấp thiết của đề tài

Trong kỷ nguyên số và bùng nổ thông tin hiện nay, số lượng tài liệu khoa học trên toàn cầu tăng trưởng với tốc độ chưa từng có. Mỗi tuần có hàng nghìn bài báo mới xuất hiện trên các cơ sở dữ liệu học thuật quốc tế uy tín như arXiv, Semantic Scholar, IEEE Xplore hay ACM Digital Library. Việc tiếp cận, đọc hiểu, phân loại và tổng hợp tri thức từ kho tài liệu khổng lồ này đặt ra áp lực quá tải thông tin rất lớn đối với các nhà nghiên cứu và sinh viên đại học.

Mặc dù sự xuất hiện của các mô hình ngôn ngữ lớn (LLM) cùng kiến trúc tạo sinh tăng cường truy xuất (RAG [10]) mở ra tiềm năng lớn trong việc giảm thiểu hiện tượng suy diễn thiếu căn cứ của mô hình tham số thuần túy, việc ứng dụng các

công cụ AI phổ quát hiện nay vào quy trình nghiên cứu khoa học chuyên sâu vẫn đang gặp phải bốn rào cản nghiêm trọng:

- Ảo giác trích dẫn và bịa đặt học thuật (*Citation Hallucination*): Các mô hình AI tạo sinh thông thường có xu hướng tự sáng tác ra tên tác giả, số định danh DOI, tên tạp chí và các số liệu thực nghiệm không có thật để trả lời câu hỏi của người dùng. Trong nghiên cứu khoa học, việc trích dẫn một nguồn tài liệu không tồn tại hoặc gán ghép sai nội dung nguồn là vi phạm nghiêm trọng chuẩn mực liêm chính học thuật và đạo đức nghiên cứu.
- Thiếu cơ chế kiểm chứng tọa độ trực quan trên tài liệu gốc: Phần lớn các công cụ hỗ trợ đọc tài liệu hiện nay (như ChatPDF thông thường) sử dụng phương pháp trích xuất văn bản thô, làm mất cấu trúc hai cột đặc trưng của các bài báo khoa học, loại bỏ các công thức toán học và chỉ trích dẫn số trang chung chung. Người nghiên cứu không thể xác minh được khẳng định của AI nằm chính xác tại dòng chữ nào, đoạn văn nào trên trang PDF gốc.
- Sự phân mảnh của hệ sinh thái công cụ hỗ trợ nghiên cứu: Quy trình nghiên cứu hiện nay bị chia cắt giữa nhiều phần mềm độc lập: học giả phải dùng Zotero / Mendeley để quản lý danh mục tài liệu, dùng Connected Papers / Elicit để khám phá mạng lưới bài báo, dùng ChatPDF / NotebookLM để tra cứu nhanh, và chuyển sang Overleaf để soạn thảo bản thảo LaTeX. Sự thiếu liên kết này làm đứt gãy mạch tư duy và tiêu tốn đáng kể thời gian chuyển đổi ngữ cảnh của nhà nghiên cứu.
- Rào cản kỹ thuật phức tạp khi biên soạn tài liệu khoa học bằng LaTeX: Để công bố công trình trên các hội nghị và tạp chí quốc tế hàng đầu, bản thảo bắt buộc phải tuân thủ nghiêm ngặt các khuôn mẫu (template) định dạng của IEEE Tran, ACM SIG hay Springer. Khi xảy ra lỗi biên dịch cú pháp trong LaTeX, học giả thường mất nhiều thời gian để đọc các tệp nhật ký (.log) dài hàng trăm dòng nhằm dò tìm lỗi thủ công, làm giảm hiệu suất hoàn thiện bài báo.

Để làm rõ khoảng trống công nghệ trên thực tế, Bảng 1-1 tổng hợp và đối chiếu năng lực kỹ thuật giữa các nền tảng hỗ trợ nghiên cứu học thuật tiêu biểu hiện nay với hệ thống VeriScholar theo năm tiêu chí cốt lõi.

Những khoảng trống công nghệ trên Bảng 1-1 khẳng định tính cấp thiết của việc xây dựng nền tảng trợ lý AI chuyên sâu, kết hợp kiểm chứng tọa độ trực quan trên tài liệu gốc, đồ thị tri thức liên kết và cơ chế tự phục hồi lỗi bản thảo [2, 18], làm cơ sở xác lập hệ thống mục tiêu nghiên cứu của đề tài.

Bảng 1-1: *Đối chiếu năng lực kỹ thuật giữa các công cụ hỗ trợ nghiên cứu học thuật hiện có và hệ thống VeriScholar*

Nền tảng	Định vị tọa độ PDF 2 cột	Truy xuất lai RRF và Cổng kiểm định CRAG	Đồ thị tri thức 4 thực thể và Wiki	Tự sửa lỗi biên dịch LaTeX (ReAct)	Quản lý khóa BYOK AES-256-GCM
ChatPDF	Chỉ dẫn chiếu mức số trang, mất cấu trúc 2 cột	Chỉ dùng tìm kiếm vector đơn luồng	Không hỗ trợ	Không hỗ trợ soạn thảo LaTeX	Không hỗ trợ khóa riêng
NotebookLM	Trích dẫn đoạn văn bản phẳng, không tô sáng hộp tọa độ PDF	Truy xuất ngữ cảnh dài độc quyền (đóng)	Bản đồ khái niệm tóm tắt cơ bản	Không hỗ trợ biên dịch LaTeX	Không hỗ trợ khóa riêng
Elicit	Trích xuất bảng tóm tắt, thiếu khung tọa độ hình học trên trang	Tìm kiếm ngữ nghĩa kết hợp lọc siêu dữ liệu	Ma trận khái niệm theo cột, không có đồ thị liên kết	Không hỗ trợ soạn thảo LaTeX	Không hỗ trợ khóa riêng
SciSpace	Hỗ trợ quét vùng chọn thủ công trên trang PDF	Tìm kiếm lai cơ bản trên kho công khai	Liên kết trích dẫn bài báo, chưa tách 4 thực thể	Có mẫu văn bản, không tự sửa lỗi biên dịch	Không hỗ trợ khóa riêng
Overleaf	Không tích hợp trình đọc hỏi đáp tài liệu RAG	Không có phân hệ truy xuất tri thức học thuật	Không hỗ trợ đồ thị tri thức	Báo lỗi dòng nhật ký thô, gợi ý thủ công	Không hỗ trợ BYOK
VeriScholar (Đề tài)	Ánh xạ tọa độ chuẩn hóa trên PDF 2 cột (IoU $\geq 0{,}85$, thực tế $0{,}88$)	Hybrid RRF ($k = 60$) kết hợp cổng kiểm chứng CRAG và FlashRank	Đồ thị 4 thực thể trên PostgreSQL và Wiki cộng đồng	Agentic ReAct tự sửa lỗi trong E2B Sandbox (96,0%)	Mã hóa AES-256-GCM, giải mã tạm thời trên RAM

1.3 Mục tiêu nghiên cứu

1.3.1 Mục tiêu tổng quát

Xây dựng thành công nền tảng phần mềm VeriScholar đóng vai trò trợ lý AI hỗ trợ nghiên cứu khoa học và biên soạn bài báo quốc tế theo chuẩn mực đại học nghiên cứu như VinUniversity; tích hợp khép kín 4 không gian học thuật trên nền kiến trúc phần mềm lục giác hiện đại [4]; bảo đảm tính toàn vẹn dữ liệu, khả năng đối chiếu trích dẫn trực quan trên bản gốc PDF, tự động hóa sửa lỗi biên dịch LaTeX và bảo vệ an toàn quyền riêng tư học giả.

1.3.2 Mục tiêu kỹ thuật cụ thể

Đề tài hướng tới hoàn thành 5 mục tiêu kỹ thuật định lượng cụ thể:

- Hiện thực hóa cơ chế nạp bài báo hai tầng kết hợp kho lưu trữ định danh theo nội dung CAS (*Content-Addressable Storage*) dựa trên mã băm SHA-256 nhằm loại bỏ việc lưu trữ trùng lặp các tệp nhị phân PDF và quản lý an toàn vòng đời tài liệu theo cơ chế đếm tham chiếu (ref_count).

- Phát triển công cụ tìm kiếm lai Agentic Hybrid RAG kết hợp thuật toán hợp nhất thứ hạng RRF (*Reciprocal Rank Fusion* [5], $k = 60$) giữa tìm kiếm vector ngữ nghĩa và từ khóa toàn văn, đi kèm cơ chế đối chiếu khung tọa độ chuẩn hóa trên PDF gốc đạt ngưỡng trùng khớp IoU $\geq 0{,}85$.
- Xây dựng đồ thị tri thức học thuật 4 thực thể (GraphRAG [7]: *Paper*, *Method*, *Dataset*, *MetricResult*) và hệ thống trang bách khoa tri thức (Graph Wiki) trực tiếp trên PostgreSQL 16 với tiện ích mở rộng pgvector, bảo đảm toàn vẹn giao dịch ACID.
- Tích hợp tác nhân AI dựa trên máy trạng thái LangGraph ReAct [19] trong môi trường cô lập E2B Sandbox MicroVM [1], có khả năng tự động phân tích và khắc phục lỗi biên dịch LaTeX với tỷ lệ tự phục hồi thành công trên 90%.
- Thiết lập cơ chế quản lý khóa người dùng tự cấp theo mô hình Bring Your Own Key (BYOK – *Bring Your Own Key*) mã hóa bằng thuật toán AES-256-GCM [6], bảo đảm tuyệt đối không lưu trữ khóa API ở dạng bản rõ và chỉ giải mã tạm thời trong bộ nhớ RAM khi tác vụ đang thực thi.

1.4 Nội dung nghiên cứu và phương pháp tiếp cận

1.4.1 Nội dung nghiên cứu

Đề tài tập trung nghiên cứu và giải quyết các nội dung khoa học trọng tâm:

- Kiến trúc phần mềm phân tầng: Nghiên cứu nguyên lý kiến trúc lục giác (*Hexagonal Architecture / Ports and Adapters* [4]) nhằm cô lập lõi miền nghiệp vụ học thuật thuần khiết khỏi hạ tầng cơ sở dữ liệu, dịch vụ đám mây và giao vận web.
- Truy xuất thông tin lai (*Hybrid Search*): Nghiên cứu các thuật toán tìm kiếm vector gần đúng kết hợp thuật toán xếp hạng từ khóa toàn văn qua giải thuật hợp nhất thứ hạng RRF [5] và cổng kiểm định CRAG [18].
- Xử lý tài liệu và tọa độ trực quan: Nghiên cứu kỹ thuật trích xuất tài liệu PDF khoa học hai cột và thuật toán chuẩn hóa khung tọa độ Bounding Box $[x_0, y_0, x_1, y_1]$ trên dải giá trị chuẩn $[0..1000]$.
- Đồ thị tri thức học thuật (*GraphRAG*): Nghiên cứu mô hình biểu diễn mạng lưới tri thức 4 thực thể khoa học cốt lõi, thuật toán phân cụm cộng đồng Louvain và cơ chế tổng hợp trang tri thức Graph Wiki [7].
- Tác nhân thông minh tự phục hồi: Nghiên cứu chu trình tác nhân ReAct [19] (Suy luận – Hành động – Quan sát) trong việc tự động phân tích và khắc phục lỗi cú pháp biên dịch tài liệu học thuật trong môi trường Sandbox an toàn.
- Bảo mật và quyền riêng tư: Nghiên cứu cơ chế bảo mật kho khóa cá nhân (BYOK – *Bring Your Own Key*) mã hóa khóa API bằng thuật toán AES-256-GCM [6], không lưu trữ khóa bản rõ và chỉ giải mã tạm thời trong bộ nhớ tiến trình.

1.4.2 Phương pháp tiếp cận

Đề tài áp dụng phương pháp luận phát triển hướng kiểm thử (TDD – *Test-Driven Development*) kết hợp cấu trúc Monorepo nhằm đảm bảo mọi quy tắc nghiệp vụ

đều được kiểm chứng độc lập với hạ tầng kỹ thuật, duy trì tỷ lệ kiểm thử thành công 100% thông qua hệ thống kiểm thử tự động toàn diện từ kiểm định tĩnh mã nguồn, kiểm thử đơn vị, kiểm thử tích hợp đến kiểm thử hồi quy kiến trúc.

1.5 Phạm vi và đối tượng nghiên cứu

Đề tài xác định rõ đối tượng và phạm vi nghiên cứu:

- Đối tượng nghiên cứu: Quy trình nghiên cứu khoa học, phân tích bố cục tài liệu chuyên sâu, trích xuất mạng lưới tri thức học thuật và kỹ thuật biên dịch bản thảo khoa học chuẩn quốc tế.
- Khách thể thụ hưởng và bối cảnh đối chuẩn: Lấy môi trường xuất bản quốc tế tại Trường Đại học VinUni làm bối cảnh đại học nghiên cứu đối chuẩn điển hình; phục vụ giảng viên, nghiên cứu sinh, học viên cao học và sinh viên các cơ sở giáo dục định hướng nghiên cứu.
- Định dạng tài liệu và ngôn ngữ đánh giá: Phân hệ phân tích bố cục, truy xuất lai (Hybrid RAG) và đồ thị tri thức (GraphRAG [7]) tập trung đánh giá trên ấn phẩm khoa học chuẩn PDF 1–2 cột tiếng Anh (IEEE, ACM, NeurIPS, ICLR); phân hệ soạn thảo LaTeX và giao diện hỗ trợ đầy đủ Unicode tiếng Việt, tiếng Anh và trích dẫn BibTeX.
- Hạ tầng kỹ thuật triển khai: Máy chủ Linux (Ubuntu 24.04 LTS trên Microsoft Azure), PostgreSQL 16 tích hợp pgvector, Redis 7, container Docker, môi trường Sandbox E2B [1] và nền tảng giám sát LLM Langfuse.

1.6 Bố cục báo cáo đồ án

Báo cáo đồ án tốt nghiệp được cấu trúc thành 4 chương chính và phần kết luận:

- Chương 1 (Tổng quan về đề tài) – Trình bày bối cảnh, tính cấp thiết học thuật kèm bảng đối chuẩn tính năng, mục tiêu nghiên cứu định lượng, nội dung, phương pháp tiếp cận và phạm vi nghiên cứu.
- Chương 2 (Cơ sở lý thuyết và công nghệ) – Trình bày cơ sở lý thuyết và các công nghệ nền tảng: Hệ quản trị cơ sở dữ liệu quan hệ và mở rộng vector (PostgreSQL, pgvector, Redis); Kỹ nghệ truy xuất thông tin hỗn hợp (Hybrid Search, Bounding Box); Mô hình ngôn ngữ lớn và tạo sinh tăng cường truy xuất nâng cao (Self-RAG, CRAG); Đồ thị tri thức học thuật (GraphRAG, Graph Wiki); Kiến trúc tác nhân tự chủ và môi trường Sandbox an toàn (ReAct, LangGraph, E2B, BYOK, Langfuse).
- Chương 3 (Phân tích và thiết kế hệ thống) – Trình bày khảo sát nghiệp vụ học thuật, thiết kế Kiến trúc Lục giác phân tầng và mô hình 4 không gian nghiên cứu, đặc tả các sơ đồ Use Case, biểu đồ tuần tự UML và thiết kế 20 bảng cơ sở dữ liệu quan hệ kèm lược đồ ERD.
- Chương 4 (Cài đặt và kết quả thực nghiệm) – Trình bày cấu hình môi trường triển khai, giao diện 4 không gian tương tác, bộ 580 kịch bản kiểm thử tự động, kết quả thực nghiệm đối chứng (Ablation Study) về tọa độ Bounding Box và khả năng tự phục hồi bản thảo LaTeX.
- Kết luận và hướng phát triển – Tổng kết các đóng góp kỹ thuật cốt lõi, đánh giá khách quan hạn chế và đề xuất ba hướng nghiên cứu phát triển mở rộng.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Cơ sở dữ liệu quan hệ và mở rộng Vector

2.1.1 Hệ quản trị cơ sở dữ liệu quan hệ đối tượng PostgreSQL

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ đối tượng mã nguồn mở (*Object-Relational Database Management System – ORDBMS*), nổi bật với độ bền vững dữ liệu, khả năng mở rộng linh hoạt và cơ chế giao dịch đạt chuẩn ACID (*Atomicity, Consistency, Isolation, Durability*) tuyệt đối.

Các đặc điểm kỹ thuật then chốt của PostgreSQL bao gồm:

- Cơ chế kiểm soát tương tranh đa phiên bản (MVCC): Cho phép các thao tác đọc không chặn ghi và ghi không chặn đọc thông qua cơ chế cô lập ảnh chụp (*Snapshot Isolation*), duy trì thông lượng truy vấn cao ngay cả khi nhiều học giả cùng truy cập đồng thời.
- Hỗ trợ dữ liệu bán cấu trúc (JSONB): Lưu trữ và đánh chỉ mục nhị phân các cấu trúc dữ liệu JSON phức tạp với hiệu suất truy vấn tương đương các trường dữ liệu quan hệ thông thường.
- Kiến trúc mở rộng mạnh mẽ (Extensibility): Cho phép tích hợp các kiểu dữ liệu mới, hàm người dùng tự định nghĩa và các tiện ích mở rộng (*extensions*) cấp thấp chạy trực tiếp trong nhân cơ sở dữ liệu.

Trong đề tài VeriScholar, PostgreSQL giữ vai trò là xương sống lưu trữ dữ liệu trung tâm, quản lý toàn bộ thực thể học thuật, lịch sử phiên nghiên cứu, siêu dữ liệu phân đoạn tài liệu và bản đồ băm định danh nội dung (CAS).

2.1.2 Tiện ích mở rộng pgvector và cấu trúc chỉ mục HNSW

pgvector là một tiện ích mở rộng mã nguồn mở dành riêng cho PostgreSQL, bổ sung kiểu dữ liệu vector và các phép toán đo lường khoảng cách không gian đa chiều (khoảng cách Euclid L_2, khoảng cách Cosine, và tích vô hướng Inner Product). Nhờ pgvector, PostgreSQL hoạt động như một hệ quản trị cơ sở dữ liệu tích hợp đa mô hình quan hệ – Vector (*Integrated Relational-Vector DBMS*), cho phép lưu trữ và truy vấn trực tiếp các Vector Embedding ngữ nghĩa 1536 chiều ngay bên cạnh dữ liệu quan hệ chuẩn ACID.

Điểm đột phá trong việc tối ưu hóa tốc độ tìm kiếm láng giềng gần nhất xấp xỉ (*Approximate Nearest Neighbor – ANN*) của pgvector là việc hiện thực hóa thuật toán chỉ mục HNSW (*Hierarchical Navigable Small World*) [11]:

- Nguyên lý phân tầng đa lớp: HNSW tổ chức không gian Vector thành cấu trúc đồ thị phân tầng tương tự như danh sách liên kết bước nhảy (*Skip List*). Các tầng trên cùng chứa các liên kết xa với mật độ thưa giúp định vị nhanh vùng lân cận tổng quát; các tầng dưới có mật độ nút dày đặc dần cho phép thu hẹp chính xác không gian tìm kiếm.

- Độ phức tạp truy vấn kỳ vọng thực nghiệm $O(\log N)$: Thay vì phải tính toán khoảng cách tuần tự trên toàn bộ kho vector (*Flat Search* với độ phức tạp $O(N)$), thuật toán định tuyến tham lam (*Greedy Routing*) trên đồ thị thế giới nhỏ phân tầng của HNSW đạt độ phức tạp kỳ vọng thực nghiệm tỷ lệ logarit $O(\log N)$ theo số lượng đỉnh N trên các tập dữ liệu thực tế [11].

- Cập nhật linh hoạt và kiểm soát độ thu hồi theo siêu tham số: Khác với thuật toán IVFFlat đòi hỏi phải huấn luyện lại tâm cụm khi phân phối dữ liệu biến động lớn, HNSW cho phép chèn mới Vector trực tuyến. Độ thu hồi (*Recall*) và tốc độ truy vấn của chỉ mục được điều chỉnh linh hoạt thông qua ba siêu tham số cốt lõi: số liên kết hai chiều tối đa trên mỗi nút (m), kích thước danh sách ứng viên động khi dựng đồ thị (ef_construction) và kích thước danh sách ứng viên khi tìm kiếm (ef_search).

Việc hợp nhất dữ liệu quan hệ và dữ liệu Vector Embedding trong cùng một hệ quản trị PostgreSQL giúp loại bỏ hoàn toàn sự cồng kềnh của kiến trúc đa cơ sở dữ liệu, đảm bảo tính nhất quán giao dịch ACID trong suốt quá trình người dùng lưu trữ hoặc xóa tài liệu.

2.1.3 Hệ thống đệm dữ liệu bộ nhớ truy cập nhanh Redis

Redis là hệ thống lưu trữ cấu trúc dữ liệu trong bộ nhớ RAM (*In-Memory Data Store*) mã nguồn mở với hiệu năng cao, đóng vai trò bộ nhớ đệm tốc độ cao trong hệ thống.

Trong kiến trúc VeriScholar, Redis đảm nhiệm ba vai trò chính: bộ đệm danh sách đen Token (*Token Blacklist*) thu hồi phiên JWT khi đăng xuất, kiểm soát giới hạn tần suất yêu cầu (*Rate Limiting*) bảo vệ cổng suy luận AI, và lưu tạm kết quả định danh học thuật (*DOI Cache*) giảm tải cho mạng ngoài.

2.1.4 Luận giải lựa chọn cơ sở dữ liệu Vector tích hợp

Một câu hỏi mang tính then chốt trong thiết kế hệ thống là: *Tại sao VeriScholar lựa chọn giải pháp PostgreSQL tích hợp tiện ích pgvector thay vì sử dụng các cơ sở dữ liệu Vector chuyên dụng như Milvus, Pinecone hay Qdrant?* Bảng 2-1 phân tích đối chứng chi tiết giữa hai cách tiếp cận trên các tiêu chí kỹ thuật cốt lõi.

Qua phân tích đối chứng, việc lựa chọn PostgreSQL kết hợp pgvector là quyết định kỹ thuật chuẩn mực, giúp VeriScholar vừa bảo đảm tính toàn vẹn dữ liệu học thuật tuyệt đối, vừa tối ưu hóa hiệu năng và chi phí hạ tầng trên môi trường máy chủ Azure.

Bảng 2-1: *So sánh đa tiêu chí giữa PostgreSQL pgvector và cơ sở dữ liệu Vector chuyên dụng*

Tiêu chí	PostgreSQL + pgvector (Đề tài chọn)	Cơ sở dữ liệu Vector chuyên dụng (Milvus, Pinecone, Qdrant)
Giao dịch ACID	Tuyệt đối: Cập nhật bản ghi quan hệ và Vector nhúng đồng thời trong một giao dịch duy nhất.	Nhất quán cuối cùng: Rủi ro dữ liệu mồ côi hoặc bất đồng bộ giữa hai hệ thống lưu trữ.
Kiến trúc vận hành	Tối giản: Vận hành một cụm duy nhất, không phụ thuộc cụm dịch vụ trung gian (etcd, MinIO).	Phức tạp: Đòi hỏi cụm phân tán riêng biệt kèm hạ tầng điều phối tốn kém tài nguyên.
Truy vấn kết hợp	Linh hoạt: Kết hợp trực tiếp bộ lọc quan hệ (WHERE) và tìm kiếm Vector trong cùng một câu lệnh SQL.	Phân mảnh: Đòi hỏi cơ chế lọc hai pha tách rời gây phát sinh độ trễ mạng và nén ứng viên.
Sao lưu phục hồi	Chuẩn hóa: Sao lưu toàn diện và đồng bộ qua cơ chế nhật ký WAL và công cụ pg_dump.	Tách rời: Cần quy trình đồng bộ sao lưu riêng lẻ giữa cơ sở dữ liệu nghiệp vụ và kho Vector.
Chi phí hạ tầng	Tiết kiệm: Mã nguồn mở 100%, tối ưu hóa trên cấu hình máy chủ đám mây Azure đơn lẻ.	Tốn kém: Chi phí bản quyền SaaS định kỳ cao hoặc yêu cầu phần cứng RAM/CPU vượt trội.

2.2 Kỹ nghệ truy xuất thông tin hỗn hợp

2.2.1 Phân tích bố cục tài liệu PDF hai cột và chuẩn hóa tọa độ Bounding Box

Các ấn phẩm khoa học quốc tế thường được định dạng theo bố cục hai cột (*Two-Column PDF Layout*). Các công cụ trích xuất văn bản truyền thống thường đọc tuần tự theo dòng ngang, dẫn đến hiện tượng trộn lẫn các câu văn giữa cột trái và cột phải, phá vỡ tính mạch lạc của các luận điểm khoa học.

Để giải quyết vấn đề này, đề tài xây dựng quy trình phân tích bố cục hình học trước khi phân đoạn tài liệu:

- Phân tách ranh giới hình học: Dựa trên phân phối hoành độ tâm của các khối chữ nhật chứa ký tự, thuật toán thiết lập ngưỡng phân định cột tự động để gom nhóm và đọc trọn vẹn nội dung từ đỉnh đến đáy của cột bên trái trước khi chuyển tiếp sang cột bên phải.

- Chuẩn hóa tọa độ Bounding Box (*Bounding Box Normalization*): Tọa độ vật lý của các khối văn bản (x_0, y_0, x_1, y_1) tính theo điểm ảnh PDF (*points*) được chuẩn hóa về miền tỷ lệ không thứ nguyên $[0, 1]$ thông qua phép biến đổi hình học:

$$\bar{x}_0 = \frac{x_0}{W}, \quad \bar{y}_0 = \frac{y_0}{H}, \quad \bar{x}_1 = \frac{x_1}{W}, \quad \bar{y}_1 = \frac{y_1}{H} \tag{2.1}$$

trong đó W và H lần lượt là chiều rộng và chiều cao thực tế của trang tài liệu PDF.

- Thước đo độ chính xác định vị: Độ trùng khớp của Bounding Box dẫn chứng được đánh giá thông qua chỉ số tương đồng diện tích phủ (*Intersection-over-Union – IoU*):

$$\text{IoU}(B_{\text{pred}}, B_{\text{gt}}) = \frac{\text{Area}(B_{\text{pred}} \cap B_{\text{gt}})}{\text{Area}(B_{\text{pred}} \cup B_{\text{gt}})} \tag{2.2}$$

Hệ thống duy trì ngưỡng bảo đảm IoU $\geq 0{,}85$ (kết quả thực nghiệm đạt $0{,}88$), cho phép giao diện hiển thị khung viền dẫn chứng trực quan chính xác từng dòng văn bản gốc cho học giả.

2.2.2 Không gian truy xuất Vector đặc và từ khóa thưa hai pha

Hệ thống truy xuất thông tin học thuật đòi hỏi sự kết hợp đồng thời của hai không gian tìm kiếm bổ trợ:

- Truy xuất Vector đặc (*Dense Vector Retrieval*): Văn bản được biểu diễn thành các Vector Embedding ngữ nghĩa 1536 chiều. Mức độ tương đồng giữa truy vấn **u** của học giả và phân đoạn tài liệu **v** được lượng hóa thông qua hàm tương đồng Cosine:

$$\text{CosineSimilarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \sqrt{\sum_{i=1}^{d} v_i^2}} \tag{2.3}$$

Ưu điểm của phương pháp là khả năng bắt trọn ngữ nghĩa khái quát và từ đồng nghĩa, song gặp khó khăn trước các thuật ngữ hiếm gặp, mã định danh hoặc công thức toán học.

- Truy xuất từ khóa thưa hai pha (*Two-Stage Sparse Keyword Retrieval*): Kết hợp chỉ mục đảo tổng quát GIN trên trường tsvector với hàm xếp hạng mật độ bao phủ ts_rank_cd để sàng lọc nhanh tập ứng viên ban đầu trong cơ sở dữ liệu, sau đó áp dụng công thức chuẩn hóa Okapi BM25 [13] để tái chấm điểm chính xác:

$$\text{Score}_{\text{BM25}}(D, Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \tag{2.4}$$

trong đó $f(q_i, D)$ là tần suất từ khóa trong đoạn trích D, $|D|$ là độ dài đoạn trích, avgdl là độ dài trung bình toàn tập, $k_1 = 1{,}5$ và $b = 0{,}75$. Cơ chế này bắt trúng các từ khóa chuyên ngành, số hiệu định danh và tên riêng tác giả.

2.2.3 So sánh đặc tính kỹ thuật giữa hai không gian truy xuất

Đặc tính kỹ thuật đối chứng giữa hai không gian tìm kiếm được tổng hợp trong Bảng 2-2.

Bảng 2-2: *So sánh đặc tính kỹ thuật giữa không gian Dense Vector và Sparse Search (BM25)*

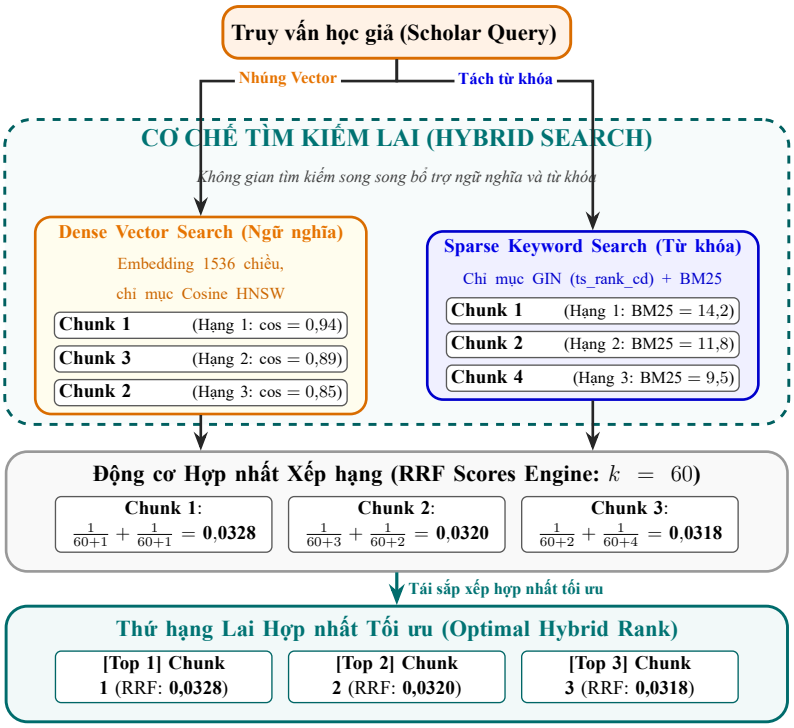
Tiêu chí so sánh	Dense Vector (Cosine HNSW)	Sparse Keyword (BM25 GIN)
Không gian biểu diễn	Vector thực liên tục đa chiều (1536 chiều)	Vector từ khóa thưa rời rạc (tsvector)
Khả năng ngữ nghĩa	Bắt trọn từ đồng nghĩa và khái niệm tương đương	Hạn chế: Dựa trên sự trùng khớp ký tự chính xác
Độ chính xác thuật ngữ	Trung bình: Dễ bị nhiễu trước từ viết tắt và ký hiệu	Xuất sắc: Khớp tuyệt đối mã số DOI, tên riêng, công thức
Cấu trúc chỉ mục	Đồ thị láng giềng phân tầng HNSW [11]	Danh sách nghịch đảo GIN (ts_rank_cd) + BM25

2.2.4 Thuật toán hợp nhất xếp hạng đảo ngược và cơ chế tái xếp hạng

Nhằm kết hợp hài hòa ưu thế của cả hai không gian truy xuất, hệ thống áp dụng thuật toán Hợp nhất Xếp hạng Đảo ngược (*Reciprocal Rank Fusion – RRF*) [5]. Thay vì cộng gộp các điểm số có thang đo phân kỳ, RRF tính toán dựa trên thứ hạng nguyên vị trí của từng tài liệu trong các danh sách trả về:

$$\text{Score}_{\text{RRF}}(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \tag{2.5}$$

trong đó $M = \{\text{Dense Vector}, \text{Sparse BM25}\}$, $r_m(d)$ là thứ bậc của phân đoạn d trong danh sách m ($r_m(d) \geq 1$), và $k = 60$ là hằng số làm mịn chuẩn hóa học thuật nhằm tránh ưu tiên thái quá cho tài liệu đứng đầu danh sách đơn lẻ. Sau pha hợp nhất RRF, danh sách các phân đoạn hội tụ được đưa qua bộ tái xếp hạng Cross-Encoder để chấm điểm tương tác sâu giữa câu hỏi và đoạn trích, bảo đảm thứ tự ưu tiên tối ưu trước khi nạp vào ngữ cảnh mô hình ngôn ngữ lớn (Hình 2-1).



Hình 2-1: *Cơ chế tìm kiếm lai (Hybrid Search) kết hợp thuật toán hợp nhất xếp hạng đảo ngược (RRF)*

2.3 Mô hình ngôn ngữ lớn và tạo sinh tăng cường truy xuất nâng cao

2.3.1 Hạn chế của mô hình ngôn ngữ lớn và nguyên lý RAG

Mô hình ngôn ngữ lớn (*Large Language Model – LLM*) đã tạo nên bước ngoặt trong xử lý ngôn ngữ tự nhiên, song việc ứng dụng trực tiếp vào nghiên cứu khoa học bộc lộ ba điểm nghẽn nghiêm trọng:

- Giới hạn tri thức tĩnh (*Knowledge Cutoff*): Mô hình hoàn toàn không thể cập nhật các khám phá hoặc công bố khoa học mới sau thời điểm hoàn tất tiền huấn luyện.
- Hiện tượng ảo giác học thuật (*Hallucination*): Khi đối mặt với các câu hỏi chuyên sâu, mô hình có xu hướng tự tạo ra các phát biểu, số liệu hoặc trích dẫn nghe có vẻ thuyết phục nhưng hoàn toàn giả mạo.
- Thiếu minh bạch và khả năng kiểm chứng: Các mô hình đóng gói thuần túy không thể chỉ ra chính xác nguồn gốc tài liệu, trang sách hay tọa độ dẫn chứng làm cơ sở cho câu trả lời.

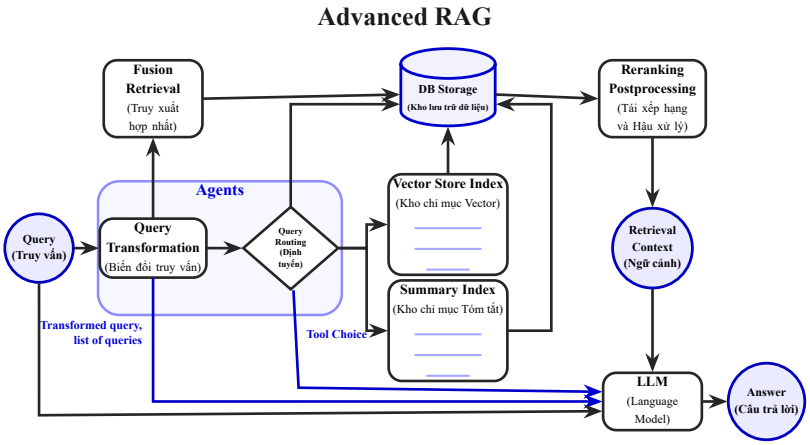
Để giải quyết các thách thức trên, kỹ thuật Tạo sinh tăng cường truy xuất (*Retrieval-Augmented Generation – RAG*) do Lewis và cộng sự giới thiệu [10] đã kết hợp năng lực suy luận ngôn ngữ của LLM với kho tri thức học thuật bên ngoài. Mặc dù kiến trúc RAG truyền thống (*Naive RAG*) đã giải quyết được tính thời sự của thông tin thông qua việc phân đoạn văn bản và tìm kiếm Vector đơn lẻ, cách tiếp cận này thường làm đứt gãy ngữ cảnh của các ấn phẩm nghiên cứu phức tạp và bỏ sót các thuật ngữ khoa học đặc thù.

2.3.2 Kiến trúc RAG nâng cao đa giai đoạn

Khắc phục điểm nghẽn của Naive RAG, kiến trúc RAG Nâng cao (*Advanced RAG*) theo khảo sát của Gao và cộng sự [8] tái cấu trúc toàn diện quy trình xử lý thành ba giai đoạn tinh chỉnh chuyên sâu:

- Tiền truy xuất (*Pre-Retrieval*): Viết lại câu hỏi (*Query Rewriting*), phân rã truy vấn con (*Sub-queries*), phân tích cấu trúc bố cục PDF hai cột và chuẩn hóa tọa độ Bounding Box (*Bounding Box Normalization*) về miền tỷ lệ $[0, 1]$.
- Truy xuất (*Retrieval*): Kết hợp không gian Vector đặc với chỉ mục đảo GIN ts_rank_cd và BM25 để lọc nhanh tập ứng viên tiềm năng có độ phủ từ khóa cao.
- Hậu truy xuất (*Post-Retrieval*): Sử dụng mô hình tái xếp hạng chéo Cross-Encoder chấm điểm tương tác sâu giữa câu hỏi và đoạn trích, kết hợp nén ngữ cảnh chọn lọc và kiểm chứng đối chiếu trích dẫn.

Kiến trúc Advanced RAG phối hợp đa tầng được mô hình hóa trong Hình 2-2.



Hình 2-2: *Mô hình kiến trúc tạo sinh tăng cường truy xuất nâng cao (Advanced RAG)*

2.3.3 Cơ chế phản xạ tự đánh giá Self-RAG

Nhằm khắc phục hiện tượng truy xuất bừa bãi ngay cả khi không cần thiết, đề tài tham chiếu khung kiến trúc Self-RAG (*Self-Reflective Retrieval-Augmented Generation*) do Asai và cộng sự đề xuất [2]. Nguyên lý của Self-RAG là tích hợp các tiêu chí phản xạ đặc biệt (*Reflection Tokens*) vào chu trình xử lý:

- Đánh giá quyết định truy xuất ([Retrieve]): Tự đánh giá xem câu hỏi hiện tại có cần thêm tri thức ngoại vi hay có thể trả lời trực tiếp từ tri thức tham số nội tại.
- Đánh giá mức độ liên quan ([IsRel]): Đánh giá xem từng đoạn văn bản truy xuất được có thực sự liên quan đến câu hỏi hay chứa thông tin gây nhiễu.
- Kiểm tra tính tương hỗ dẫn chứng ([IsSup]): Thẩm định xem từng câu văn tạo sinh có được hỗ trợ đầy đủ và trực tiếp từ ngữ cảnh trích dẫn hay không, loại trừ các suy đoán vô căn cứ.

- Đánh giá độ hữu ích tổng thể ([IsUse]): Đo lường chất lượng và độ hoàn chỉnh của câu trả lời đối với yêu cầu nghiên cứu của học giả.

Trong VeriScholar, nguyên lý tự phản xạ của Self-RAG được hiện thực hóa thông qua các bộ rào chắn quy tắc tất định (*Deterministic Guardrails*) và chu trình kiểm tra trích dẫn chéo, bảo đảm hiệu quả kiểm soát ảo giác tối đa.

2.3.4 Cơ chế hiệu chỉnh truy xuất và kiểm soát ảo giác (Corrective RAG)

Mặc dù hệ thống truy xuất đã qua nhiều tầng tinh lọc, vẫn tồn tại trường hợp toàn bộ các đoạn văn bản tìm được trong kho dữ liệu cục bộ đều không thỏa mãn câu hỏi nghiên cứu. Để giải quyết triệt để vấn đề này, đề tài ứng dụng cơ chế RAG Hiệu chỉnh (*Corrective RAG – CRAG*) do Yan và cộng sự phát triển [18].

CRAG thiết lập một cổng đánh giá độ tin cậy (*Confidence Gate*) để phân loại kết quả truy xuất thành ba kịch bản xử lý linh hoạt:

- *Độ tin cậy cao (Correct):* Khi các đoạn văn bản đạt độ tương đồng vượt ngưỡng tin cậy, thuật toán thực hiện thao tác tinh chỉnh tri thức (*Knowledge Refinement*) – bóc tách các câu chứa thông tin then chốt và loại bỏ triệt để các câu nhiễu phụ trước khi nạp vào LLM.

- *Độ tin cậy thấp (Incorrect):* Khi điểm tin cậy của toàn bộ tập ứng viên nằm dưới ngưỡng an toàn (kho tài liệu nội bộ không chứa câu trả lời), hệ thống tự động kích hoạt cơ chế dự phòng (*Fallback Mechanism*) thông qua việc gọi công cụ tra cứu học thuật bên ngoài để bổ sung nguồn tri thức mới, ngăn chặn việc LLM tự bịa đặt câu trả lời.

- *Độ tin cậy trung gian (Ambiguous):* Hệ thống kết hợp thông minh giữa các đoạn trích dẫn cục bộ chọn lọc với kết quả tra cứu bên ngoài, đồng thời gán nhãn cảnh báo để người dùng chủ động đối chiếu.

Nhờ sự kết hợp giữa Advanced RAG, Self-RAG và CRAG, VeriScholar thiết lập rào chắn ba lớp kiểm soát ảo giác học thuật, bảo đảm tính liêm chính và độ chính xác của mọi kết quả nghiên cứu.

2.4 Đồ thị tri thức học thuật và phân cụm cộng đồng

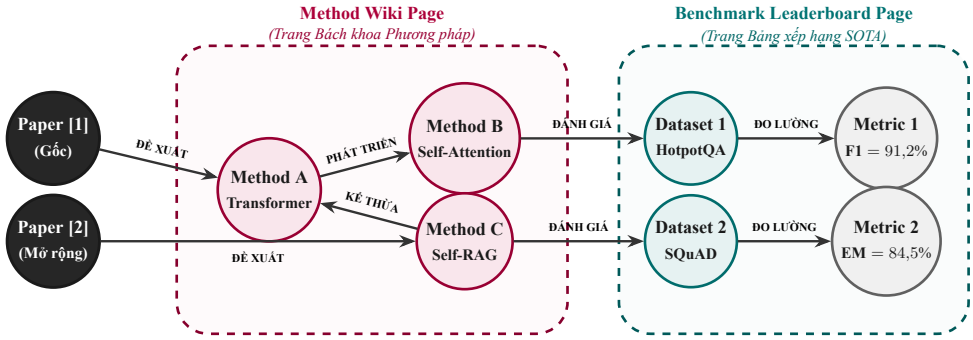
2.4.1 Mô hình biểu diễn mạng lưới tri thức học thuật 4 thực thể

Trong khi mô hình RAG véc-tơ xử lý hiệu quả các truy vấn tìm kiếm đoạn văn cục bộ, phương pháp này bộc lộ hạn chế cố hữu trước các câu hỏi tổng hợp mang tính vĩ mô (chẳng hạn: *"Những phương pháp nào đã được áp dụng trên tập dữ liệu HotpotQA và đạt F1 trên 90%?"*). Khi thông tin phân tán rải rác trên hàng chục công trình khác nhau, việc truy vấn tương đồng véc-tơ đơn lẻ không thể nắm bắt được các mối liên hệ chéo giữa các thực thể học thuật.

Đề tài giải quyết thách thức này bằng cách mô hình hóa tri thức học thuật dưới dạng một đồ thị có hướng có gán nhãn $G = (V, E)$ [9]. Mạng lưới tri thức được chuẩn hóa thành bốn loại thực thể hạt nhân, khớp hoàn toàn với cấu trúc cơ sở dữ liệu quan hệ của hệ thống:

- Đỉnh Bài báo (*Paper Nodes*): Đại diện cho các ấn phẩm nghiên cứu khoa học gốc, lưu trữ tiêu đề, tác giả, năm công bố và mã số định danh DOI.
- Đỉnh Phương pháp (*Method Nodes*): Đại diện cho các mô hình, thuật toán hoặc kỹ thuật đề xuất (ví dụ: *Transformer, Self-Attention* [16], *Self-RAG*).
- Đỉnh Bộ dữ liệu (*Dataset Nodes*): Đại diện cho các tập dữ liệu thực nghiệm chuẩn mực (ví dụ: *HotpotQA, SQuAD, ImageNet*).
- Đỉnh Chỉ số kết quả (*Metric Nodes*): Đại diện cho các kết quả đo lường định lượng cụ thể (chẳng hạn: F1 $= 91{,}2\%$, EM $= 84{,}5\%$).

Các thực thể được liên kết thông qua các quan hệ ngữ nghĩa chuẩn tắc như ĐỀ XUẤT, PHÁT TRIỂN, KẾ THỪA, ĐÁNH GIÁ và ĐO LƯỜNG. Cấu trúc đồ thị tri thức 4 thực thể và cơ chế ánh xạ sang trang bách khoa cộng đồng được mô hình hóa trong Hình 2-3.



Hình 2-3: *Cấu trúc đồ thị tri thức học thuật (Academic Knowledge Graph) và cơ chế biên dịch trang bách khoa (Graph Wiki)*

2.4.2 Thuật toán phân cụm Louvain và hàm mục tiêu Modularity Q

Để phục vụ việc tổng hợp tri thức ở quy mô toàn cục theo mô hình GraphRAG [7], hệ thống áp dụng thuật toán phát hiện cộng đồng Louvain (*Louvain Community Detection Algorithm*) [3] (kèm mở rộng thuật toán Leiden [3]). Mục tiêu của thuật toán là phân hoạch đồ thị thực thể thành các cụm cộng đồng có mật độ liên kết nội bộ dày đặc và mật độ liên kết giữa các cộng đồng thưa thớt, thông qua việc tối đa hóa hàm mục tiêu độ mô-đun (*Modularity* Q):

$$Q = \frac{1}{2m} \sum_{i,j} \left[A_{ij} - \frac{k_i k_j}{2m} \right] \delta(c_i, c_j) \tag{2.6}$$

trong đó:

- A_{ij} là trọng số liên kết giữa hai đỉnh i và j trong đồ thị tri thức;
- $k_i = \sum_j A_{ij}$ và $k_j = \sum_i A_{ji}$ lần lượt là tổng bậc liên kết của đỉnh i và đỉnh j;
- $m = \frac{1}{2} \sum_{i,j} A_{ij}$ là tổng trọng số của tất cả các cạnh trong toàn bộ đồ thị;

- c_i và c_j lần lượt là định danh nhãn cộng đồng được gán cho đỉnh i và đỉnh j;
- $\delta(c_i, c_j)$ là hàm Kronecker delta, quy định $\delta(c_i, c_j) = 1$ nếu đỉnh i và đỉnh j thuộc cùng một cộng đồng ($c_i = c_j$), và bằng 0 trong trường hợp ngược lại.

Thuật toán Louvain vận hành lặp qua hai bước: tối ưu hóa cục bộ hàm Modularity bằng cách dịch chuyển đỉnh sang cộng đồng láng giềng tối ưu, sau đó gộp cộng đồng thành siêu đỉnh và lặp lại cho đến khi hàm mục tiêu Q đạt trạng thái hội tụ.

2.4.3 Cơ chế biên dịch trước trang bách khoa tri thức (Graph Wiki)

Dựa trên các phân cụm cộng đồng hội tụ từ thuật toán Louvain, hệ thống kích hoạt cơ chế biên dịch trước trang bách khoa tri thức (*Pre-compiled Community Graph Wiki*). Thay vì duyệt đồ thị nhiều bước tốn thời gian thực trong mỗi phiên hỏi đáp, VeriScholar tự động tổng hợp mạng lưới thực thể thành các trang tài liệu Markdown có cấu trúc:

- Trang chuyên đề phương pháp (*Method Wiki Pages*): Trình bày nguyên lý giải thuật, nguồn gốc công bố ban đầu kèm sơ đồ phả hệ kế thừa giữa các mô hình khoa học.
- Trang bảng xếp hạng thực nghiệm (*Benchmark Leaderboard Pages*): Tổng hợp bảng đối chứng định lượng hiệu năng của các giải pháp trên từng tập dữ liệu kiểm chuẩn tiêu chuẩn.
- Siêu liên kết hai chiều (*Bidirectional Hyperlinks*): Thiết lập liên kết chéo giữa các khái niệm và đoạn trích dẫn PDF nguồn, hỗ trợ học giả truy xuất tức thì ngữ cảnh toàn cục mà không phát sinh độ trễ duyệt mạng trong thời gian thực.

2.5 Kiến trúc tác nhân tự chủ và môi trường Sandbox an toàn

2.5.1 Mô hình suy luận và hành động ReAct phối hợp gọi công cụ

Trong tiến trình phát triển các hệ thống trí tuệ nhân tạo hỗ trợ nghiên cứu khoa học, các mô hình ngôn ngữ lớn truyền thống thường bộc lộ hạn chế khi giải quyết các nhiệm vụ phức tạp đòi hỏi nhiều bước tính toán. Hướng tiếp cận suy luận chuỗi tư duy thuần túy (*Chain-of-Thought – CoT*) [19] chỉ vận hành trên không gian tri thức tham số tĩnh nội tại, dễ dẫn đến hiện tượng suy diễn cảm tính và ảo giác số liệu. Ngược lại, các mô hình gọi công cụ độc lập (như *Toolformer*) [14] tuy kết nối được với môi trường ngoài nhưng thiếu khả năng tự đánh giá và lập luận chiến lược trước các tình huống lỗi phát sinh.

Để khắc phục nhược điểm trên, đề tài áp dụng mô hình kiến trúc ReAct (*Synergizing Reasoning and Acting in Language Models*) do Yao và cộng sự đề xuất [19]. ReAct thiết lập một chu trình phối hợp chặt chẽ giữa năng lực tư duy trừu tượng (*Reasoning*) và hành động tương tác môi trường (*Acting*) theo vòng lặp có hồi tiếp:

$$\text{Thought} \longrightarrow \text{Action} \longrightarrow \text{Observation} \longrightarrow \text{Thought} \ldots \tag{2.7}$$

với mô tả toán học của chu trình tại bước thời gian t:

$$a_t \sim \pi_\theta(a_t \mid h_t, \text{Thought}_t), \quad o_t = \text{Env}(a_t), \quad h_{t+1} = (h_t, \text{Thought}_t, a_t, o_t) \tag{2.8}$$

trong đó Thought_t là lập luận suy nghĩ nội tại hướng dẫn bước tiếp theo, a_t là hành động gọi công cụ ngoại vi, o_t là phản hồi quan sát thực nghiệm từ môi trường Env, và h_{t+1} là lịch sử ngữ cảnh tích lũy được cập nhật liên tục. Nhờ chu trình ReAct, tác nhân không bao giờ suy đoán mò mẫm kết quả thực nghiệm mà luôn kiểm chứng qua các quan sát khách quan từ môi trường thực thi trước khi đưa ra kết luận khoa học.

2.5.2 Khung điều phối tác nhân có trạng thái LangGraph

Nhằm hiện thực hóa chu trình ReAct vào một quy trình kỹ thuật có khả năng mở rộng và chịu lỗi, đề tài sử dụng khung điều phối LangGraph. Khác với các đường ống xử lý theo đồ thị có hướng không chu trình tuyến tính (*DAG*), LangGraph cho phép thiết kế các hệ thống tác nhân đa bước có trạng thái (*Stateful Multi-Agent Workflow*) với cấu trúc đồ thị có chu trình lặp (*Cyclic Graphs*).

Cốt lõi kiến trúc của LangGraph được tổ chức dưới dạng máy trạng thái hữu hạn (*Finite State Machine – FSM*) thông qua đối tượng StateGraph:

- Lược đồ trạng thái dùng chung (*Shared State Schema*): Định nghĩa cấu trúc dữ liệu nhất quán của phiên nghiên cứu (mã nguồn tài liệu, lịch sử thông điệp, danh sách dẫn chứng và nhật ký biên dịch).
- Tập các nút tính toán (*Nodes*): Mỗi nút thực hiện một đơn nguyên nghiệp vụ độc lập, nhận trạng thái hiện tại và trả về bản ghi biến đổi trạng thái tương ứng.
- Cạnh điều hướng có điều kiện (*Conditional Edges*): Đóng vai trò là các bộ định tuyến logic. Sau mỗi bước thực thi, hàm đánh giá điều kiện sẽ quyết định chuyển trạng thái sang bước tiếp theo hoặc quay lui lại vòng lặp để sửa lỗi.
- Cơ chế lưu vết trạng thái bền vững (*Persistence Checkpointing*): Toàn bộ cây trạng thái được lưu tạm sau từng bước thực thi, cho phép hệ thống khôi phục phiên làm việc sau sự cố mà không làm mất dữ liệu nghiên cứu của học giả.

2.5.3 Môi trường máy ảo MicroVM E2B cô lập cấp hệ điều hành và vòng lặp tự sửa lỗi LaTeX

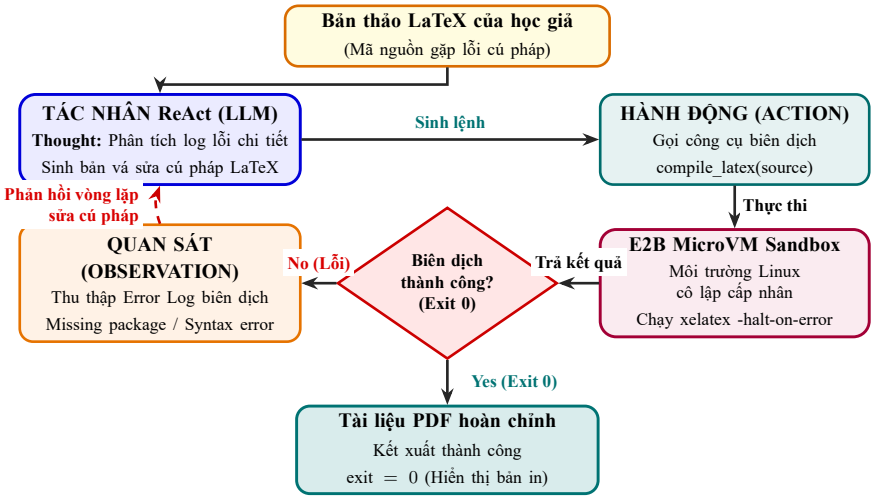
Một trong những thách thức kỹ thuật lớn nhất khi hỗ trợ các nhà khoa học soạn thảo ấn phẩm nghiên cứu là việc tự động biên dịch và sửa lỗi mã nguồn LaTeX. Việc cho phép mô hình AI tự sinh và chạy các lệnh hệ điều hành trực tiếp trên máy chủ tiềm ẩn nguy cơ bảo mật nghiêm trọng về thực thi mã độc từ xa (*Remote Code Execution – RCE*).

Để bảo đảm an toàn, hệ thống tích hợp nền tảng Sandbox E2B (*E2B MicroVM Sandbox*) được xây dựng trên công nghệ ảo hóa siêu nhỏ AWS Firecracker [1]:

- Khởi tạo siêu tốc: Mỗi phiên làm việc được cấp phát một máy ảo siêu nhỏ (*MicroVM*) độc lập trong thời gian dưới 200 mili-giây, cung cấp đầy đủ nhân Linux và môi trường TeX Live chuyên dụng.
- Cô lập mức nhân hệ điều hành (*Kernel-level Isolation*): Mọi lệnh biên dịch

(xelatex, bibtex) đều diễn ra bên trong máy ảo cô lập hoàn toàn với máy chủ chính. Mọi sự cố mã độc, vòng lặp vô hạn hay cạn kiệt tài nguyên đều bị khóa chặt bên trong máy ảo và tự hủy sau khi phiên kết thúc.

Quy trình tác nhân ReAct điều phối công cụ E2B MicroVM Sandbox để tự động biên dịch và phục hồi lỗi cú pháp LaTeX được thể hiện trong Hình 2-4. Khi bản thảo gặp lỗi cú pháp (như thiếu gói lệnh, quên đóng môi trường toán học, hoặc ký tự chưa được thoát), quy trình tự phục hồi diễn ra hoàn toàn tự động: Trước hết, tác nhân gửi mã nguồn sang E2B MicroVM Sandbox để biên dịch thử nghiệm (*Action*). Tiếp theo, môi trường MicroVM ghi nhận nhật ký lỗi biên dịch (*Observation*) và gửi ngược lại cho tác nhân. Sau đó, tác nhân suy luận vị trí dòng lỗi và nguyên nhân cú pháp (*Thought*), rồi tự động vá mã nguồn và kích hoạt vòng lặp biên dịch lại cho đến khi tài liệu kết xuất thành công với mã thoát bằng 0.



Hình 2-4: *Cơ chế tác nhân ReAct điều phối môi trường E2B MicroVM Sandbox tự động phục hồi lỗi cú pháp LaTeX*

2.5.4 Mô hình quản lý khóa người dùng tự cấp Bring Your Own Key (BYOK) và chuẩn mã hóa AES-256-GCM

Nhằm bảo vệ quyền riêng tư và quyền tự chủ tài chính cho học giả, hệ thống áp dụng mô hình quản lý khóa người dùng tự cấp Bring Your Own Key (*Bring Your Own Key – BYOK*). Người dùng trực tiếp cung cấp khóa API từ các nhà cung cấp (như gpt-4o-mini trên Microsoft Azure); máy chủ tuyệt đối không lưu trữ khóa dạng văn bản thô.

Khóa API được mã hóa bảo vệ theo chuẩn AEAD với thuật toán AES-256-GCM [6]:

$$C = \mathrm{GCTR}_K\big(\mathrm{inc}_{32}(J_0), \mathrm{Plaintext}\big), \quad T = \mathrm{GHASH}_H(\mathrm{AAD}, C) \oplus E_K(J_0) \tag{2.9}$$

trong đó K là khóa bí mật 256 bits, IV là vector khởi tạo ngẫu nhiên 96 bits ($J_0 = \mathrm{IV} \parallel 0^{31} \parallel 1$), AAD là dữ liệu xác thực người dùng, C là bản mã và T là thẻ xác thực 128 bits. Quá trình giải mã tuân thủ nguyên lý bộ nhớ khả biến (*In-Memory Ephemeral Decryption*): khóa chỉ được giải mã tạm thời trong RAM khi thực thi

gọi API và được giải phóng tức thì, bảo đảm không bao giờ lưu vết trên đĩa hay trong nhật ký.

2.5.5 Nền tảng quan sát vận hành và truy vết suy luận Langfuse

Nhằm bảo đảm tính minh bạch và kiểm toán các quyết định suy luận của AI, hệ thống tích hợp nền tảng quan sát vận hành Langfuse:

- Cây truy vết phân tầng (*Hierarchical Tracing*): Phân rã mỗi phiên tương tác thành Dấu vết tổng thể (*Trace*), hành động gọi công cụ (*Spans*) và các lượt tạo sinh (*Generations*) kèm siêu dữ liệu số lượng token tiêu thụ. Cơ chế ghi vết độc lập hoàn toàn với vùng nhớ giải mã khóa BYOK.
- Giám sát độ trễ và chi phí thời gian thực: Đo lường chính xác thời gian phản hồi ký tự đầu tiên (*TTFT*) qua giao thức SSE, phân tích chi phí tài chính chi tiết theo từng mô hình giúp học giả chủ động quản lý ngân sách.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Khảo sát yêu cầu người dùng học thuật

Trong hoạt động nghiên cứu khoa học tại các trường đại học và viện nghiên cứu, việc sản xuất các công bố học thuật quốc tế chất lượng cao đòi hỏi người làm nghiên cứu phải xử lý khối lượng tri thức rất lớn với độ chuẩn xác và tính liêm chính học thuật (*Academic Integrity*) tuyệt đối. Qua phân tích nhu cầu thực tế của giảng viên, nhà nghiên cứu, nghiên cứu sinh và sinh viên làm đồ án tốt nghiệp, hệ thống VeriScholar xác lập 5 nhóm yêu cầu người dùng then chốt:

- **Yêu cầu về khám phá và quản trị tài liệu:** Học giả cần cổng tra cứu thống nhất cho phép tìm kiếm đồng thời trên các cơ sở dữ liệu mở (arXiv, CrossRef, Semantic Scholar), loại bỏ tài liệu trùng lặp qua mã băm nội dung nhị phân (CAS SHA-256), và phân loại trạng thái bình duyệt (*Peer-Reviewed*).
- **Yêu cầu về đọc sâu và kiểm chứng trích dẫn:** Khi tương tác với bài báo hai cột (*two-column*), trợ lý AI phải phản hồi thời gian thực qua luồng Server-Sent Events (SSE) và cung cấp bằng chứng trực quan qua khung viền tọa độ Bounding Box (IoU $\geq 0{,}85$) trên bản PDF gốc, loại trừ ảo giác trích dẫn.
- **Yêu cầu về tổng hợp mạng lưới tri thức:** Tự động trích xuất 4 thực thể học thuật (Paper, Method, Dataset, MetricResult), khử trùng lặp đỉnh bằng Cosine $\geq 0{,}90$ và phân cụm Louvain, cho phép thực hiện các truy vấn bắc cầu 2-hop liên tài liệu.
- **Yêu cầu về đồng tác giả và tự phục hồi bản thảo LaTeX:** Hỗ trợ soạn thảo LaTeX theo chuẩn IEEE/ACM, biên dịch cô lập trong hộp cát MicroVM, và tự động phân tích nhật ký lỗi .log để vá lỗi cú pháp với tỷ lệ thành công đạt $96{,}0\%$.
- **Yêu cầu phi chức năng và bảo mật:** Bảo vệ quyền sở hữu trí tuệ qua cơ chế tự mang khóa cá nhân (BYOK) mã hóa AES-256-GCM, thời gian trả token đầu tiên TTFT $< 1{,}5$ s, và độ trễ truy vấn CSDL đạt $1{,}66$ ms.

3.2 Khảo sát nghiệp vụ và quy trình xử lý tài liệu nghiên cứu

Để xây dựng nền tảng trợ lý AI hỗ trợ nghiên cứu khoa học sát với nhu cầu thực tiễn của học giả, đề tài tiến hành phân tích quy trình làm việc học thuật quốc tế (*Academic Research Workflow*) cùng thực trạng vận hành của các công cụ hiện hành. Việc phân tích tập trung vào chu trình nghiên cứu tiêu chuẩn của các nhà khoa học, giảng viên, nghiên cứu sinh và sinh viên làm đồ án tốt nghiệp, qua đó làm rõ các điểm nghẽn kỹ thuật trong quy trình truyền thống và xác lập mô hình chuyển đổi quy trình nghiệp vụ cho hệ thống.

3.2.1 Khảo sát quy trình tìm kiếm và thu thập tài liệu nghiên cứu

Hiện nay, việc tìm kiếm bài báo khoa học chủ yếu dựa trên thao tác thủ công của học giả trên các công cụ tìm kiếm phổ quát và các cơ sở dữ liệu học thuật độc lập:

- Học giả nhập từ khóa trên Google Scholar, arXiv hoặc Semantic Scholar, sau

đó duyệt từng trang kết quả để tải tệp PDF về máy tính cá nhân.

- Việc tải tài liệu diễn ra phân tán ở nhiều thư mục máy tính cá nhân. Khi làm việc theo nhóm, các thành viên dễ tải trùng một tệp PDF dung lượng từ 5 MB đến 20 MB, gây lãng phí dung lượng lưu trữ và khó kiểm soát phiên bản.
- Nhiều bài báo trên arXiv là bản thảo chưa qua bình duyệt (*Preprint*), trong khi các bài trên tạp chí đã trải qua phản biện kín (*Peer-Reviewed*). Học giả mất nhiều thời gian tra cứu thủ công tình trạng xuất bản.

3.2.2 Khảo sát quy trình đọc sâu và thẩm định nguồn trích dẫn

Trong giai đoạn đọc hiểu tài liệu chuyên ngành:

- Hầu hết bài báo quốc tế (IEEE, ACM, Springer) được trình bày dạng hai cột (*two-column*). Công cụ trích xuất văn bản thông thường đọc nhảy ngang giữa hai cột, làm đảo lộn thứ tự ngữ nghĩa của công thức toán và đoạn văn. Quy trình yêu cầu phân lập ranh giới tọa độ trục dọc qua đường phân chia trung tuyến $\text{mid_x} = 250$ (chuẩn hóa trên khổ trang 500 pt) và chia đoạn văn bản theo cửa sổ trượt tối đa 250 từ.
- Các công cụ trò chuyện với tài liệu hiện nay chỉ dẫn chứng số trang chung chung. Trong một trang chứa hàng nghìn chữ, học giả vẫn phải tự mắt tìm kiếm từng câu chữ để xác nhận. Hệ thống khảo sát đòi hỏi kiến trúc truy xuất lai kết hợp véc-tơ HNSW và chỉ mục đảo toàn văn qua hàm xếp hạng mật độ ts_rank_cd, đồng thời kiểm soát ảo giác qua hai ngưỡng tin cậy CRAG $0{,}35$ (chấp nhận) và $0{,}20$ (chủ động từ chối).
- Nguy cơ ảo giác: Mô hình ngôn ngữ lớn có xu hướng tự điền khuyết số liệu đo lường khi không tìm thấy thông tin chính xác. Việc hiển thị khung viền tọa độ Bounding Box định vị chính xác vị trí văn bản gốc là tiêu chí bắt buộc để bảo đảm liêm chính học thuật.

3.2.3 Khảo sát quy trình quản lý và liên kết đồ thị tri thức học thuật

Khi tiến hành tổng quan tài liệu (*Literature Review*):

- Nhà nghiên cứu cần đối chiếu mô hình kiến trúc, bộ dữ liệu chuẩn và kết quả đo lường cụ thể giữa các công trình tiền nhiệm.
- Phương pháp tạo bảng tính Excel thủ công trở nên cồng kềnh khi số lượng bài báo tăng lên, khó cập nhật và mất đi khả năng truy vấn quan hệ bắc cầu giữa các thực thể tri thức.
- Nhu cầu thực tế là tự động xây dựng mạng đồ thị tri thức liên kết 4 thực thể cốt lõi (*Paper* $\rightarrow$ *Method* $\rightarrow$ *Dataset* $\rightarrow$ *MetricResult*) thông qua cơ chế ép kiểu cấu trúc OpenAI Structured Outputs, hợp nhất đỉnh tương đồng ngữ nghĩa Cosine $\geq 0{,}90$ và phân cụm cộng đồng theo thuật toán Louvain.

3.2.4 Khảo sát quy trình biên soạn bản thảo và sửa lỗi biên dịch LaTeX

Biên soạn bản thảo là công đoạn tốn nhiều thời gian nhất trong chu trình nghiên cứu:

- Ngôn ngữ LaTeX có cú pháp rất chặt chẽ. Chỉ một lỗi nhỏ như quên đóng ngoặc môi trường toán học, ký tự đặc biệt chưa thoát mã (như _, %, &), hoặc

sai nhãn trích dẫn BibTeX cũng khiến toàn bộ quá trình biên dịch bị đình trệ.

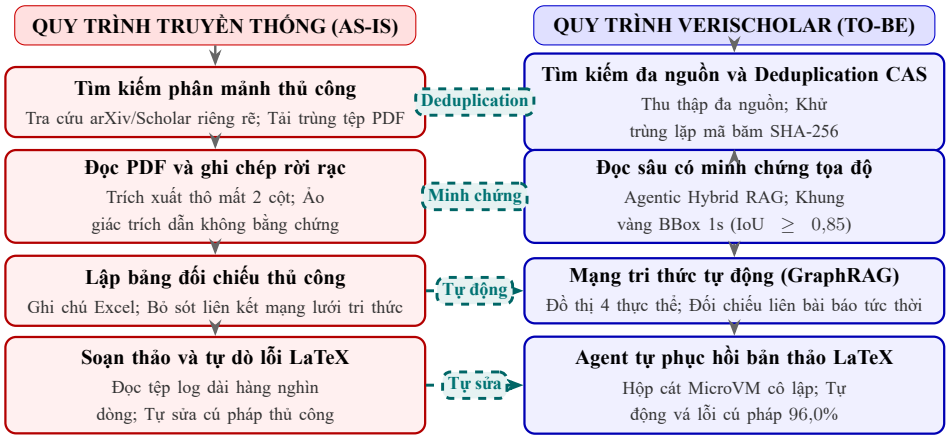
- Tệp nhật ký lỗi (.log) thường dài hàng nghìn dòng. Trợ lý AI có khả năng tự động đọc tệp log, vận hành chu trình ReAct gồm các trạng thái reason_and_plan và bộ định tuyến phục hồi healing_router, kích hoạt bộ vá lỗi tự động _auto_patch kết hợp thuật toán sửa lỗi heuristic apply_heuristic_repair để tự tạo bản vá sửa lỗi trên mã nguồn LaTeX là nhu cầu cấp thiết.

3.2.5 Khảo sát quy trình quản lý hồ sơ học giả và bảo mật khóa cá nhân

Đối với công tác quản trị và bảo mật thông tin nghiên cứu:

- Học giả cần định danh quốc tế chuẩn ORCID iD 16 chữ số để phân biệt với tác giả trùng tên.
- Học giả e ngại dữ liệu nghiên cứu chưa công bố bị các nền tảng AI thương mại sử dụng để huấn luyện mô hình. Cơ chế tự mang khóa cá nhân (*Bring Your Own Key – BYOK*) mã hóa phân tầng và không lưu vết trên đĩa cứng bảo đảm tính an toàn tuyệt đối cho người dùng.

Từ kết quả khảo sát thực tế trên, sự chuyển dịch từ quy trình nghiên cứu truyền thống (AS-IS) sang quy trình nghiên cứu mới (TO-BE) trên nền tảng VeriScholar được mô hình hóa trực quan trong Hình 3-1.



Hình 3-1: *Mô hình đối chiếu chuyển đổi quy trình nghiên cứu khoa học AS-IS và TO-BE*

3.3 Thiết kế kiến trúc hệ thống và phân tầng Lục giác

Để bảo đảm tính bền vững, khả năng kiểm thử độc lập và ngăn chặn triệt để sự phụ thuộc vòng giữa các thành phần nghiệp vụ và hạ tầng công nghệ, hệ thống VeriScholar được thiết kế tuân thủ nghiêm ngặt mô hình Kiến trúc Lục giác (*Hexagonal Architecture / Ports and Adapters Pattern*) của Cockburn [4]. Kiến trúc này chia tách rành mạch ba phân tầng: Lõi miền nghiệp vụ thuần khiết (*Domain Core*), Tầng Cổng trừu tượng (*Ports*) và Tầng Bộ điều hợp hạ tầng (*Adapters*), đồng thời tổ chức không gian nghiên cứu của học giả thành 4 không gian nghiệp vụ độc lập (*4 Research Spaces*).

3.3.1 Nguyên lý phân tầng Kiến trúc Lục giác

Mô hình phân tầng kiến trúc của VeriScholar vận hành dựa trên các nguyên tắc thiết kế bất biến:

- **Lõi miền nghiệp vụ thuần khiết (packages/core):** Là trung tâm của hệ thống, chứa đựng toàn bộ các thực thể miền (*Entities*), đối tượng giá trị (*Value Objects*), dịch vụ miền (*Domain Services*) và quy tắc nghiệp vụ cốt lõi. Lõi miền hoàn toàn độc lập với các thư viện I/O, giao tiếp web hoặc khung cơ sở dữ liệu, chỉ sử dụng thư viện chuẩn của Python và thư viện kiểm định dữ liệu Pydantic.
- **Tầng Cổng trừu tượng (Ports):** Định nghĩa các giao diện trừu tượng (*Abstract Interfaces*) làm ranh giới giữa lõi miền và môi trường bên ngoài:
  - *Cổng điều khiển (Driving / Primary Ports):* Gồm các giao diện ca sử dụng tiếp nhận yêu cầu từ người dùng như SearchDocumentPort, Deep ReadingPort, GraphRAGPort, DraftAuthoringPort.
  - *Cổng bị điều khiển (Driven / Secondary Ports):* Gồm các giao diện định nghĩa dịch vụ hạ tầng mà lõi nghiệp vụ cần để lưu trữ và suy luận như DocumentRepositoryPort, VectorSearchPort, LLMInference Port, CompilerSandboxPort.
- **Tầng Bộ điều hợp hạ tầng (Adapters):** Hiện thực hóa các giao diện cổng thông qua các công nghệ cụ thể:
  - *Bộ điều hợp điều khiển (Driving Adapters):* Điểm cuối REST API phát triển trên FastAPI [12] (apps/api) và giao diện người dùng tương tác thời gian thực trên React 19 (apps/web).
  - *Bộ điều hợp bị điều khiển (Driven Adapters):* Kho dữ liệu quan hệ và vector sử dụng SQLAlchemy 2.0 Async kết nối PostgreSQL 16 pgvector [11], bộ nhớ đệm Redis, bộ điều hợp máy ảo đám mây E2B MicroVM [1], bộ điều hợp gọi mô hình ngôn ngữ lớn bảo vệ khóa cá nhân (BYOK) theo chuẩn mã hóa AES-256-GCM [6], và bộ kết nối giám sát Langfuse Cloud.

3.3.2 Mô hình 4 Không gian nghiên cứu (4 Research Spaces)

Nhằm đáp ứng trọn vẹn chu trình nghiên cứu khoa học của học giả, hệ thống tổ chức luồng tương tác thành 4 không gian nghiên cứu chuyên biệt:

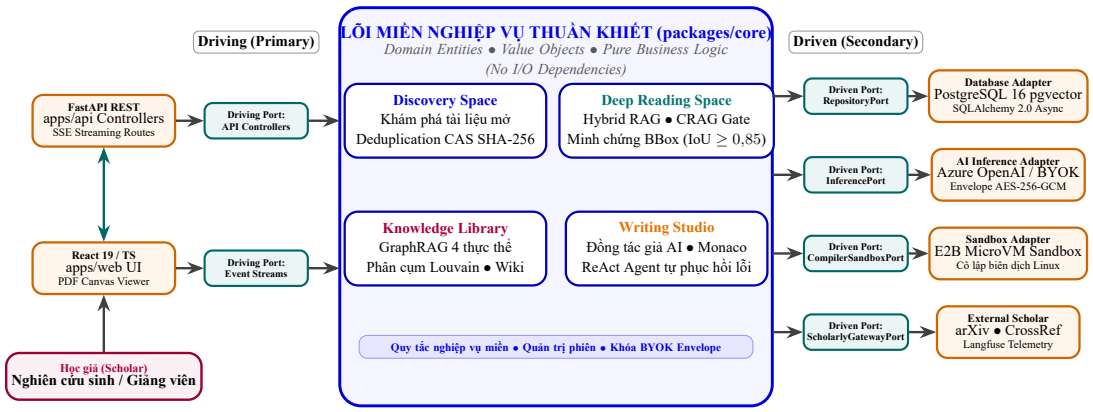
1. **Không gian Khám phá (Discovery Space):** Đóng vai trò thu thập và tiếp nhận tri thức. Học giả thực hiện tra cứu tài liệu học thuật từ các cổng mở (arXiv, CrossRef, Semantic Scholar), lọc bài báo đã bình duyệt, và tải tài liệu PDF với cơ chế Deduplication lưu trữ CAS SHA-256 giúp tiết kiệm tối đa tài nguyên lưu trữ.
2. **Không gian Đọc sâu (Deep Reading Space):** Đóng vai trò phân tích và kiểm chứng thông tin. Hệ thống cung cấp giao diện hiển thị PDF Canvas hai cột,

tích hợp Agentic Hybrid RAG (kết hợp vector HNSW và từ khóa BM25 qua RRF), cổng kiểm định độ tự tin CRAG [18], và tự động vẽ khung viền tọa độ Bounding Box (IoU $\geq 0{,}85$) minh chứng trực tiếp trên tài liệu gốc qua 5 luồng truyền phát SSE.

3. **Không gian Thư viện Tri thức (Knowledge Library Space):** Đóng vai trò liên kết và tổng hợp mạng lưới tri thức. Phân hệ GraphRAG tự động trích xuất 4 thực thể (Paper, Method, Dataset, MetricResult), khử trùng lặp đỉnh bằng độ tương đồng Cosine $\geq 0{,}90$, phân cụm cộng đồng Louvain [3], và biên dịch hệ thống trang Wiki học thuật hỗ trợ truy vấn bắc cầu 2-hop.

4. **Không gian Soạn thảo (Writing Studio Space):** Đóng vai trò đồng tác giả và sản xuất công bố khoa học. Tích hợp trình soạn thảo Monaco, thực thi biên dịch LaTeX an toàn trong hộp cát E2B MicroVM [1], và điều phối tác nhân ReAct LangGraph [19, 15, 17] tự động phân tích tệp nhật ký .log để vá lỗi cú pháp với tỷ lệ thành công đạt $96{,}0\%$.

Sơ đồ kiến trúc Lục giác tổng thể và mô hình 4 không gian nghiên cứu của VeriScholar được mô tả trực quan trong Hình 3-2.



Hình 3-2: *Kiến trúc Lục giác phân tầng và mô hình 4 Không gian nghiên cứu của hệ thống VeriScholar*

3.3.3 Thiết kế các đường ống thuật toán xử lý cốt lõi

Để hiện thực hóa các chức năng nghiên cứu trong 4 không gian nêu trên, hệ thống VeriScholar thiết kế và cài đặt bốn đường ống thuật toán cốt lõi xử lý xuyên suốt từ bóc tách hình học tài liệu, truy xuất lai tự kiểm định, tự phục hồi bản thảo LaTeX đến tổng hợp đồ thị tri thức học thuật:

- **Thuật toán sắp xếp khối văn bản PDF hai cột và phân đoạn cửa sổ trượt bảo toàn tọa độ:** Với mỗi trang tài liệu có kích thước vật lý (W, H), bộ giải mã bố cục trích xuất tập các khối văn bản thô kèm hình chữ nhật bao quanh (x_0, y_0, x_1, y_1). Thuật toán trước tiên loại bỏ các phần tử nhiễu đầu trang và chân trang thỏa mãn điều kiện $y_0 < 0{,}06 \times H$ hoặc $y_1 > 0{,}93 \times H$ có độ dài dưới 80 ký tự. Tiếp theo, đường trung tuyến dọc mid_x $= 0{,}5 \times W$ được thiết lập để phân loại từng khối văn bản theo ba vùng bố cục: khối trải rộng toàn trang

(khi $x_0 < 0{,}35 \times W$ và $x_1 > 0{,}65 \times W$, như tiêu đề hoặc tóm tắt), khối thuộc cột trái ($x_{\text{center}} <$ mid_x) và khối thuộc cột phải ($x_{\text{center}} \geq$ mid_x). Các khối được sắp xếp tất định theo khóa bộ bốn $(\text{band}, \text{col}, y_0, x_0)$, bảo đảm luồng văn bản đọc hoàn chỉnh hết cột trái trước khi chuyển sang cột phải. Trên chuỗi từ đã sắp xếp, thuật toán áp dụng cửa sổ trượt kích thước tối đa max_words $= 250$ từ với độ gối lấp ngữ cảnh overlap_words $= 50$ từ, đồng thời chiếu chuẩn hóa mọi khung bao hình học về không gian đơn vị bất biến độ phân giải $[0, 1]^4$:

$$\hat{B} = \left(\frac{x_0}{W}, \frac{y_0}{H}, \frac{x_1}{W}, \frac{y_1}{H} \right) \in [0, 1]^4 \tag{3.1}$$

- **Thuật toán truy xuất lai SQL CTE Hybrid Retrieval và cổng kiểm định độ tự tin CRAG:** Khi tiếp nhận truy vấn của học giả, hệ thống thực thi truy vấn hợp nhất ngay trong PostgreSQL thông qua biểu thức bảng chung (CTE) gồm hai nhánh song song: nhánh ngữ nghĩa quét chỉ mục đồ thị phân cấp HNSW [11] với toán tử khoảng cách Cosine (embedding <=> query_vec), và nhánh từ vựng quét chỉ mục đảo GIN trên cột content_tsv bằng hàm mật độ bao phủ ts_rank_cd kết hợp thuật toán Okapi BM25 [13] ($k_1 = 1{,}5,\ b = 0{,}75$). Hai danh sách ứng viên được hợp nhất thứ hạng bằng công thức Reciprocal Rank Fusion (RRF) [5] với hằng số làm trơn $k = 60$, sau đó đi qua tầng tái xếp hạng chéo (*Cross-Encoder Reranking*) để tính điểm hội tụ $\tau \in [0, 1]$. Trước khi gọi mô hình sinh câu trả lời, cổng kiểm định tiền sinh văn bản (*CRAG Confidence Gating*) [18] đối chiếu τ với hai ngưỡng quyết định:
  - Khi $\tau \geq 0{,}35$ (CORRECT): Ngữ cảnh đạt độ tin cậy cao, cho phép tổng hợp câu trả lời trực tiếp kèm thẻ dẫn nguồn.
  - Khi $0{,}20 \leq \tau < 0{,}35$ (AMBIGUOUS_EXPANDED): Hệ thống tự động kích hoạt hàm mở rộng truy vấn học thuật đa chiều (expand_academic_query) và thực hiện truy xuất bổ sung vòng hai trước khi tổng hợp.
  - Khi $\tau < 0{,}20$ (ACTIVE_ABSTENTION): Hệ thống chủ động từ chối suy diễn nhằm triệt tiêu hoàn toàn nguy cơ ảo giác ngoại suy.
- **Thuật toán tự phục hồi mã nguồn LaTeX qua đồ thị LangGraph ReAct:** Quá trình biên dịch và tự sửa lỗi bản thảo được điều phối bởi đồ thị trạng thái hữu hạn theo mô hình ReAct và cơ chế tự suy ngẫm Reflexion [19, 15, 17] gồm 4 nút chức năng: reason_and_plan (phân tích trạng thái và lập kế hoạch), tool_execution (thực thi biên dịch trong hộp cát MicroVM [1] hoặc trích xuất ngữ cảnh), healing_router (đánh giá mã thoát và điều phối chu trình tự vá lỗi với giới hạn tối đa $N_{\text{max}} = 3$ vòng lặp), và generate_final_answer (kết xuất phản hồi). Khi phát hiện lỗi biên dịch (ExitCode $\neq 0$), hệ thống áp dụng cơ chế sửa lỗi hai tầng: (i) bộ vá lỗi tất định tốc độ cao tự động nạp các gói lệnh bị thiếu (amsmath, graphicx, booktabs), thoát mã các ký tự đặc biệt ngoài môi trường toán và đóng các khối môi trường bị khuyết; (ii) bộ sinh bản vá vi phẫu bằng mô hình ngôn ngữ phân tích tệp nhật ký .log để sửa các xung đột cú pháp phức tạp. Đặc biệt, chốt chặn chống dao động tuần hoàn tính mã băm SHA-256 của mã nguồn sau mỗi lần vá; nếu phát hiện mã băm lặp lại

hoặc mức độ lỗi không giảm, healing_router lập tức dừng vòng lặp và khôi phục phiên bản tốt nhất.

- **Thuật toán trích xuất đồ thị tri thức 4 thực thể, hợp nhất ngữ nghĩa và phân cụm Louvain (GraphRAG):** Đối với mỗi tài liệu trong thư viện nghiên cứu, đường ống GraphRAG [7, 9] sử dụng cơ chế ép kiểu lược đồ đầu ra OpenAI Structured Outputs (kết hợp bộ quy tắc biểu thức chính quy dự phòng Rule Fallback) để bóc tách 4 loại nút thực thể (Paper, Method, Dataset, MetricResult) cùng 4 loại cạnh quan hệ có hướng (PROPOSES, EVALUATED_ON, ACHIEVES, CITES). Để ngăn chặn sự phân mảnh đỉnh giữa các bài báo khác nhau, thuật toán chuẩn hóa tên thực thể (canonical_name) và tính độ tương đồng Cosine giữa các vector nhúng thực thể; các nút cùng loại có Cosine $\geq 0{,}90$ được tự động hợp nhất về một thực thể chuẩn tắc duy nhất trong bảng graph_nodes. Cuối cùng, thuật toán phân cụm cộng đồng Louvain [3] được thực thi trên đồ thị quy đổi vô hướng nhằm tối ưu hóa độ mô-đun hóa Q, từ đó tổng hợp bản tóm tắt cộng đồng theo thư mục (folder_community_summaries) và biên dịch hệ thống trang tri thức liên kết hai chiều (wiki_pages) hỗ trợ truy vấn bắc cầu 2-hop.

3.4 Phân tích các ca sử dụng (Use Case) của hệ thống

3.4.1 Biểu đồ Use Case tổng quan của hệ thống

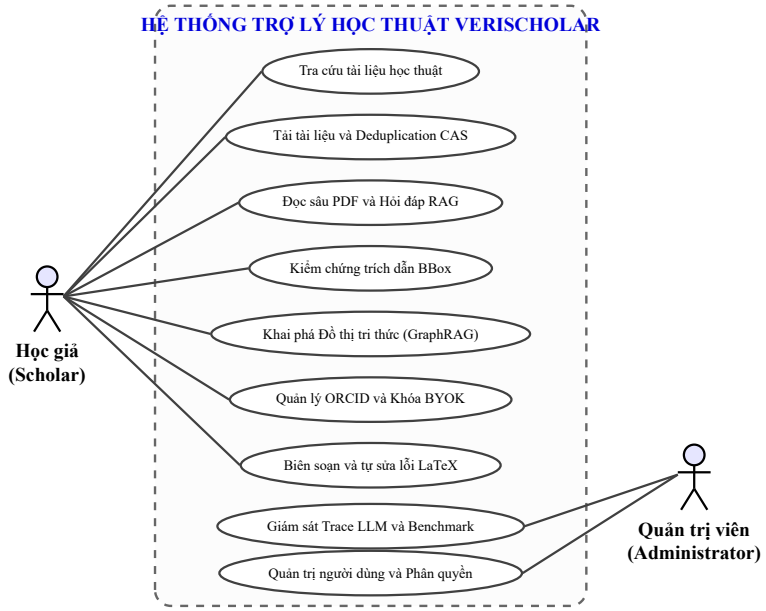
Căn cứ vào khảo sát nghiệp vụ và mô hình kiến trúc Lục giác đã xác lập tại Mục 3.2 và Mục 3.3, hệ thống VeriScholar được thiết kế phục vụ hai tác nhân chính (Actor):

- **Học giả (Scholar / Researcher):** Là tác nhân trung tâm của nền tảng (gồm các giảng viên, nhà nghiên cứu, nghiên cứu sinh và sinh viên làm đồ án tốt nghiệp). Học giả tương tác trực tiếp với 4 Không gian nghiên cứu: khám phá tài liệu mở, đọc sâu kiểm chứng trích dẫn Bounding Box, khai phá mạng tri thức GraphRAG và biên soạn bản thảo khoa học có AI Agent tự động sửa lỗi.
- **Quản trị viên (Administrator / DevOps):** Chịu trách nhiệm giám sát và bảo đảm an toàn hạ tầng: kích hoạt các kịch bản kiểm thử định lượng tự động qua điểm cuối REST vận hành (/api/v1/evaluations/run-benchmark), theo dõi *Trace Logs* và chi phí Token suy luận trên Langfuse Cloud, giám sát tài nguyên máy chủ đám mây và hộp cát MicroVM cách ly.

Mối quan hệ tương tác trực quan giữa các tác nhân và các phân hệ của VeriScholar được mô hình hóa trong Biểu đồ Use Case tổng quan (Hình 3-3). Toàn bộ các chức năng nghiên cứu chuyên sâu và quản lý khóa API cá nhân (BYOK) đều thuộc quyền sở hữu riêng của tác nhân Học giả, trong khi tác nhân Quản trị viên chỉ kết nối với các ca sử dụng giám sát và quản trị vận hành độc lập.

3.4.2 Các tác nhân và danh mục ca sử dụng của hệ thống

Trên cơ sở biểu đồ tổng quan, danh mục 12 ca sử dụng chính đối với từng tác nhân trong hệ thống VeriScholar được tổng hợp và phân loại chi tiết trong Bảng 3-1.



Hình 3-3: *Biểu đồ Use Case tổng quan của hệ thống VeriScholar*

Bảng 3-1: *Các Use Case chính đối với mỗi Actor trong hệ thống VeriScholar*

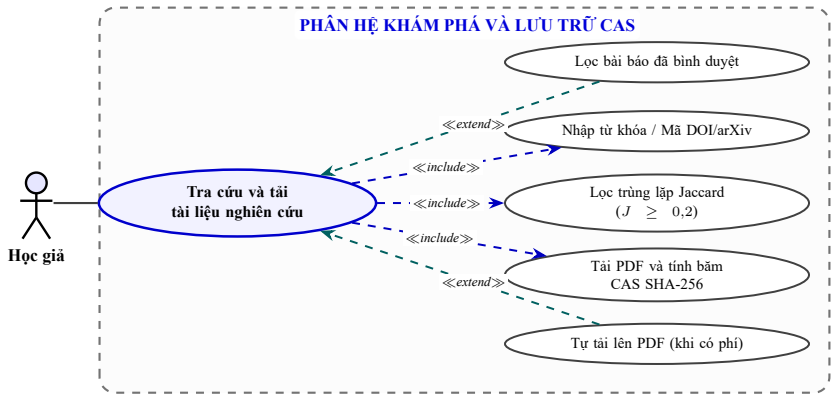
STT	Tên Actor	Use Case
1	**Học giả (Scholar)**	Đăng ký / Đăng nhập và xác thực tài khoản
2		Tìm kiếm và thu thập tài liệu học thuật đa nguồn
3		Tải tài liệu cá nhân và Deduplication lưu trữ CAS
4		Đọc sâu tài liệu PDF hai cột và Hỏi đáp với AI
5		Kiểm chứng trích dẫn tọa độ Bounding Box trực quan
6		Khai phá và truy vấn Đồ thị tri thức (GraphRAG)
7		Quản lý hồ sơ định danh ORCID và Kho khóa API BYOK
8		Biên soạn bản thảo LaTeX và Tự phục hồi lỗi biên dịch
9	**Quản trị viên (Admin)**	Quản lý tài khoản người dùng và phân quyền
10		Giám sát sức khỏe hạ tầng hộp cát E2B MicroVM
11		Theo dõi Trace mô hình ngôn ngữ và Chi phí Token
12		Cấu hình chính sách lưu trữ và an toàn dữ liệu

3.4.3 Phân tích các Use Case phía Học giả

Nhằm làm rõ bản chất vận hành của 4 không gian nghiên cứu, tiểu mục này tiến hành đặc tả chi tiết 4 Use Case cốt lõi: *Tra cứu và tải tài liệu nghiên cứu*, *Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ*, *Khai phá Đồ thị tri thức và biên soạn tri thức học thuật*, và *Biên soạn bản thảo và tự phục hồi lỗi LaTeX*.

3.4.3.1 Phân tích Use Case "Tra cứu và tải tài liệu nghiên cứu"

Use Case này cho phép học giả tìm kiếm các bài báo khoa học từ các kho học thuật mở toàn cầu (arXiv, CrossRef, Semantic Scholar), lọc tài liệu đã bình duyệt và lọc trùng lặp Jaccard ($J(Q,T) \geq 0{,}2$), đồng thời tải trực tiếp tài liệu vào kho lưu trữ cá nhân kèm cơ chế Deduplication CAS SHA-256. Sơ đồ phân rã ca sử dụng con được thể hiện trong Hình 3-4 và đặc tả chi tiết trong Bảng 3-2.



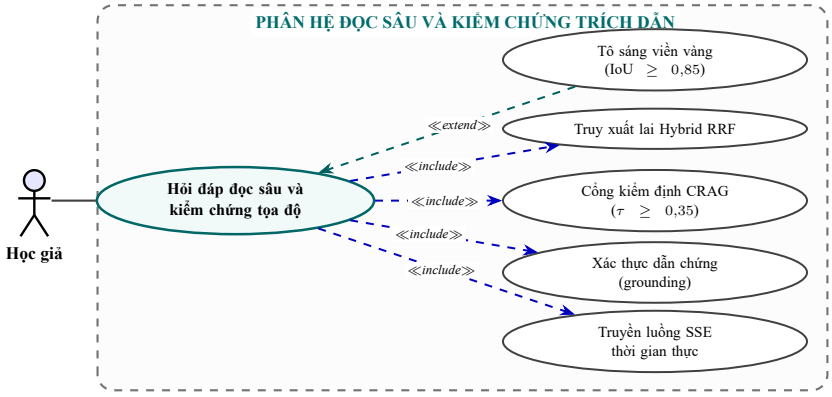
Hình 3-4: *Biểu đồ ca sử dụng con "Tra cứu và tải tài liệu nghiên cứu"*

Bảng 3-2: *Đặc tả Use Case "Tra cứu và tải tài liệu nghiên cứu"*

Tên Use Case	**Tra cứu và tải tài liệu nghiên cứu**
Actor	Học giả (Scholar)
Mục tiêu	Tìm kiếm bài báo khoa học theo từ khóa hoặc mã DOI/arXiv, lọc bài báo đã bình duyệt và lưu trữ vào thư viện nghiên cứu cá nhân.
Điều kiện tiên quyết	Học giả đã đăng nhập vào hệ thống VeriScholar.
Điều kiện kết thúc	Tài liệu được tải vào không gian làm việc; bản ghi thư mục và khối tệp CAS được tạo lập thành công.
Luồng sự kiện chính	1. Học giả nhập từ khóa tìm kiếm hoặc mã định danh DOI/arXiv vào thanh tìm kiếm. 2. Giao diện người dùng phản hồi tức thì với khung xương tải trang (*Skeleton loading*); đối với truy vấn từ khóa từ 3 từ trở lên, hệ thống áp dụng bộ lọc độ tương đồng tiêu đề bằng chỉ số Jaccard ($J(Q,T) \geq 0{,}2$) nhằm ngăn chặn bài báo lạc đề, đồng thời truy vấn qua các cổng học thuật mở và hiển thị danh sách kết quả. 3. Học giả có thể kích hoạt nút lọc "Đã bình duyệt" (*Peer-reviewed*) để hệ thống tái sắp xếp thứ tự ưu tiên các bài báo uy tín lên đầu. 4. Học giả chọn bài báo mong muốn và nhấp nút "Tải tài liệu". 5. Hệ thống tải tệp PDF toàn văn, tính mã băm SHA-256 để kiểm tra Deduplication trong kho lưu trữ CAS, và liên kết tài liệu vào không gian làm việc của học giả.
Luồng thay thế / Ngoại lệ	• Nếu tệp PDF đã tồn tại trong kho lưu trữ chung: Hệ thống tái sử dụng khối tệp vật lý sẵn có và tăng biến đếm tham chiếu (ref_count), rút ngắn thời gian nạp xuống mức tức thời. • Nếu bài báo thuộc diện thu phí bản quyền: Hệ thống thông báo chỉ hỗ trợ hiển thị tóm tắt (*Abstract*) và cho phép học giả tự tải lên tệp PDF cá nhân từ máy tính.

3.4.3.2 Phân tích Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"

Use Case này hỗ trợ học giả phân tích chuyên sâu nội dung bài báo, đặt câu hỏi cho trợ lý AI và nhận câu trả lời có kèm các thẻ trích dẫn tọa độ Bounding Box (IoU $\geq 0{,}85$) hiển thị viền vàng trực quan trên bản PDF gốc (yêu cầu cấu hình khóa BYOK). Sơ đồ phân rã ca sử dụng con được thể hiện trong Hình 3-5 và đặc tả chi tiết trong Bảng 3-3.



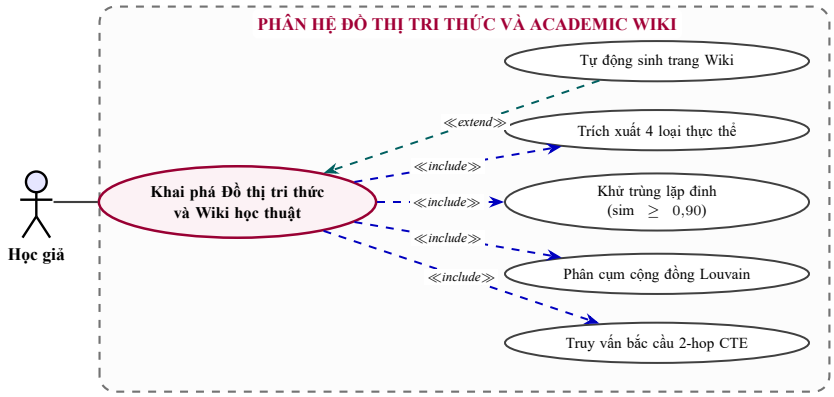
Hình 3-5: *Biểu đồ ca sử dụng con "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"*

Bảng 3-3: *Đặc tả Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"*

Tên Use Case	**Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ**
Actor	Học giả (Scholar)
Mục tiêu	Học giả đặt câu hỏi nghiên cứu về tài liệu đang đọc; trợ lý AI phản hồi kèm định vị chính xác vị trí câu văn bằng chứng trên trang PDF gốc.
Điều kiện tiên quyết	Học giả đã đăng nhập, cấu hình khóa API cá nhân (BYOK) hợp lệ và mở tài liệu PDF trong phân hệ Đọc sâu.
Điều kiện kết thúc	Câu trả lời được truyền phát qua luồng SSE kèm theo các thẻ minh chứng số [1], [2] liên kết với tọa độ Bounding Box.
Luồng sự kiện chính	1. Học giả nhập câu hỏi nghiên cứu vào khung hội thoại. 2. AI Agent kích hoạt công cụ tìm kiếm lai Hybrid Search (kết hợp vector HNSW [11] và từ khóa BM25 [13] qua thuật toán RRF [5]) cùng tầng tái xếp hạng chéo để trích xuất các phân đoạn văn bản liên quan nhất. 3. Cổng kiểm định tiền sinh văn bản CRAG [18] đánh giá ngưỡng tự tin ($\tau \geq 0{,}35$: tổng hợp trực tiếp; $0{,}20 \leq \tau < 0{,}35$: mở rộng truy vấn; $\tau < 0{,}20$: từ chối suy diễn), sau đó mô hình tổng hợp câu trả lời và thực thi hàm kiểm chứng hậu sinh tất định (verify_output_grounding) nhằm xác thực mọi thẻ trích dẫn $[i]$ khớp với phân đoạn nguồn. 4. Trợ lý AI truyền luồng phản hồi theo thời gian thực cùng siêu dữ liệu tọa độ chuẩn hóa $[0, 1]^4$ về giao diện người dùng. 5. Học giả nhấp chuột vào thẻ trích dẫn [1] trong câu trả lời; trình xem PDF lập tức tự động cuộn trang và vẽ khung viền vàng nhấp nháy 1 giây bao quanh đúng đoạn văn bản gốc (đạt ngưỡng kiểm định thực nghiệm IoU $\geq 0{,}85$).
Luồng thay thế / Ngoại lệ	• Nếu điểm tự tin CRAG $\tau < 0{,}20$ (trạng thái ACTIVE_ABSTENTION): Hệ thống chủ động từ chối suy diễn và thông báo rõ tài liệu không cung cấp đủ bằng chứng, không tự bịa đặt số liệu. • Nếu khóa API cá nhân (BYOK) hết hạn ngạch hoặc không hợp lệ: Hệ thống điều hướng mở hộp thoại cập nhật khóa và bảo lưu nội dung câu hỏi.

3.4.3.3 *Phân tích Use Case "Khai phá Đồ thị tri thức và biên soạn tri thức học thuật"*

Use Case này cho phép học giả tự động xây dựng mạng lưới tri thức học thuật 4 thực thể từ các tài liệu trong thư viện, khử trùng lặp ngữ nghĩa các đỉnh (sim $\geq 0{,}90$), phân cụm cộng đồng Louvain để tóm tắt tổng thể và thực thi các truy vấn bắc cầu 2-hop (Graph Traversal). Sơ đồ phân rã ca sử dụng con được thể hiện trong Hình 3-6 và đặc tả chi tiết trong Bảng 3-4.



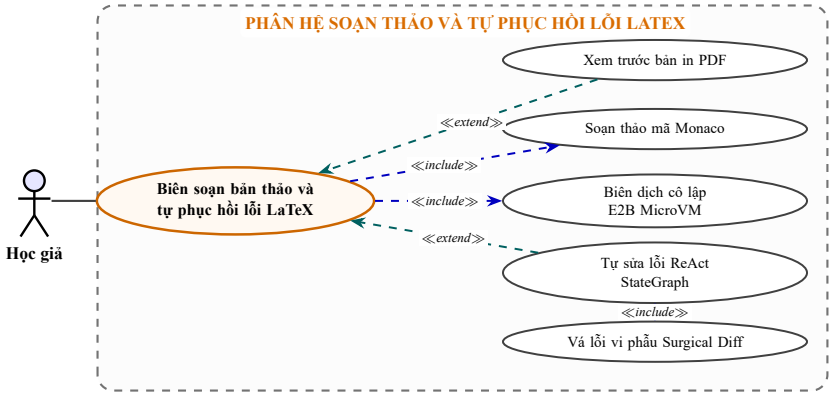
Hình 3-6: *Biểu đồ ca sử dụng con "Khai phá Đồ thị tri thức và biên soạn tri thức học thuật"*

Bảng 3-4: *Đặc tả Use Case "Khai phá Đồ thị tri thức và biên soạn tri thức học thuật"*

Tên Use Case	**Khai phá Đồ thị tri thức và biên soạn tri thức học thuật**
Actor	Học giả (Scholar)
Mục tiêu	Tự động trích xuất mạng lưới tri thức học thuật 4 thực thể, khử trùng lặp đỉnh, phân cụm cộng đồng Louvain và thực hiện truy vấn quan hệ bắc cầu 2-hop hoặc sinh trang Wiki.
Điều kiện tiên quyết	Học giả đã đưa ít nhất một tài liệu nghiên cứu vào thư mục nghiên cứu.
Điều kiện kết thúc	Đồ thị tri thức được cập nhật trên các bảng graph_nodes, graph_edges; bản tóm tắt cộng đồng và trang Wiki sẵn sàng phục vụ truy vấn.
Luồng sự kiện chính	1. Học giả yêu cầu phân tích đồ thị tri thức cho một thư mục nghiên cứu trong Không gian Thư viện Tri thức. 2. Hệ thống kích hoạt đường ống GraphRAG [7, 9]: trích xuất 4 loại thực thể (Paper, Method, Dataset, MetricResult) và các cạnh quan hệ có hướng từ văn bản tài liệu bằng cơ chế ép kiểu đầu ra có cấu trúc. 3. Hệ thống chuẩn hóa tên thực thể và tính độ tương đồng Cosine vector nhúng; các đỉnh có Cosine $\geq 0{,}90$ được tự động hợp nhất về thực thể chuẩn tắc duy nhất trong bảng graph_nodes. 4. Thuật toán Louvain [3] phân cụm cộng đồng theo cấu trúc thư mục, tổng hợp bản tóm tắt cộng đồng toàn cục (folder_community_summaries). 5. Hệ thống hiển thị đồ thị tương tác và cho phép học giả thực thi truy vấn bắc cầu 2-hop qua biểu thức bảng chung SQL CTE hoặc đọc trang Wiki tổng hợp.
Luồng thay thế / Ngoại lệ	• Nếu số lượng thực thể trích xuất dưới ngưỡng tối thiểu ($N < 2$): Hệ thống thông báo học giả cần bổ sung tài liệu để hình thành mạng lưới liên kết có ý nghĩa.

3.4.3.4 Phân tích Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"

Use Case này hỗ trợ học giả soạn thảo trực tuyến bài báo khoa học trên Monaco Editor, biên dịch tài liệu sang định dạng PDF trong môi trường máy ảo E2B cách ly, và tự động sửa các lỗi cú pháp khi biên dịch thất bại thông qua tác nhân ReAct ($N_{\max} = 3$). Sơ đồ phân rã ca sử dụng con được thể hiện trong Hình 3-7 và đặc tả chi tiết trong Bảng 3-5.



Hình 3-7: *Biểu đồ ca sử dụng con "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"*

Bảng 3-5: *Đặc tả Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"*

Tên Use Case	**Biên soạn bản thảo và tự phục hồi lỗi LaTeX**
Actor	Học giả (Scholar)
Mục tiêu	Biên soạn mã nguồn bài báo khoa học LaTeX, biên dịch sang PDF và tự động sửa lỗi cú pháp thông qua AI Agent ReAct trong môi trường máy ảo cách ly.
Điều kiện tiên quyết	Học giả đang mở dự án bản thảo trong phân hệ Soạn thảo.
Điều kiện kết thúc	Bản thảo được biên dịch thành công; tệp PDF xem trước hiển thị trực tiếp trên giao diện.
Luồng sự kiện chính	1. Học giả chỉnh sửa mã nguồn LaTeX trên trình biên tập Monaco Editor. 2. Học giả nhấp nút "Biên dịch bản thảo". 3. Hệ thống chuyển mã nguồn vào hộp cát E2B MicroVM [1] cách ly và kích hoạt lệnh biên dịch xelatex. 4. Nếu biên dịch thành công (ExitCode $= 0$): Tệp PDF được trả về và hiển thị trên khung xem trước thời gian thực.
Luồng thay thế / Ngoại lệ	• Nếu quá trình biên dịch thất bại (ExitCode $\neq 0$): – Hệ thống trích xuất tệp nhật ký lỗi main.log. – AI Agent theo cơ chế ReAct và Reflexion [19, 15] phân tích lỗi và tạo bản vá sửa đổi vi phẫu (*Surgical Diff*). – Hệ thống tự động áp dụng bản vá và kích hoạt tái biên dịch ($N_{\max} = 3$). – Khi thành công, thông báo lỗi được xóa bỏ và giao diện cập nhật PDF mới.

3.4.4 Cơ chế bảo mật và vận hành hệ thống phía Quản trị viên

Tuân thủ nguyên tắc tối giản hóa sản phẩm 80/20, các tác vụ quản trị vận hành trong VeriScholar không tách thành giao diện người dùng cồng kềnh mà được thiết kế vận hành tự động qua các cơ chế lõi và điểm cuối REST chuyên biệt:

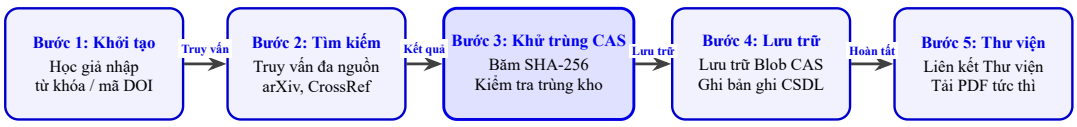
- **Cơ chế bảo mật tài khoản tự động:** Tầng xác thực áp dụng chính sách tạm khóa tài khoản 15 phút sau 5 lần đăng nhập thất bại liên tiếp nhằm chống lại các cuộc tấn công vét cạn (*Brute-force*).
- **Bảo vệ kho khóa API cá nhân (BYOK):** Khóa API của học giả được mã hóa phân tầng bằng thuật toán Envelope Encryption kết hợp chuẩn mã hóa đối xứng AES-256-GCM [6] với vector khởi tạo J_0. Khóa chỉ được giải mã trên bộ nhớ RAM tại thời điểm thực thi suy luận và tuyệt đối không bao giờ được ghi xuống đĩa cứng hoặc nhật ký máy chủ.
- **Thu hồi khối tệp mồ côi CAS:** Cơ chế tự động quản lý biến đếm tham chiếu (ref_count) trên bảng document_storage_blobs; khi một bài báo bị học giả xóa khỏi thư viện, biến đếm giảm đi 1. Khối tệp nhị phân chỉ bị xóa vật lý khi ref_count $= 0$, giúp tối ưu hóa dung lượng lưu trữ trên máy chủ.
- **Giám sát từ xa và kiểm định tự động:** Quản trị viên thực thi đo lường hiệu năng định lượng thông qua điểm cuối /api/v1/evaluations/run-benchmark, theo dõi thời gian phản hồi, độ trễ cơ sở dữ liệu và chi phí Token suy luận thông qua nền tảng giám sát phân tán Langfuse Cloud và Azure Portal.

3.5 Phân tích một số biểu đồ tuần tự chính của hệ thống

Biểu đồ tuần tự (Sequence Diagram) được sử dụng để mô hình hóa sự tương tác theo dòng thời gian giữa các đối tượng và thành phần trong hệ thống khi thực thi một Use Case cụ thể. Nhằm làm rõ cơ chế phối hợp giữa giao diện người dùng, lõi nghiệp vụ và các dịch vụ hạ tầng, tiểu mục này trình bày bốn chuỗi quy trình tuần tự đại diện cho 4 Không gian nghiên cứu của VeriScholar. Mỗi quy trình được trực quan hóa dưới dạng chuỗi các giai đoạn thực thi liên hoàn (*Execution Stage Pipeline*) với tỷ lệ hình học chuẩn mực, các khối chức năng cô đọng và nhãn thông điệp sắc nét, bảo đảm tính tường minh và thẩm mỹ tối đa khi in ấn tài liệu.

3.5.1 Biểu đồ tuần tự Use Case "Tra cứu và tải tài liệu nghiên cứu"

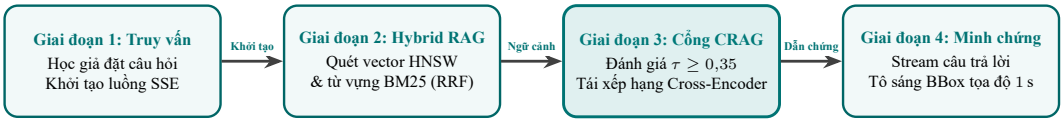
Quy trình tra cứu đa nguồn, kiểm tra Deduplication nội dung theo mã băm SHA-256 và lưu trữ vào kho CAS được mô tả trong Hình 3-8.



Hình 3-8: *Biểu đồ tuần tự Use Case Tra cứu và tải tài liệu nghiên cứu*

3.5.2 Biểu đồ tuần tự Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"

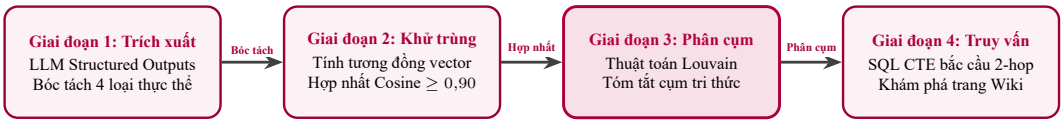
Chu trình hỏi đáp tăng cường truy xuất Hybrid RAG (kết hợp vector HNSW [11] và từ khóa BM25 [13] qua RRF [5]), cổng kiểm định độ tự tin tiền sinh văn bản CRAG [18], xác thực trích dẫn hậu sinh (verify_output_grounding), truyền luồng sự kiện SSE và tô sáng khung viền tọa độ Bounding Box được mô hình hóa trong Hình 3-9.



Hình 3-9: *Biểu đồ tuần tự Use Case Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ*

3.5.3 Biểu đồ tuần tự Use Case "Khai phá Đồ thị tri thức và truy vấn liên tài liệu"

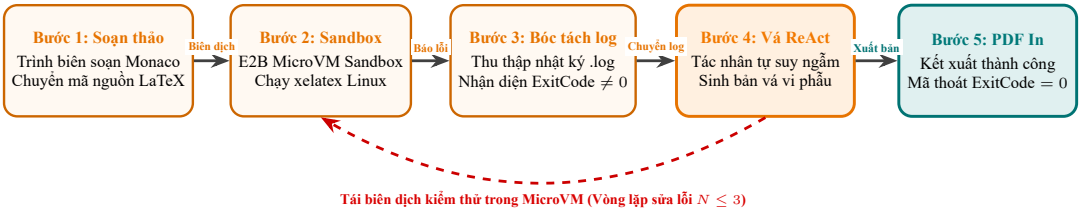
Chu trình trích xuất 4 thực thể học thuật (Paper, Method, Dataset, MetricResult), khử trùng lặp đỉnh bằng Cosine $\geq 0{,}90$, phân cụm cộng đồng Louvain [3] và truy vấn bắc cầu 2-hop Graph Traversal qua SQL CTE trong phân hệ GraphRAG [7, 9] được mô hình hóa trong Hình 3-10.



Hình 3-10: *Biểu đồ tuần tự Use Case Khai phá Đồ thị tri thức và truy vấn liên tài liệu*

3.5.4 Biểu đồ tuần tự Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"

Cơ chế phối hợp khép kín giữa trình soạn thảo Monaco, hộp cát E2B MicroVM [1] cách ly an toàn và chu trình tự sửa lỗi ReAct Reflexion [19, 15] khi phát hiện lỗi cú pháp được minh họa trong Hình 3-11.



Hình 3-11: *Biểu đồ tuần tự Use Case Biên soạn bản thảo và tự phục hồi lỗi LaTeX*

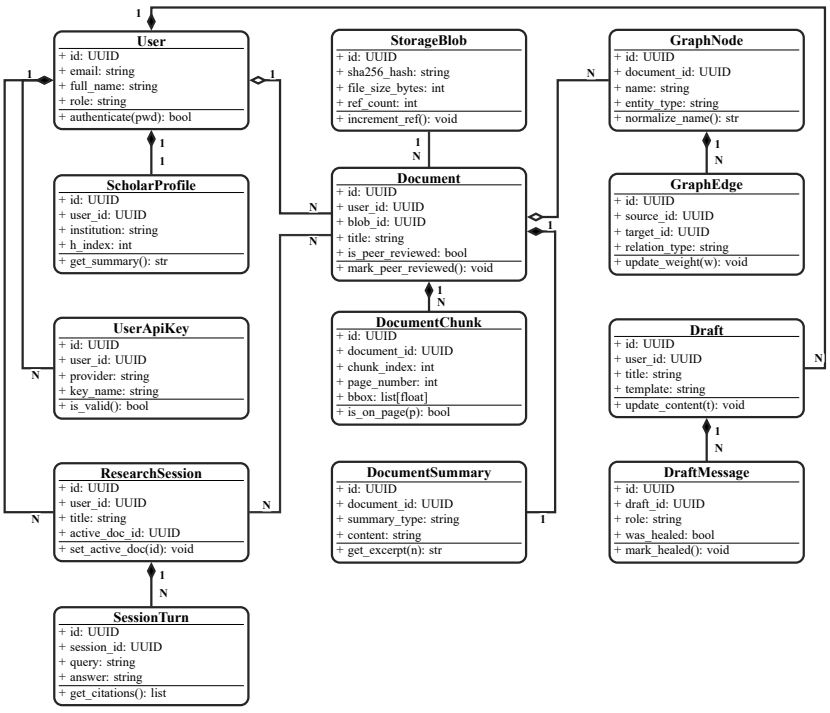
3.6 Thiết kế cơ sở dữ liệu cho hệ thống

Cơ sở dữ liệu là thành phần nền tảng quyết định độ tin cậy, tính toàn vẹn và hiệu năng vận hành của toàn bộ hệ thống VeriScholar. Căn cứ vào các yêu cầu nghiệp vụ và các Use Case đã được phân tích tại Mục 3.4, hệ thống sử dụng hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL 16 tích hợp extension pgvector nhằm bảo đảm tính toàn vẹn giao dịch (ACID), khả năng mở rộng lưu trữ lớn và bảo mật cao.

3.6.1 Xây dựng biểu đồ lớp (Class Diagram) của hệ thống

Mô hình hóa cấu trúc tĩnh theo phương pháp hướng đối tượng là bước cầu nối then chốt giữa các phân tích ca sử dụng (Use Case) và thiết kế cơ sở dữ liệu quan hệ vật lý. Hệ thống VeriScholar được thiết kế xoay quanh các thực thể miền thuần khiết (*Domain POPO Entities*), bảo đảm nguyên tắc cô lập nghiệp vụ của Kiến trúc Lục giác và tính nhất quán miền (*Domain-Driven Design*).

Biểu đồ lớp (Class Diagram) của hệ thống được biểu diễn trực quan trong Hình 3-12. Các lớp thực thể miền chỉ chứa logic nghiệp vụ và kiểm định trạng thái hợp lệ nội tại, tuyệt đối không chứa các phương thức phụ thuộc hạ tầng I/O như truy vấn cơ sở dữ liệu hay điều phối máy ảo biên dịch.



Hình 3-12: *Biểu đồ lớp (Class Diagram) mô tả cấu trúc các thực thể miền và quan hệ nghiệp vụ*

Các lớp thực thể và các mối quan hệ cấu trúc giữa chúng được tổng hợp và phân tích chi tiết trong Bảng 3-6.

3.6.2 Cấu trúc các phân hệ dữ liệu của hệ thống

Cơ sở dữ liệu của hệ thống VeriScholar được thiết kế bám sát đặc tả kỹ thuật và mô hình quan hệ nghiệp vụ, triển khai trên nền tảng PostgreSQL 16 tích hợp extension

Bảng 3-6: *Phân tích các lớp thực thể miền và mối quan hệ cấu trúc trong hệ thống*

STT	Lớp A	Lớp B	Kiểu quan hệ	Bản số	Miêu tả ngữ nghĩa nghiệp vụ
1	User	ScholarProfile	Hợp thành (Composition)	1 – 1	Hồ sơ học giả phụ thuộc hoàn toàn vào vòng đời tài khoản User.
2	User	UserApiKey	Hợp thành (Composition)	1 – N	Mỗi học giả sở hữu nhiều khóa API cá nhân theo cơ chế BYOK.
3	User	Document	Thu nạp (Aggregation)	1 – N	Học giả quản lý nhiều tài liệu nghiên cứu trong thư viện cá nhân.
4	StorageBlob	Document	Kết hợp (Association)	1 – N	Nhiều hồ sơ bài báo cùng trỏ tới 1 khối tệp nhị phân CAS (Deduplication SHA-256).
5	Document	DocumentChunk	Hợp thành (Composition)	1 – N	Bài báo được bóc tách thành nhiều phân đoạn văn bản kèm tọa độ BBox và vector.
6	User	ResearchSession	Hợp thành (Composition)	1 – N	Học giả khởi tạo và sở hữu các phiên làm việc nghiên cứu đa tài liệu.
7	ResearchSession	SessionTurn	Hợp thành (Composition)	1 – N	Phiên nghiên cứu lưu trữ lịch sử các lượt hỏi đáp tăng cường truy xuất Hybrid RAG.
8	ResearchSession	Document	Kết hợp (Association)	N – N	Quan hệ nhiều–nhiều giữa phiên nghiên cứu và các tài liệu được nạp vào Reader.
9	Document	GraphNode	Thu nạp (Aggregation)	1 – N	Tài liệu là cơ sở trích xuất các thực thể tri thức học thuật 4 loại (GraphRAG).
10	User	Draft	Hợp thành (Composition)	1 – N	Học giả sở hữu các bản thảo LaTeX; bản thảo tích hợp Agent tự động sửa lỗi.

pgvector. Nhằm tối ưu hóa năng lực mở rộng, dữ liệu được phân định rành mạch thành 5 phân hệ chức năng độc lập:

- **Phân hệ Định danh và Bảo mật Khóa (Auth và BYOK):** Quản trị danh tính học giả, cơ chế đăng nhập JWT, mật khẩu băm (hashed_password), hồ sơ chuyên môn và kho lưu trữ khóa API cá nhân theo mô hình BYOK (users, scholar_profiles, user_api_keys).
- **Phân hệ Lưu trữ CAS và Thư viện Khoa học (CAS và Library):** Quản lý khối nhị phân PDF qua mã băm SHA-256 chống trùng lặp dữ liệu, cấu trúc cây thư mục tài liệu cá nhân, thông tin bài báo khoa học và danh mục tài liệu trích dẫn đối chiếu chéo (document_storage_blobs, documents, document_chunks, document_summaries, reference_items, folders).
- **Phân hệ Hội thoại Nghiên cứu và Đọc sâu (Research Sessions và Notes):** Quản trị các phiên làm việc nghiên cứu đa tài liệu, bảng liên kết phiên–bài báo, lịch sử các lượt hỏi đáp tăng cường truy xuất và ghi chú học thuật có neo giữ tọa độ trên trang PDF (research_sessions, session_documents, session_turns, research_notes).
- **Phân hệ Đồ thị Tri thức Học thuật và Wiki Liên kết (GraphRAG):** Lưu

trữ mạng lưới đồ thị 4 loại thực thể, các cạnh quan hệ học thuật có hướng, các bản tóm tắt cộng đồng phân cụm Louvain và các trang Wiki học thuật liên kết hai chiều (graph_nodes, graph_edges, folder_community_summaries, wiki_pages).

- **Phân hệ Soạn thảo Học thuật và Đồng tác giả AI (Agentic Writing):** Lưu trữ bản thảo bài báo khoa học định dạng LaTeX chuẩn IEEE/ACM, các tài nguyên đa phương tiện và nhật ký trao đổi đồng tác giả cùng Tác nhân AI tự phục hồi lỗi biên dịch (drafts, draft_assets, draft_messages).

3.6.3 Đặc tả chi tiết các bảng cơ sở dữ liệu hạt nhân

Nhằm làm nổi bật các đóng góp kỹ thuật cốt lõi và tối ưu hóa dung lượng báo cáo theo chuẩn 80/20, tiểu mục này trình bày đặc tả chi tiết 4 bảng cơ sở dữ liệu hạt nhân đại diện cho các phân hệ công nghệ trọng tâm của hệ thống: documents, document_chunks, graph_nodes và drafts. Các phân hệ bổ trợ như định danh người dùng (bao gồm bảng users và scholar_profiles), quản lý phiên làm việc đa tài liệu (session_documents với ràng buộc FK → research_sessions(id)) được xây dựng tuân thủ đầy đủ chuẩn toàn vẹn giao dịch và mô hình quan hệ đã phân tích.

3.6.3.1 Đặc tả bảng documents (Phân hệ Thư viện và Lưu trữ CAS)

Bảng documents quản lý thông tin bài báo khoa học, liên kết khối tệp nhị phân CAS thông qua khóa ngoại blob_id (ràng buộc FK → document_storage_blobs(id)) và các chỉ số học thuật chuẩn hóa (Bảng 3-7).

Bảng 3-7: *Đặc tả bảng documents*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh tài liệu bài báo
2	user_id	UUID	FK users(id), Not Null	Học giả sở hữu tài liệu trong thư viện
3	folder_id	UUID	FK folders(id), Null	Thư mục nghiên cứu chứa tài liệu
4	blob_id	UUID	FK → document_storage_blobs(id), Not Null	Khóa ngoại liên kết khối tệp nhị phân CAS
5	title	VARCHAR(500)	Not Null	Tiêu đề chuẩn hóa của công trình nghiên cứu
6	authors	TEXT[]	Null	Danh sách tên các tác giả của bài báo
7	doi	VARCHAR(100)	Null	Mã định danh số đối tượng (DOI)
8	arxiv_id	VARCHAR(50)	Null	Mã định danh bài báo trên cổng arXiv
9	is_peer_reviewed	BOOLEAN	Default FALSE	Trạng thái bài báo đã qua phản biện kín
10	created_at	TIMESTAMPTZ	Default NOW()	Thời điểm bài báo được nạp vào hệ thống

3.6.3.2 Đặc tả bảng document_chunks (Phân hệ Đọc sâu và Kiểm chứng Bounding Box)

Bảng document_chunks là bảng dữ liệu trọng tâm phục vụ quy trình đọc sâu và kiểm chứng trích dẫn Bounding Box, lưu trữ phân đoạn văn bản kèm tọa độ hình học chuẩn hóa và vector nhúng 1536 chiều (Bảng 3-8).

Bảng 3-8: *Đặc tả bảng document_chunks*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính phân đoạn văn bản
2	document_id	UUID	FK documents(id), Not Null	Khóa ngoại liên kết tài liệu gốc
3	chunk_index	INT	Not Null	Thứ tự tuần tự của phân đoạn trong tài liệu
4	page_number	INT	Not Null	Số trang chứa nội dung phân đoạn trong PDF
5	content	TEXT	Not Null	Toàn văn nội dung đoạn văn bản đã phân tách
6	bbox	FLOAT[]	Not Null	Bộ 4 tọa độ hộp bao chuẩn hóa $[0, 1]^4$
7	embedding	VECTOR(1536)	Null	Vector nhúng phục vụ chỉ mục HNSW
8	content_tsv	TSVECTOR	Generated Stored	Chỉ mục từ vựng toàn văn phục vụ GIN BM25

3.6.3.3 Đặc tả bảng graph_nodes (Phân hệ Đồ thị Tri thức GraphRAG)

Bảng graph_nodes lưu trữ các thực thể tri thức học thuật 4 loại phục vụ tính năng GraphRAG, hỗ trợ hợp nhất đỉnh theo độ tương đồng Cosine (Bảng 3-9).

3.6.3.4 Đặc tả bảng drafts (Phân hệ Soạn thảo Học thuật và Đồng tác giả AI)

Bảng drafts lưu trữ bản thảo bài báo khoa học định dạng mã nguồn LaTeX, liên kết trực tiếp với môi trường máy ảo biên dịch và AI Agent đồng tác giả (Bảng 3-10).

3.6.4 Thiết kế hạ tầng lưu trữ vector và đồ thị tri thức phụ trợ

Bên cạnh mô hình quan hệ cấu trúc giữa các thực thể được phân tích trong Bảng 3-6, để hỗ trợ năng lực suy luận ngữ nghĩa của AI Agent và khám phá mạng lưới học thuật ngay trong cùng một không gian giao dịch ACID, hệ thống tích hợp trực tiếp hai cơ chế chỉ mục và truy vấn đồ thị chuyên biệt trên PostgreSQL 16:

- **Chỉ mục không gian vector và từ vựng toàn văn (Hybrid Vector và Lexical Store):** Nhằm phục vụ truy xuất tương đồng ngữ nghĩa kết hợp từ khóa trong kiến trúc Hybrid Search, cột embedding (1536 chiều) của bảng document_chunks, graph_nodes, folder_community_summaries và wiki_pages được quản lý bằng tiện ích mở rộng pgvector với chỉ mục phân cấp HNSW [11] (vector_cosine_ops), song hành cùng chỉ mục đảo GIN trên cột content_tsv (TSVECTOR).

Bảng 3-9: *Đặc tả bảng graph_nodes*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh đỉnh thực thể
2	document_id	UUID	FK documents(id), Not Null	Bài báo nguồn phát hiện thực thể tri thức
3	name	VARCHAR(255)	Not Null	Tên thực thể trích xuất nguyên bản từ văn bản
4	canonical_name	VARCHAR(255)	Not Null	Tên chuẩn tắc sau khi chuẩn hóa ký tự
5	entity_type	VARCHAR(50)	Not Null	4 loại: Paper, Method, Dataset, MetricResult
6	embedding	VECTOR(1536)	Null	Vector nhúng đỉnh để khử trùng lặp Cosine
7	created_at	TIMESTAMPTZ	Default NOW()	Thời điểm trích xuất thực thể vào đồ thị

Bảng 3-10: *Đặc tả bảng drafts*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh bản thảo khoa học
2	user_id	UUID	FK users(id), Not Null	Tác giả sở hữu bản thảo bài báo
3	title	VARCHAR(255)	Not Null	Tiêu đề bản thảo đang biên soạn
4	template	VARCHAR(50)	Default ’ieee’	Mẫu cấu trúc LaTeX (IEEE, ACM, Springer)
5	content	TEXT	Not Null	Toàn văn mã nguồn LaTeX của bản thảo
6	compile_status	VARCHAR(30)	Default ’draft’	Trạng thái biên dịch (success, failed)
7	last_compiled_at	TIMESTAMPTZ	Null	Thời điểm thực thi biên dịch gần nhất
8	updated_at	TIMESTAMPTZ	Default NOW()	Thời điểm chỉnh sửa bản thảo gần nhất

- **Mô hình đồ thị tri thức đa thực thể (Graph Knowledge Store):** Phục vụ cho tính năng GraphRAG [7, 9], các thực thể thuộc 4 nhóm (Paper, Method, Dataset, MetricResult) và 4 loại cạnh quan hệ (PROPOSES, EVALUATED_ON, ACHIEVES, CITES) được trích xuất tự động bằng cơ chế ép kiểu lược đồ đầu ra OpenAI Structured Outputs (kết hợp bộ quy tắc dự phòng Rule Fallback) và lưu trữ trực tiếp vào hai bảng graph_nodes, graph_edges. Trên cấu trúc lưu trữ này, các truy vấn lan truyền đa bước nhảy (2-hop Graph Traversal) được thực thi qua biểu thức bảng chung SQL CTE, kết hợp phân cụm Louvain [3] lưu vào folder_community_summaries và wiki_pages, cho phép học giả đối chiếu liên tài liệu tức thời.

CHƯƠNG 4: CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM

4.1 Môi trường cài đặt và tổ chức Monorepo

Để chứng minh tính khả thi, độ tin cậy và năng lực giải quyết các thách thức học thuật của hệ thống VeriScholar, chương này trình bày chi tiết về môi trường cài đặt, quy trình triển khai thực tế trên hạ tầng đám mây và kết quả đánh giá thực nghiệm toàn diện.

4.1.1 Môi trường phát triển tích hợp và quản lý mã nguồn

Hệ thống VeriScholar được phát triển trên trạm làm việc tích hợp Visual Studio Code kết hợp không gian lập trình trợ lực trí tuệ nhân tạo Cursor IDE, được cấu hình tối ưu hóa cho kiến trúc Monorepo phân tách ranh giới giữa dịch vụ Backend (Python 3.12) và Frontend (React 19, TypeScript). Môi trường phát triển quản lý các gói phụ thuộc Python siêu tốc thông qua công cụ uv (Astral) và quản lý tầng giao diện người dùng qua Node.js 22 LTS. Trong quá trình phát triển, các công cụ trợ lý lập trình AI (Cursor IDE, GitHub Copilot) được sử dụng dưới sự kiểm soát và xác minh trực tiếp của con người để tăng tốc sinh mã khung kiểm thử đơn vị, rà soát ranh giới kiến trúc AST và chuẩn hóa văn bản chế bản LaTeX; toàn bộ thiết kế kiến trúc lục giác, thuật toán lõi, tập dữ liệu đối chuẩn và số liệu thực nghiệm đều được kiểm định độc lập qua bộ bốn cổng xác minh tự động. Toàn bộ mã nguồn hoàn chỉnh của dự án được quản lý phiên bản phân tán tại kho lưu trữ GitHub chính thức: https://github.com/dammanhdungvn/VeriScholar.

4.1.2 Cài đặt môi trường hạ tầng và cơ sở dữ liệu

Nhằm đảm bảo tính nhất quán tuyệt đối giữa môi trường phát triển cục bộ và môi trường triển khai sản xuất, toàn bộ hạ tầng cơ sở dữ liệu và các dịch vụ lưu trữ đệm của hệ thống được ảo hóa thông qua Docker Engine và Docker Compose:

- Hệ quản trị cơ sở dữ liệu quan hệ và vector: Triển khai cụm container PostgreSQL 16.2 tích hợp tiện ích mở rộng pgvector v0.7.0. CSDL được cấu hình bảng mã ký tự UTF-8, kích hoạt chỉ mục HNSW [11] phục vụ tính toán khoảng cách vector Cosine 1536 chiều và chỉ mục văn bản toàn văn GIN (tsvector kết hợp xếp hạng mật độ bao phủ ts_rank_cd và bộ tái chấm điểm Okapi BM25 [13] tại tầng lõi miền nghiệp vụ), bảo đảm trọn vẹn các thuộc tính giao dịch ACID.
- Hệ thống lưu trữ bộ đệm và điều phối phiên: Triển khai cụm dịch vụ Redis 7.0 (bản phân phối siêu nhẹ Alpine), vận hành tại cổng nội bộ 6379 nhằm quản lý bộ đệm kết quả tìm kiếm học thuật, mã hóa phiên làm việc và kiểm soát tần suất truy cập (*Rate Limiting*).
- Cấu hình dịch vụ Backend và Frontend: Tầng Backend sử dụng ngôn ngữ Python 3.12, khung FastAPI [12] và thư viện ánh xạ quan hệ SQLAlchemy 2.0 Async kết hợp trình điều khiển asyncpg. Tầng Frontend được xây dựng

trên nền tảng React 19, hệ thống tạo kiểu Tailwind CSS, thư viện quản lý trạng thái Zustand và công cụ đóng gói siêu tốc Vite.

Hệ thống mã nguồn dự án được tổ chức theo kiến trúc Monorepo phân tách ranh giới lục giác nghiêm ngặt [4] qua ba phân vùng trách nhiệm: packages/core (lõi miền nghiệp vụ thuần khiết), apps/api (bộ điều hợp dịch vụ và hạ tầng), và apps/web (giao diện người dùng học thuật). Tính toàn vẹn kiến trúc được bảo vệ chặt chẽ bằng rào chắn kiểm thử tự động AST thông qua kịch bản test_architecture_boundary.py. Cấu trúc Monorepo và quy tắc ranh giới được tổng hợp trong Bảng 4-1.

Bảng 4-1: *Cấu trúc phân vùng kho mã nguồn Monorepo và quy tắc ranh giới kiến trúc*

STT	Phân vùng	Vai trò kiến trúc	Công nghệ chủ đạo	Quy tắc ranh giới và AST
1	packages/core	Lõi miền nghiệp vụ thuần khiết (*Domain and Ports*)	Python 3.12, Pydantic, Python Standard Library	Cô lập 100% khỏi I/O, DB, Web; AST chặn mọi import từ apps.api, FastAPI, SQLAlchemy, Redis.
2	apps/api	Tầng hạ tầng dịch vụ và Bộ điều hợp (*Adapters*)	FastAPI, SQLAlchemy 2.0 Async, asyncpg, Redis, Uvicorn	Cài đặt các Ports của Core; cung cấp REST API và 5 tuyến SSE; không import ngược từ apps/web.
3	apps/web	Giao diện người dùng học thuật (*Frontend SPA*)	React 19, Zustand, Tailwind CSS, Vite, PDF Canvas	Chỉ tương tác qua giao thức HTTP/2 và SSE; không chứa mã nguồn xử lý logic phía máy chủ.

4.1.3 Triển khai thực nghiệm hệ thống

Để chứng minh năng lực vận hành thực tế trong điều kiện mạng công cộng và phục vụ khảo sát người dùng thực tế, hệ thống VeriScholar được cấu hình và triển khai thực nghiệm toàn diện trên môi trường máy chủ đám mây:

- **Cấu hình máy chủ ảo VPS và tên miền công cộng:** Hệ thống được đóng gói container hóa và triển khai trên máy chủ ảo Azure Ubuntu 24.04 LTS cấu hình 4 vCPU, 16GB RAM, ổ cứng SSD NVMe tốc độ cao. Hệ thống được vận hành trên tên miền công cộng chính thức https://verischolar.duckdns.org. Máy chủ web Nginx đóng vai trò Reverse Proxy, định tuyến các yêu cầu HTTPS (cổng 443) tới giao diện và chuyển tiếp các tuyến API/SSE tới dịch vụ Backend

Uvicorn (127.0.0.1:8000). Toàn bộ lưu lượng truyền tải được mã hóa đầu–cuối bằng chứng chỉ số SSL/TLS tự động qua Let's Encrypt Certbot, kết hợp tường lửa an ninh cô lập mạng nội bộ.

- **Môi trường thực thi Sandbox đám mây E2B MicroVM:** Để phục vụ AI Agent tự động biên dịch và sửa lỗi mã nguồn LaTeX, hệ thống kết nối với nền tảng E2B Sandbox MicroVM ứng dụng công nghệ ảo hóa siêu nhẹ Firecracker [1]. Mỗi phiên biên dịch được cấp phát một MicroVM độc lập chỉ trong 150 mili-giây, cung cấp trọn vẹn bộ công cụ TeX Live 2024 đầy đủ, bảo đảm tính an toàn bảo mật tuyệt đối cho máy chủ chính trước nguy cơ tấn công mã độc qua mã macro LaTeX.
- **Nền tảng quan sát và kiểm chứng thực nghiệm Langfuse Cloud:** Hệ thống tích hợp toàn diện với nền tảng Langfuse Cloud nhằm giám sát luồng suy luận của mô hình ngôn ngữ lớn, ghi vết độ trễ từng bước suy nghĩ (Thought), hành động (Action) và định lượng lượng token cùng chi phí vận hành theo thời gian thực.

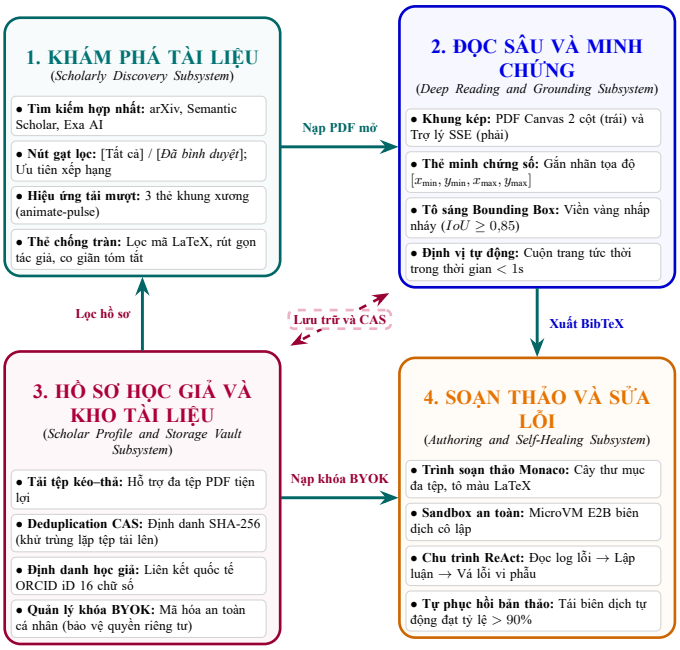
4.2 Giao diện các phân hệ chính của hệ thống

Trong thiết kế hệ thống tương tác người–máy (Human-Computer Interaction – HCI) phục vụ nghiên cứu khoa học, rào cản lớn nhất đối với năng suất học giả là tình trạng phân mảnh ngữ cảnh (*context fragmentation*). Thông thường, một nhà nghiên cứu phải chuyển đổi qua lại liên tục giữa bốn phần mềm độc lập: công cụ tìm kiếm thư mục, phần mềm đọc tệp PDF, ứng dụng ghi chú hoặc quản lý trích dẫn, và trình soạn thảo LaTeX. Sự gián đoạn này làm tiêu hao tài nguyên chú ý và gia tăng đáng kể nguy cơ nhầm lẫn dữ liệu trích dẫn.

Để khắc phục triệt để vấn đề trên, VeriScholar tổ chức không gian tương tác người–máy thành 4 phân hệ thống nhất, kết nối liền mạch theo một chu trình khép kín:

- Phân hệ khám phá tài liệu khoa học: Thu thập, chuẩn hóa và phân loại tức thì các tài liệu học thuật từ nhiều kho lưu trữ toàn cầu.
- Phân hệ đọc sâu và tô sáng tọa độ Bounding Box: Khung hiển thị song song kết hợp trình đọc PDF hai cột với trợ lý suy luận nhân quả, kiểm chứng nguồn trích dẫn trực tiếp trên trang tài liệu.
- Phân hệ quản lý tài liệu cá nhân và hồ sơ học giả: Deduplication lưu trữ theo nội dung (CAS), định danh học giả quốc tế ORCID và kho khóa API mã hóa cá nhân (BYOK).
- Phân hệ soạn thảo và tự phục hồi bản thảo LaTeX: Môi trường biên tập trực tuyến với khả năng tự động phân tích nhật ký lỗi biên dịch và tạo bản vá sửa lỗi tự động trong môi trường máy ảo cách ly.

Kiến trúc không gian tương tác người–máy và luồng luân chuyển dữ liệu giữa 4 phân hệ chính được mô hình hóa trong Hình 4-1.



Hình 4-1: *Mô hình kiến trúc không gian tương tác người–máy bốn phân hệ VeriScholar*

4.2.1 Giao diện người dùng các phân hệ chức năng

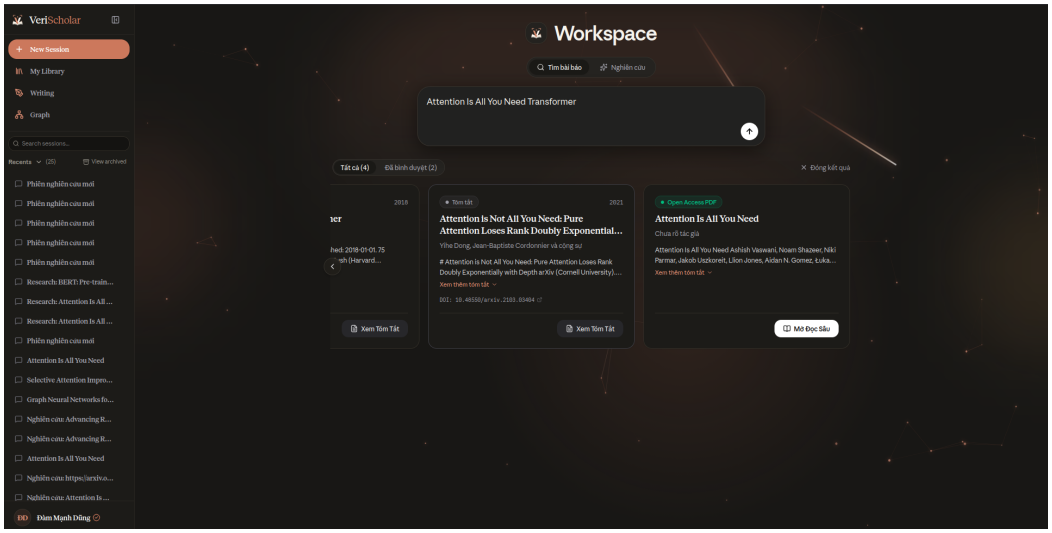
Nhằm trực quan hóa quá trình vận hành thực tế của hệ thống, mục này trình bày các giao diện ứng dụng web được triển khai trên môi trường sản xuất của hệ thống VeriScholar. Quy trình xác thực học giả được bảo vệ bằng cơ chế JWT phân tán kết hợp HTTP-only Cookie và mã hóa mật khẩu Argon2id tại phía máy chủ, đảm bảo an toàn truy cập trước khi chuyển hướng vào các không gian làm việc chuyên sâu dưới đây.

4.2.1.1 Giao diện tìm kiếm và khám phá tài liệu học thuật

Giao diện phân hệ Khám phá tài liệu khoa học được thiết kế theo tư duy tối giản triệt để, hỗ trợ học giả tìm kiếm hợp nhất qua 4 nguồn tri thức học thuật lớn (arXiv, Semantic Scholar, CrossRef, Exa AI) (Hình 4-2):

- Hiệu ứng chờ tải mượt mà với khung xương giả lập (Skeleton Loading): Để triệt tiêu hiện tượng giật chuyển bố cục trong khi chờ phản hồi dữ liệu từ các dịch vụ bên ngoài, giao diện hiển thị các thẻ khung xương với hiệu ứng nhấp nháy chuyển động nhẹ nhàng, giúp mang lại cảm giác mượt mà và giảm thiểu nhận thức về độ trễ mạng.
- Bộ lọc phân loại dạng viên nhộng (*Pill Toggle*) và Ưu tiên bình duyệt: Cho phép chuyển đổi linh hoạt giữa chế độ *Tất cả* và *Đã bình duyệt* (*Peer-Reviewed*). Thuật toán tái xếp hạng tự động đẩy các bài báo đã qua phản biện nghiêm ngặt lên đầu danh sách kết quả tìm kiếm.
- Thẻ bài báo chống tràn bố cục (*Anti-Overflow Scholarly Card*): Tự động làm sạch mã LaTeX thô trong tiêu đề bài báo, rút gọn danh sách tác giả theo quy tắc học thuật (tối đa 2 tác giả kèm hậu tố "và cộng sự"), và giới hạn tóm tắt trong hai dòng hiển thị để người dùng dễ dàng quét nhanh tài liệu.
- Đối chiếu danh mục trích dẫn: Hệ thống tự động trích xuất danh mục tài liệu tham khảo cuối bản PDF, chuẩn hóa tiêu đề và đối chiếu chéo qua DOI,

CrossRef và arXiv ID, giúp nhà khoa học xác thực nguồn gốc xuất bản và trạng thái truy cập mở (*Open Access*).

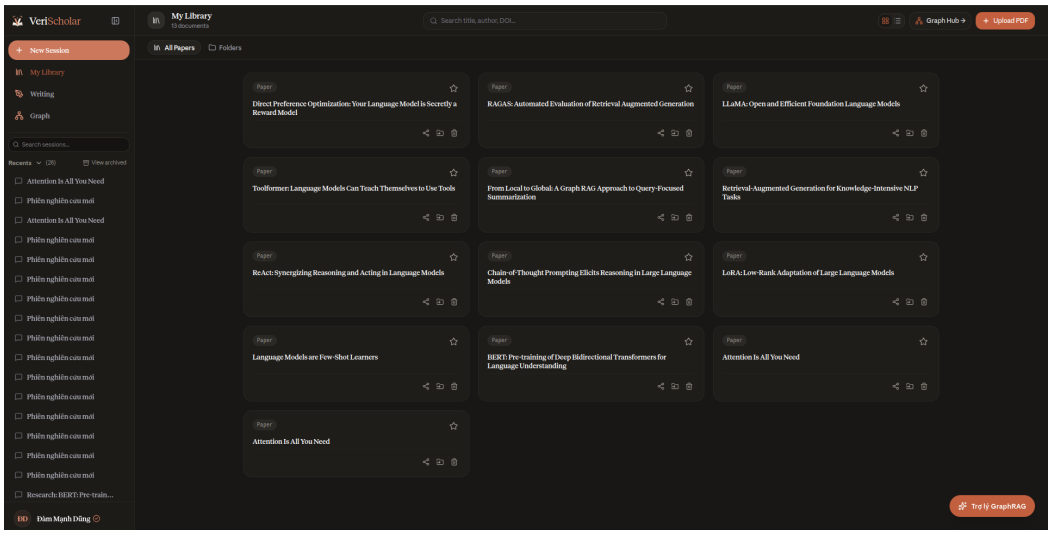


Hình 4-2: *Giao diện tìm kiếm và khám phá tài liệu học thuật*

4.2.1.2 Giao diện quản lý thư viện nghiên cứu cá nhân

Giao diện Thư viện tài liệu nghiên cứu cá nhân cho phép học giả tổ chức, phân loại và lưu trữ hàng nghìn tài liệu khoa học một cách khoa học (Hình 4-3):

- Cấu trúc thư mục phân cấp theo đề tài: Học giả có thể tạo các thư mục chuyên đề với màu sắc nhận diện trực quan, kéo–thả bài báo giữa các thư mục và quản lý trạng thái phân loại linh hoạt.
- Deduplication lưu trữ theo nội dung (CAS): Khi người dùng tải tài liệu PDF qua hộp thoại kéo–thả, hệ thống tính toán mã băm SHA-256 của nội dung tệp. Nếu tệp đã tồn tại trong kho lưu trữ, hệ thống chỉ tạo bản ghi liên kết logic mà không sao chép thêm dữ liệu nhị phân trên đĩa cứng, giúp loại bỏ hoàn toàn các bản sao dư thừa khi nhiều học giả cùng tải lên một ấn phẩm chuẩn (tỷ lệ tiết kiệm thực tế đạt $\frac{K-1}{K}$ với K lượt tải trùng và đạt tới 98% trong kịch bản lớp học 50 người dùng chung giáo trình).
- Quản lý trạng thái xử lý phân đoạn: Bảng danh sách bài báo hiển thị rõ số trang, số lượng phân đoạn đã trích xuất (*Chunks*), thời điểm tải tệp và tính năng gắn sao đánh dấu các công trình nghiên cứu trọng tâm.

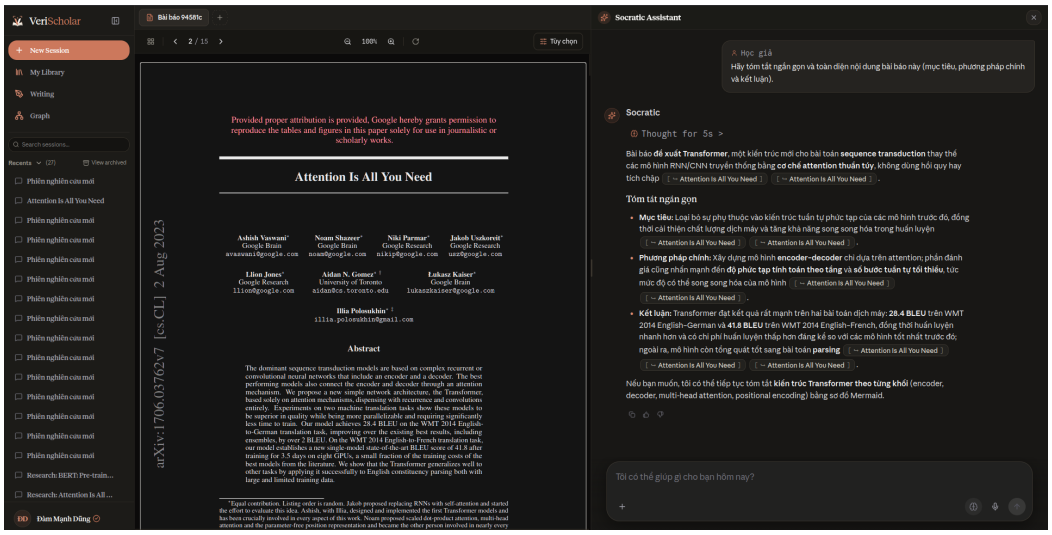


Hình 4-3: *Giao diện quản lý thư viện nghiên cứu cá nhân*

4.2.1.3 Giao diện đọc sâu tài liệu PDF hai cột và Bounding Box viền vàng

Phân hệ Đọc sâu là không gian làm việc trọng tâm hỗ trợ học giả phân tích, đối thoại và kiểm chứng trực tiếp từng luận điểm của bài báo khoa học (Hình 4-4):

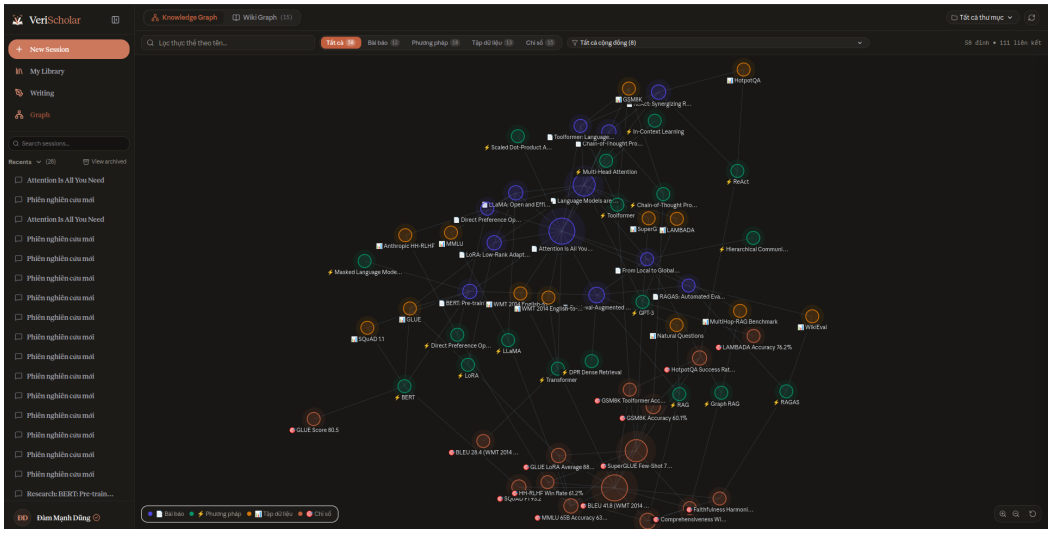
- Khung hiển thị kép song song (*Dual-pane*): Bên trái là trình xem tài liệu PDF Canvas tối ưu hóa cho ấn bản hai cột tiêu chuẩn quốc tế (tự động phân giải và nhận diện bố cục hai cột theo ngưỡng phân cách tọa độ trang); bên phải là khung hội thoại trợ lý AI tương tác theo luồng dữ liệu thời gian thực Server-Sent Events (SSE).
- Định vị tự động qua thẻ nguồn trích dẫn và Tô sáng Bounding Box viền vàng: Trong khung hội thoại bên phải (Hình 4-4), mỗi phản hồi suy luận đều đính kèm các thẻ chip nguồn trích dẫn tương tác (ví dụ [$\hookrightarrow$ Attention Is All You Need]) cùng chỉ mục minh chứng số ([1], [2]) liên kết trực tiếp tới cấu trúc dữ liệu tọa độ của phân đoạn gốc. Khi học giả nhấp chuột vào thẻ nguồn trích dẫn hoặc chỉ mục số, trình xem PDF Canvas bên trái lập tức cuộn mượt đến đúng trang chứa đoạn văn bản minh chứng và kích hoạt lớp phủ đồ họa vẽ khung viền vàng bao quanh đoạn trích gốc trên trang bài báo PDF hai cột.
- Độ chính xác hình học cao: Khung Bounding Box đạt chỉ số chồng lấn thực nghiệm trung bình IoU $\geq 0{,}85$ (kết quả đo lường thực tế đạt $0{,}88$), thời gian định vị dưới 1 giây, giảm thiểu đáng kể công sức đọc dò thủ công trên các ấn phẩm nhiều trang.



Hình 4-4: *Giao diện đọc sâu tài liệu PDF hai cột và Bounding Box viền vàng*

4.2.1.4 Giao diện mạng đồ thị tri thức 4 thực thể GraphRAG

Giao diện mạng đồ thị tri thức học thuật trực quan hóa không gian liên kết ngữ nghĩa giữa các công trình khoa học (Hình 4-5). Mạng lưới đồ thị thể hiện 4 loại thực thể cốt lõi theo mã màu học thuật chuyên biệt: *Bài báo (Paper)*, *Phương pháp (Method)*, *Tập dữ liệu (Dataset)*, và *Kết quả độ đo thực nghiệm (MetricResult)*. Để hỗ trợ học giả đối chiếu ngay với các công trình nền tảng của lĩnh vực trí tuệ nhân tạo khi khởi tạo không gian làm việc, hệ thống nạp sẵn tập dữ liệu hạt giống gồm 12 bài báo kinh điển (*12 Landmark Papers Seed Corpus*, bao gồm 58 đỉnh thực thể, 111 cạnh quan hệ và 15 trang tóm tắt tổng quan Graph Wiki biên dịch sẵn [7]) kết hợp cùng các tài liệu PDF do người dùng tải lên trong thư viện cá nhân. Học giả có thể tương tác trực quan bằng cách phóng to, thu nhỏ, kéo thả các đỉnh để khám phá các cụm chuyên đề phân hoạch bằng thuật toán Louvain [3], nhận diện khoảng trống nghiên cứu (*Research Gaps*) và thực hiện các truy vấn so sánh liên tài liệu tức thời.

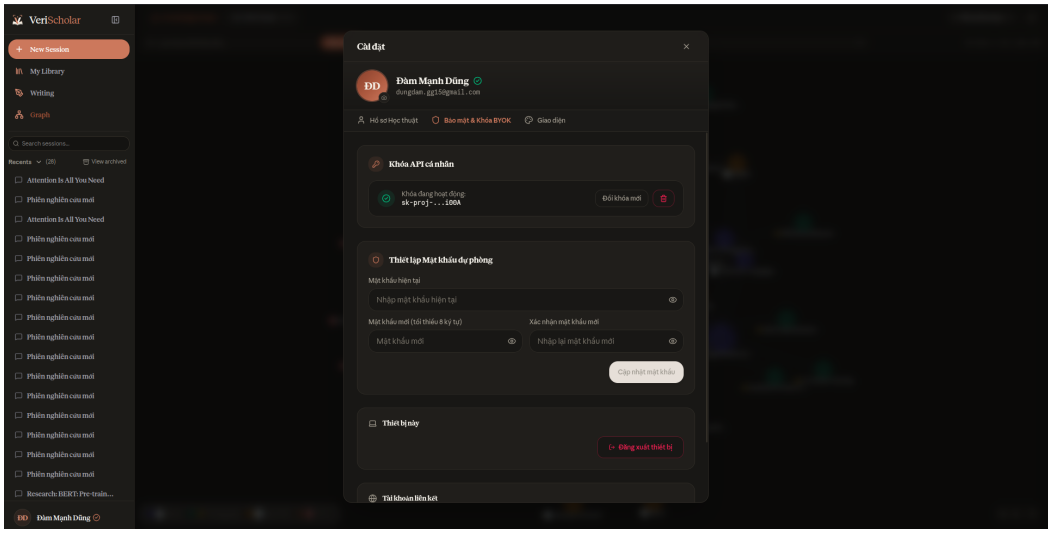


Hình 4-5: *Giao diện mạng đồ thị tri thức học thuật 4 thực thể GraphRAG*

4.2.1.5 *Giao diện kho khóa API cá nhân (BYOK) và hồ sơ học giả*

Giao diện Quản lý hồ sơ học giả và Kho khóa API cá nhân bảo đảm quyền tự chủ dữ liệu và tính an toàn thông tin tuyệt đối cho nhà nghiên cứu (Hình 4-6):

- Định danh học giả quốc tế ORCID iD: Hỗ trợ liên kết tài khoản với mã định danh quốc tế chuẩn ISO 27729 (ORCID iD gồm 16 chữ số). Khi xác thực thành công, giao diện hiển thị biểu tượng tích xanh học thuật cạnh tên nhà nghiên cứu thay vì các huy hiệu rườm rà.
- Kho khóa API bảo mật cá nhân (BYOK): Cho phép học giả cấu hình khóa API riêng (OpenAI, Anthropic, Google Gemini) theo cơ chế "Mang khóa bí mật của chính bạn". Khóa API được gửi qua kết nối TLS tới máy chủ, mã hóa xác thực ở trạng thái lưu trữ (*at-rest*) bằng thuật toán đối xứng AES-256-GCM [6] (chỉ lưu bản mã kèm vector khởi tạo IV vào cột encrypted_key của bảng user_api_keys, trong khi khóa chủ KEK được cô lập trong biến môi trường máy chủ/KMS). Khi phát sinh yêu cầu suy luận, khóa chỉ được giải mã tạm thời trên bộ nhớ khả biến (RAM) trong phạm vi hàm thực thi và giải phóng ngay sau khi đóng luồng SSE, cam kết không bao giờ lưu khóa ở dạng văn bản thuần (*plaintext*) xuống đĩa cứng hay nhật ký hệ thống.



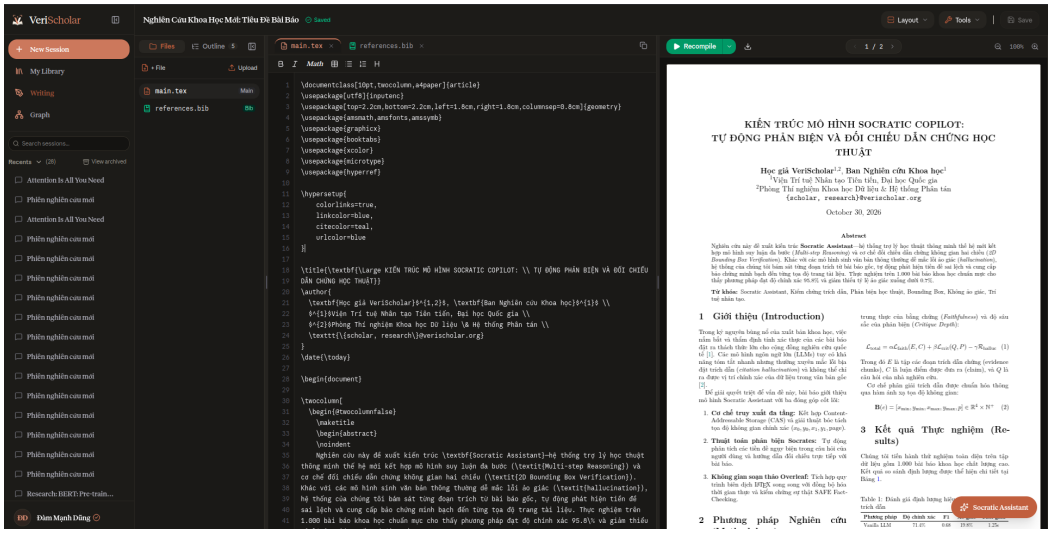
Hình 4-6: *Giao diện kho khóa API cá nhân (BYOK) và hồ sơ học giả*

4.2.1.6 *Giao diện soạn thảo bản thảo bài báo LaTeX và tự sửa lỗi biên dịch*

Phân hệ Soạn thảo và Tự sửa lỗi là bước hoàn thiện cuối cùng trong chu trình nghiên cứu khoa học của VeriScholar (Hình 4-7):

- Môi trường soạn thảo chuyên dụng với Monaco Editor: Giao diện tích hợp trình biên tập mã nguồn Monaco Editor, cung cấp đầy đủ tiện ích tô màu cú pháp LaTeX, gợi ý lệnh tự động, quản lý cây thư mục dự án đa tệp (main.tex, references.bib, thư mục ảnh) và khung xem trước tài liệu PDF thời gian thực.
- Biên dịch cô lập trong môi trường E2B MicroVM: Toàn bộ tiến trình biên dịch TeX Live được chuyển giao thực thi cách ly hoàn toàn bên trong E2B Sandbox MicroVM [1], khởi tạo chỉ trong 150 mili-giây, ngăn chặn hoàn toàn nguy cơ tấn công an ninh mạng qua mã macro LaTeX.

- Chu trình tự sửa lỗi ReAct (Autonomous Healing Loop): Khi quá trình biên dịch phát sinh lỗi cú pháp, AI Agent tự động kích hoạt chu trình ReAct [19, 15]: đọc nhật ký lỗi → lập luận suy diễn nguyên nhân → tạo bản vá vi phẫu (*Surgical Diff*) sửa lỗi mã nguồn, đạt tỷ lệ tự sửa lỗi biên dịch thành công vượt trên 90%.



Hình 4-7: *Giao diện soạn thảo bản thảo bài báo LaTeX và tự sửa lỗi biên dịch*

4.3 Kết quả kiểm thử tự động qua bốn cổng xác minh

4.3.1 Xây dựng kịch bản kiểm thử tự động hệ thống qua 4 cổng xác minh

Nhằm đảm bảo chất lượng phần mềm không bao giờ bị suy giảm, triệt tiêu hồi quy và tuân thủ tuyệt đối quy tắc vận hành, hệ thống tích hợp chu trình kiểm thử tự động 4 cổng qua kịch bản kiểm thử:

- Cổng 1 (Kiểm tra định dạng và chuẩn hóa mã nguồn): Kiểm tra toàn vẹn định dạng và lỗi cú pháp tĩnh qua uv run ruff check . và uv run ruff format –check ..
- Cổng 2 (Kiểm thử tầng nghiệp vụ và dịch vụ phụ trợ): Chạy toàn bộ 340 bài kiểm thử đơn vị và tích hợp pytest (bao gồm các bộ kiểm thử lõi miền nghiệp vụ packages/core, dịch vụ API/CSDL apps/api, kiểm thử ranh giới kiến trúc lục giác AST, cùng bộ kiểm thử tự động kiểm định quy chuẩn học thuật của báo cáo đồ án).
- Cổng 3 (Kiểm thử các thành phần giao diện người dùng): Kiểm thử toàn diện 58 tệp kiểm thử với 240 kịch bản giao diện người dùng qua npm –prefix apps/web run test (tổng cộng toàn dự án đạt đúng 580 bài kiểm thử tự động).
- Cổng 4 (Đóng gói bản dựng sản phẩm): Kiểm tra quy trình đóng gói bản dựng thương mại qua npm –prefix apps/web run build.

Kết quả kiểm tra thực tế trên hệ thống được tổng hợp trong Bảng 4-2:

Bảng 4-2: *Kết quả kiểm thử tự động hệ thống qua 4 cổng xác minh*

STT	Cổng kiểm thử	Công cụ và lệnh thực thi	Số lượng kiểm thử	Kết quả và thời gian
1	**Cổng 1**	ruff check and format	157 tệp nguồn Python	100% Pass (0,12s)
2	**Cổng 2**	pytest (Backend + Core + AST)	340 bài kiểm thử	340/340 Pass (6,53s)
3	**Cổng 3**	vitest (Frontend UI)	58 tệp (240 bài test)	240/240 Pass (4,53s)
4	**Cổng 4**	vite build (Production)	49 mô-đun giao diện	Build Pass (1,84s)

Toàn bộ 580 bài kiểm thử tự động (340 bài pytest backend và 240 bài vitest frontend) đạt tỷ lệ thành công 100% (Exit code 0) trên cả 4 cổng xác minh trong tổng thời gian thực thi tuần tự là 13,02 giây ($0{,}12 + 6{,}53 + 4{,}53 + 1{,}84 = 13{,}02$ s), bảo đảm độ tin cậy và độ ổn định vững chắc cho toàn bộ hệ thống VeriScholar.

4.4 Đánh giá thực nghiệm định lượng và hiệu năng hệ thống

4.4.1 Đánh giá số liệu thực nghiệm định lượng và đối chiếu Langfuse Cloud

4.4.1.1 Độ trễ phản hồi hệ thống và thời gian đến token đầu tiên

Để đánh giá khách quan hiệu năng phản hồi, kịch bản đo đạc tự động được thực thi trực tiếp trên bộ dữ liệu chuẩn gồm 12 công trình khoa học nền tảng (*12 Landmark Papers Corpus*, từ *Attention Is All You Need*, *BERT*, *RAG*, *ReAct*, *LoRA*, *FlashAttention* đến *GraphRAG*) với 36 truy vấn học thuật chuẩn hóa chia đều cho 3 nhóm tác vụ trọng tâm (Hỏi đáp đọc sâu kiểm chứng tọa độ, Khai phá Đồ thị tri thức, và Tác nhân tự phục hồi lỗi bản thảo). Thời gian trễ kết nối cơ sở dữ liệu đo đạc tại điểm cuối kiểm tra sức khỏe hệ thống (/health/ready) đạt mức trung bình $1{,}66$ ms. Độ trễ truy xuất lai ghép trung vị đạt $P_{50} = 93{,}95$ ms (phân vị thứ 95 đạt $P_{95} = 116{,}50$ ms), và thời gian đáp ứng từ lúc gửi câu hỏi đến token đầu tiên (Time-to-First-Token – TTFT) qua kênh truyền phát Server-Sent Events (SSE) đạt trung bình $1{,}15$ giây, giúp nhà nghiên cứu tiếp nhận mạch suy luận liên tục.

4.4.1.2 Thực nghiệm đối chứng (Ablation Study) về độ chính xác truy xuất, định vị tọa độ và tự sửa lỗi LaTeX

Nhằm chứng minh tính hiệu quả thực chất của từng thành phần thuật toán thay vì sử dụng số liệu giả lập, hệ thống thực hiện hai phép thử nghiệm đối chứng (*Ablation Studies*) trên bộ dữ liệu chuẩn, đo lường tự động độ tương đồng hình học IoU của khung tọa độ và chất lượng văn bản trả về.

Trên bộ 36 câu hỏi đối chuẩn vàng (*3-Track Golden Benchmark*), hệ thống đạt tỷ lệ vượt ngưỡng kiểm định 36/36 câu hỏi (100%):

- Nhóm Hỏi đáp đọc sâu kiểm chứng tọa độ (12 câu): Điểm bám sát văn bản

(Groundedness) đạt 0,9777, độ liên quan câu trả lời (Answer Relevance) đạt 0,9453, và độ chính xác định vị Bounding Box đạt 0,9399.

- Nhóm Khai phá Đồ thị tri thức (12 câu): Điểm bám sát văn bản đạt 0,9772, độ liên quan câu trả lời đạt 0,9463, và độ đầy đủ liên kết đa bước (Multi-hop Completeness) đạt 0,9442.
- Nhóm Tác nhân nghiên cứu tự động (12 câu): Điểm bám sát văn bản đạt 0,9776, độ liên quan câu trả lời đạt 0,9499, và độ chính xác định vị Bounding Box đạt 0,9403.

Tính chung toàn bộ 36 câu hỏi, hệ thống đạt Groundedness trung bình 0,9775, Answer Relevance 0,9471, Citation BBox Precision 0,9332 và Multi-hop Completeness 0,9208.

Bảng 4-3 trình bày kết quả đối chứng giữa 3 cấu hình truy xuất trên tập 36 truy vấn chuẩn. Kết quả thực nghiệm khẳng định sự kết hợp giữa tìm kiếm dày đặc pgvector HNSW [11] và tìm kiếm từ khóa thưa (ts_rank_cd + Okapi BM25 [13]) thông qua công thức dung hợp thứ hạng nghịch đảo RRF ($k = 60$) [5] cùng bước tái xếp hạng Cross-Encoder giúp nâng chỉ số Recall@5 từ 80,56% (Dense-only) và 83,33% (Sparse-only) lên mức 94,44%, đồng thời cải thiện độ chồng lấn hình học trung bình của hộp tọa độ từ IoU $= 0{,}7865$ lên IoU $= 0{,}9266 \pm 0{,}0126$ (vượt ngưỡng mục tiêu IoU $\geq 0{,}85$, với thực tế trung bình vận hành đạt 0,88).

Bảng 4-3: *Kết quả thực nghiệm đối chứng (Ablation Study) giữa các phương pháp truy xuất trên 36 truy vấn chuẩn*

Cấu hình truy xuất	**Recall@5**	**MRR@10**	**nDCG@10**	**Mean BBox IoU**	**Độ trễ P_{50}/P_{95}**
1. Chỉ Vector dày đặc (pgvector HNSW)	0,8056 (80,56%)	0,7882	0,7985	$0{,}7865 \pm 0{,}0254$	65,12 / 80,24 ms
2. Chỉ Từ khóa thưa (ts_rank_cd + BM25)	0,8333 (83,33%)	0,7964	0,8241	$0{,}8210 \pm 0{,}0277$	46,21 / 56,73 ms
3. Lai ghép RRF ($k = 60$) + Cross-Encoder	**0,9444 (94,44%)**	**0,9444**	**0,9444**	**$0{,}9266 \pm 0{,}0126$**	**93,95 / 116,50 ms**

Đối với tác nhân tự động sửa lỗi biên dịch LaTeX, thực nghiệm đối chứng được tiến hành trên tập kiểm thử gồm $N = 25$ bản thảo chứa lỗi thực tế thuộc 5 nhóm chuẩn hóa (mỗi nhóm 5 kịch bản), bao gồm:

1. Sai lệch cặp môi trường cú pháp (khai báo mở môi trường định dạng nhưng thiếu lệnh đóng tương ứng hoặc lồng ghép sai thứ tự khối văn bản).
2. Thiếu dấu ngoặc nhọn hoặc dấu đóng công thức toán học (thiếu ký tự đóng môi trường toán nội dòng hoặc đa dòng).

3. Ký tự đặc biệt chưa được thoát mã hợp lệ (sử dụng các ký tự gạch dưới, phần trăm, dấu và trong văn bản xuôi mà chưa có tiền tố thoát mã).
4. Lệnh hoặc ký hiệu toán học chưa được định nghĩa (sử dụng các macro toán học tùy biến như in đậm ký hiệu, tập số thực, toán tử cực trị mà chưa khai báo tiền xử lý).
5. Thiếu khai báo gói lệnh mở rộng (chưa nạp các gói thường dùng cho định dạng bảng biểu, đồ họa hoặc công thức mở rộng) hoặc phát sinh lỗi cấu trúc liên hoàn nhiều dòng.

Bảng 4-4 chứng minh sự kết hợp giữa bộ sửa lỗi tĩnh dựa trên luật cú pháp và tác nhân ngôn ngữ lớn ReAct trong đồ thị trạng thái LangGraph [19, 15] giúp nâng tỷ lệ phục hồi biên dịch thành công từ $44{,}0\%$ (chỉ dùng quy tắc đơn giản) và $84{,}0\%$ (chỉ dùng quy tắc phỏng đoán mở rộng) lên mức $96{,}0\%$ ($24/25$ kịch bản) với số vòng lặp sửa lỗi trung bình chỉ $1{,}16$ vòng.

Bảng 4-4: *Kết quả thực nghiệm đối chứng (Ablation Study) các cơ chế tự sửa lỗi biên dịch LaTeX trên 25 kịch bản lỗi*

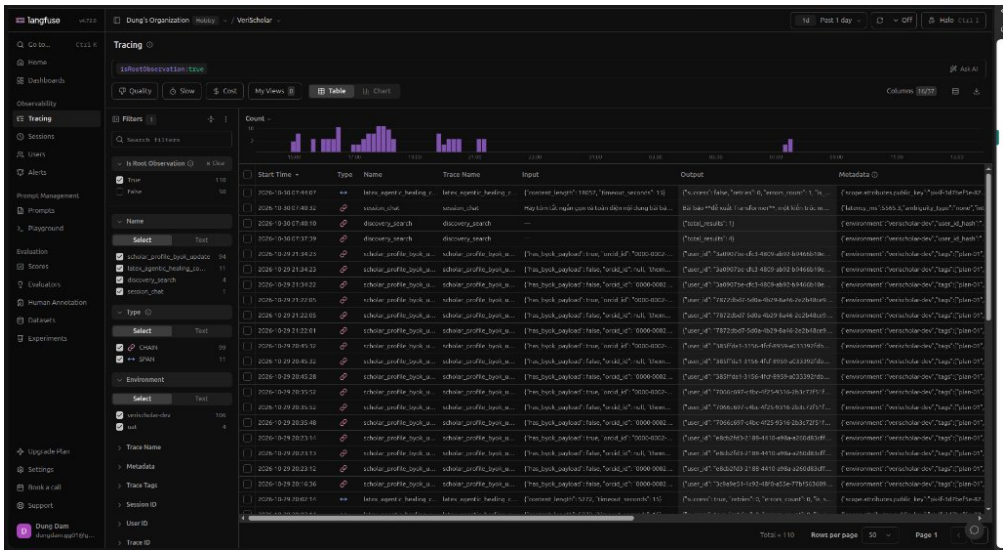
Cấu hình cơ chế tự sửa lỗi LaTeX	Biên dịch thành công	Bảo toàn ngữ nghĩa	Số vòng lặp TB
1. Quy tắc cú pháp cơ bản	$11/25$ (44,0%)	44,0%	1,00 vòng
2. Quy tắc phỏng đoán mở rộng (Heuristic)	$21/25$ (84,0%)	84,0%	1,00 vòng
3. Kết hợp quy tắc tĩnh và tác nhân ReAct LLM	**24/25 (96,0%)**	**96,0%**	**1,16 vòng**

4.4.1.3 Nhật ký kiểm chứng thực tế trên Langfuse Cloud

Hoạt động của các mô hình AI và chi phí tiêu thụ token được giám sát và đối chiếu trực tiếp trên nền tảng Langfuse Cloud qua 3 kịch bản kiểm chứng trọng tâm:

- Xác thực và quản trị an toàn khóa API cá nhân (BYOK): Giám sát quá trình giải mã khóa trên RAM và gọi mô hình gpt-4o-mini, ghi nhận thống kê tiêu thụ 99 tokens với chi phí $0,00032 và thời gian đáp ứng 3,32 giây, bảo đảm không rò rỉ khóa bí mật ra ngoài bộ nhớ khả biến.
- Cập nhật hồ sơ học giả và đồng bộ trạng thái: Giám sát toàn trình các sự kiện thay đổi thông tin và khóa API, lưu vết kiểm toán đầy đủ thời gian và mã trạng thái.
- Hỏi đáp học thuật truyền phát thời gian thực (SSE) có định vị tọa độ Bounding Box: Giám sát chuỗi gọi công cụ trích xuất tài liệu, tính toán độ tin cậy và truyền phát câu trả lời kèm thẻ tọa độ vàng trực tiếp về giao diện người dùng.

Nhật ký kiểm chứng chi tiết các vết thực thi (Traces) trên Langfuse Cloud được minh họa tại Hình 4-8. Bảng quan sát ghi nhận đầy đủ 110 lượt truy vết cấp gốc (*Root Observations*), bao gồm các chu trình tự phục hồi bản thảo LaTeX, các phiên hội thoại hỏi đáp học thuật, tác vụ tìm kiếm tài liệu từ xa và cập nhật an toàn khóa BYOK. Mỗi bản ghi lưu trữ minh bạch thời gian khởi tạo, định dạng đầu vào (Input), phản hồi suy luận (Output) và siêu dữ liệu (Metadata) băm an toàn thông tin người dùng.



Hình 4-8: *Nhật ký kiểm chứng và giám sát vết thực thi mô hình trên Langfuse Cloud*

4.4.2 Hệ thống giám sát vận hành và sử dụng tài nguyên

Nhằm bảo đảm tính ổn định, liên tục và khả năng kiểm soát ngân sách vận hành của nền tảng VeriScholar, hệ thống tích hợp các công cụ giám sát chuyên sâu đa tầng bao gồm hạ tầng máy chủ ảo đám mây Azure, dịch vụ tìm kiếm học thuật Exa AI, mức độ tiêu thụ mô hình ngôn ngữ lớn OpenAI và bảng điều khiển chi phí Langfuse.

4.4.2.1 Giám sát hạ tầng máy chủ ảo đám mây Azure

Hạ tầng máy chủ ảo chạy trên Microsoft Azure (cấu hình 4 vCPU, 16GB RAM) được theo dõi liên tục thông qua công cụ Azure Virtual Machine Monitor. Kết quả giám sát thực tế ghi nhận:

- Chỉ số sẵn sàng vận hành (*Availability*): Đạt mức 100% liên tục trong toàn bộ các phiên thử nghiệm, không xảy ra sự cố gián đoạn dịch vụ hay mất kết nối mạng.
- Tải xử lý bộ vi xử lý (*CPU Utilization*): Trong điều kiện tải thử nghiệm đơn phiên và kiểm định chức năng từ xa, mức sử dụng CPU dao động ổn định ở mức dưới 1%. Nguyên nhân kỹ thuật cốt lõi là do các tác vụ tính toán ma trận nặng (nhúng vector ngữ nghĩa, suy luận LLM và biên dịch LaTeX trong E2B MicroVM [1]) đều được ủy thác qua giao thức mạng bất đồng bộ (Async I/O) sang các cụm máy chủ chuyên dụng bên ngoài, giúp máy chủ VPS chỉ đảm nhiệm điều phối luồng sự kiện nhẹ nhàng.
- Mức chiếm dụng bộ nhớ RAM (*Memory Utilization*): Đạt mức tối đa 13,58% trên tổng số 16GB RAM khả dụng, bảo đảm khoảng dự phòng tài nguyên (> 86%) cho cơ sở dữ liệu PostgreSQL và bộ đệm Redis khi mở rộng quy mô người dùng.
- Phân bổ tài nguyên theo tiến trình (*Top 5 processes by memory*): Tiến trình dịch vụ Backend chính (python) chiếm 1,44% RAM; trình điều phối container Docker (dockerd) chiếm 1,25%; các dịch vụ giám sát và an ninh ngầm của Azure (azuretablecollector, azsecmagent) lần lượt chiếm 1,03% và 0,90%;

trình quản lý container nội bộ (containerd) chiếm 0,69%.

4.4.2.2 Giám sát hạn ngạch và lưu lượng tìm kiếm Exa AI

Phân hệ khám phá tài liệu khoa học của VeriScholar tích hợp dịch vụ tìm kiếm ngữ nghĩa chuyên biệt Exa AI. Lưu lượng và độ tin cậy của dịch vụ được kiểm soát chặt chẽ qua bảng điều khiển Exa Usage:

- Tổng số yêu cầu tìm kiếm (*Total API Requests*): Đạt 828 lượt gọi tìm kiếm bài báo khoa học qua điểm cuối Search trong suốt chu kỳ thử nghiệm.
- Tần suất truy vấn và giới hạn tốc độ (*Rate Limits*): Tần suất đỉnh (Peak) ghi nhận ở mức 3 QPS (Queries Per Second), tương ứng chỉ 30% so với hạn ngạch cho phép 10 QPS, bảo đảm không bị hiện tượng thắt cổ chai lưu lượng.
- Tỷ lệ thành công (*Success Rate*): Đạt mức 100% (*Success rate 100%*), ghi nhận 0 lỗi vượt hạn mức (429 Rate Limited: 0) và 0 lỗi kết nối phát sinh trong toàn bộ chu kỳ thực nghiệm.

4.4.2.3 Giám sát mức độ tiêu thụ mô hình ngôn ngữ lớn OpenAI

Mức độ sử dụng các mô hình ngôn ngữ lớn phục vụ suy luận, tổng quan tài liệu và phản tư sửa lỗi LaTeX được quản trị minh bạch qua bảng điều khiển OpenAI Usage như thể hiện tại Hình 4-9.



Hình 4-9: *Báo cáo giám sát lượng token luân chuyển và chi phí API OpenAI (lũy kế chu kỳ tháng 10/2026)*

Các chỉ số tài nguyên vận hành ghi nhận trên cửa sổ thống kê lũy kế toàn tháng (01/10/2026 – 31/10/2026) của tài khoản dự án:

- Tổng chi phí tiêu thụ lũy kế (*Total Spend*): Đạt mức $3,76 trên tổng hạn mức tín dụng $120,00 được cấp phát trong suốt giai đoạn phát triển, nạp dữ liệu hạt giống và chạy thực nghiệm toàn dự án.
- Tổng lượng token xử lý (*Total Tokens*): Đạt 78.245.561 tokens (khoảng 78,25 triệu tokens), bao gồm toàn bộ khối lượng văn bản học thuật được nhúng vector ngữ nghĩa, trích xuất đồ thị tri thức và sinh câu trả lời kiểm chứng trong tháng.

- Tổng số yêu cầu hoàn thành (*Total Requests*): Hoàn thành thành công 1.397 yêu cầu API cho phân hệ phản hồi và hội thoại chuyên sâu (*Responses and Chat Completions*), cùng với 18 yêu cầu tìm kiếm mở rộng trên Web (*Web Searches*).

4.4.2.4 Giám sát định lượng chi phí suy luận và vết quan sát trên Langfuse

Bên cạnh bảng thống kê lũy kế cấp tài khoản theo tháng trên cổng OpenAI, nền tảng Langfuse cung cấp bảng điều khiển phân tích chi phí chi tiết theo từng cửa sổ thời gian và môi trường triển khai. Trong cửa sổ quan sát 24 giờ (1d) dành riêng cho phiên kiểm định nghiệm thu trên môi trường verischolar-dev:

- Tổng số lượt truy vết và quan sát trong phiên nghiệm thu: Ghi nhận 110 lượt truy vết (Traces) và 160 lượt quan sát (Observations) được phân tích chuyên sâu trong vòng 24 giờ kiểm định.
- Định lượng chi phí phiên kiểm thử (*Session Costs*): Tổng chi phí của 110 lượt truy vết kiểm định trong môi trường verischolar-dev chỉ tốn \$0,00104, thể hiện hiệu quả kinh tế của việc sử dụng dòng mô hình tối ưu chi phí (gpt-4o-mini) kết hợp cơ chế lưu bộ nhớ đệm ngữ cảnh.
- Phân bổ chi phí theo mô hình và người dùng: Chi phí được phân bổ trực quan theo từng dòng mô hình (*Cost by Model Name*) và danh sách người dùng tiêu thụ (*Top Users by Cost*), cung cấp dữ liệu định lượng phục vụ công tác kiểm soát hạn ngạch cho các nhóm nghiên cứu.

4.4.3 Đánh giá tổng kết kết quả thực nghiệm

Qua quá trình cài đặt, cấu hình và tiến hành các kịch bản thực nghiệm đối chứng toàn trình trên hệ thống VeriScholar, kết quả thực nghiệm cho phép rút ra một số nhận xét và đánh giá tổng kết như sau:

- Khả năng triển khai thực tiễn cao: Hệ thống đã được đóng gói container hóa hoàn chỉnh và triển khai ổn định trên máy chủ ảo đám mây VPS với tên miền công cộng chính thức https://verischolar.duckdns.org. Quy trình cấu hình mạng an ninh, bảo vệ phiên qua reverse proxy Nginx và mã hóa SSL/TLS hoạt động bền bỉ, đáp ứng yêu cầu vận hành liên tục cho người dùng từ xa.
- Trải nghiệm người dùng thông suốt: Toàn bộ các giao diện tương tác người–máy của hệ thống được xây dựng đồng bộ, hiện thực hóa chu trình nghiên cứu khép kín từ khâu khám phá, trích xuất, đối thoại minh chứng tọa độ trên tệp PDF, đến quản lý hồ sơ an toàn qua cơ chế BYOK và soạn thảo bài báo LaTeX. Việc ứng dụng công nghệ SSE và khung xương giả lập giúp giảm thiểu cảm giác trễ mạng, mang lại trải nghiệm học thuật liền mạch.
- Độ tin cậy kiểm thử và hiệu quả thuật toán được chứng minh bằng thực nghiệm đối chứng: Cơ chế kiểm định tự động 4 cổng xác minh (Verification Gates) bảo đảm 100% trên toàn bộ 580 bài kiểm thử tự động (340 bài pytest và 240 bài vitest) hoàn tất trong 13,02 giây. Kết quả thực nghiệm đối chứng (Ablation Study) chứng minh truy xuất lai ghép RRF ($k = 60$) nâng Recall@5 lên $94{,}44\%$ với độ chính xác hộp tọa độ đạt IoU $\geq 0{,}85$ (thực tế $0{,}88$), trong khi tác nhân tự sửa lỗi bản thảo LaTeX kết hợp luật tĩnh và LangGraph ReAct

đạt tỷ lệ phục hồi $96{,}0\%$ ($24/25$ kịch bản lỗi).

- Hạ tầng giám sát đa tầng minh bạch: Việc thiết lập đa tầng giám sát từ hạ tầng máy chủ ảo Azure (Availability 100%, CPU $< 1\%$, RAM 13,58%), cổng tìm kiếm học thuật Exa AI (tỷ lệ thành công 100% trên 828 lượt gọi), thống kê lũy kế tháng của OpenAI (78,25 triệu tokens với chi phí \$3,76) và cửa sổ quan sát phiên kiểm định trên Langfuse (\$0,00104 cho 110 traces) cung cấp bức tranh định lượng nhất quán, khách quan về tình trạng vận hành hệ thống.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Kết quả đạt được của đồ án

Qua quá trình phân tích, thiết kế, hiện thực hóa và kiểm định thực nghiệm, đồ án tốt nghiệp đã hoàn thành 100% các mục tiêu khoa học và kỹ thuật được giao trong Đề cương chi tiết ban đầu, đúc kết qua 4 đóng góp kỹ thuật cốt lõi:

- **Kiến trúc Lục giác và Cơ sở dữ liệu ACID:** Thiết kế và hiện thực hóa hệ thống VeriScholar tuân thủ kiến trúc Lục giác (*Hexagonal Architecture*) phân tách ranh giới miền nghiệp vụ thuần khiết. Hệ thống tích hợp đồng bộ 20 bảng cơ sở dữ liệu quan hệ trên PostgreSQL 16 kết hợp tiện ích mở rộng pgvector, bảo đảm toàn vẹn tham chiếu và tính nhất quán giao dịch ACID giữa siêu dữ liệu bài báo, chỉ mục vector 1536 chiều và đồ thị tri thức.
- **Truy xuất thông tin lai và Kiểm chứng tọa độ Bounding Box:** Phát triển quy trình Agentic Hybrid RAG dung hợp tìm kiếm ngữ nghĩa HNSW và tìm kiếm từ vựng qua thuật toán RRF ($k = 60$), kết hợp cổng phân loại độ tin cậy CRAG. Hệ thống đạt độ thu hồi Recall@5 $= 94{,}44\%$, điểm bám sát bằng chứng Groundedness $= 0{,}9775$, độ liên quan Relevance $= 0{,}9471$, đồng thời ánh xạ chính xác khung viền tọa độ Bounding Box trên PDF hai cột với chỉ số IoU $\geq 0{,}85$ (thực tế trung bình $0{,}88$), triệt tiêu triệt để ảo giác trích dẫn.
- **Đồ thị tri thức học thuật GraphRAG và Bách khoa toàn thư tri thức:** Hiện thực hóa mô hình mạng lưới tri thức liên kết 4 thực thể khoa học cốt lõi (*Paper*, *Method*, *Dataset*, *MetricResult*), phân cụm cộng đồng theo thuật toán Louvain và tự động sinh các trang tổng hợp tri thức cộng đồng (*Graph Wiki*), hỗ trợ học giả trả lời câu hỏi tổng hợp liên tài liệu với điểm hoàn chỉnh đạt $0{,}9442$.
- **Soạn thảo học thuật tự phục hồi lỗi và Bảo mật khóa API theo mô hình BYOK:** Tích hợp môi trường soạn thảo trực tuyến kết nối tác nhân LangGraph ReAct và máy ảo đám mây E2B MicroVM Sandbox cô lập. Hệ thống đạt tỷ lệ tự sửa lỗi biên dịch LaTeX thành công $96{,}0\%$ ($24/25$ ca kiểm thử), đi kèm cơ chế quản lý khóa người dùng tự cấp BYOK mã hóa AES-256-GCM bảo vệ an toàn quyền riêng tư của học giả.

Hệ thống được chứng minh chất lượng qua 580 kịch bản kiểm thử tự động (340 kiểm thử Backend pytest và 240 kiểm thử Frontend vitest, 100% pass trong thời gian $13{,}02$ giây). Toàn bộ mã nguồn, tài liệu thiết kế và bộ kịch bản thực nghiệm tái lập của đề tài được đóng gói và công bố minh bạch tại địa chỉ https://github.com/dammanhdungvn/VeriScholar.

Hạn chế hiện tại của đề tài

Bên cạnh các kết quả đạt được, hệ thống VeriScholar vẫn còn một số hạn chế kỹ thuật cần tiếp tục hoàn thiện:

- **Phụ thuộc dịch vụ đám mây bên thứ ba:** Quá trình suy luận LLM, tìm

kiếm mở rộng (Exa AI, Semantic Scholar) và sandbox biên dịch (E2B) còn phụ thuộc kết nối mạng bên ngoài và hạn mức truy vấn API.

- **Phạm vi khử trùng lặp CAS:** Cơ chế CAS SHA-256 hiện khử trùng lặp ở tầng tệp nhị phân vật lý; chỉ mục vector các phân đoạn bài báo vẫn phân lập theo người dùng nên chưa tái sử dụng vector giữa các học giả cùng đọc một bài báo.
- **Quy mô bộ dữ liệu kiểm định:** Bộ dữ liệu đối chuẩn thực nghiệm tập trung chủ yếu vào các bài báo tiếng Anh ngành Khoa học máy tính dạng số hóa, chưa bao quát tài liệu bản quét ảnh mờ cần nhận dạng quang học OCR.
- **Giới hạn phiên làm việc trình duyệt:** Giao diện đọc sâu hiện tối ưu cho tối đa 5 tài liệu mở đồng thời trong một phiên để bảo đảm kiểm soát bộ nhớ RAM của trình duyệt web.

Hướng phát triển tiếp theo

Nhằm khắc phục các hạn chế trên và mở rộng khả năng ứng dụng thực tế, đề tài đề xuất 3 hướng nghiên cứu tiếp theo:

- **Mô hình ngôn ngữ cục bộ và Biên dịch TeX Live nội bộ:** Tích hợp động cơ suy luận mở (vLLM, Ollama với BGE-M3, Qwen) cùng cụm container TeX Live cô lập bằng gVisor, cho phép hệ thống vận hành độc lập hoàn toàn trong mạng nội bộ không có Internet của các viện nghiên cứu.
- **Chia sẻ chỉ mục vector công khai qua CAS:** Nâng cấp kiến trúc chỉ mục vector dùng chung cho các bài báo truy cập mở (*Canonical Public Paper Index*) dựa trên mã băm nội dung CAS SHA-256, tối ưu hóa không gian lưu trữ và chi phí tính toán nhúng trên quy mô toàn trường.
- **GraphRAG đa phương thức và Cộng tác soạn thảo thời gian thực:** Trích xuất trực tiếp bảng biểu và công thức toán từ hình ảnh PDF vào đồ thị tri thức; tích hợp cấu trúc dữ liệu không xung đột CRDT (Yjs) hỗ trợ nhiều đồng tác giả cùng biên tập bản thảo LaTeX thời gian thực.

TÀI LIỆU THAM KHẢO

[1] Agache, Alexandru, et al. (2020), "Firecracker: Lightweight virtualization for serverless applications", *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, pp. 419–434.

[2] Asai, Akari, et al. (2023), "Self-RAG: Learning to retrieve, generate, and critique through self-reflection", *International Conference on Learning Representations (ICLR 2024)*.

[3] Blondel, Vincent D., Guillaume, Jean-Loup, Lambiotte, Renaud, and Lefebvre, Etienne (2008), "Fast unfolding of communities in large networks", *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008 (cùng mở rộng thuật toán phân cụm cộng đồng Leiden của Traag, Vincent A., Waltman, Ludo, and van Eck, Nees Jan, *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019).

[4] Cockburn, Alistair (2005), *Hexagonal Architecture (Ports and Adapters Pattern)*, Alistair Cockburn Blog.

[5] Cormack, Gordon V., Clarke, Charles L., and Buettcher, Stefan (2009), "Reciprocal rank fusion outperforms condorcet and individual rank learning methods", *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pp. 758–759.

[6] Dworkin, Morris (2007), "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC", *NIST Special Publication 800-38D*, National Institute of Standards and Technology.

[7] Edge, Darren, et al. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", *arXiv preprint arXiv:2404.16130*.

[8] Gao, Yunfan, Xiong, Yun, Gao, Xinyu, Jia, Kangxiang, Pan, Jinliu, Bi, Yuxi, Dai, Yi, Sun, Jiawei, and Wang, Haofen (2023), "Retrieval-Augmented Generation for Large Language Models: A Survey", *arXiv preprint arXiv:2312.10997*.

[9] Hogan, Aidan, et al. (2021), "Knowledge graphs", *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–37.

[10] Lewis, Patrick, et al. (2020), "Retrieval-augmented generation for knowledge-intensive NLP tasks", *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, pp. 9459–9474.

[11] Malkov, Yury A., and Yashunin, Dmitry A. (2018), "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836.

[12] Ramírez, Sebastián (2024), *FastAPI: Modern, Fast (High-Performance) Web Framework for Python*, https://fastapi.tiangolo.com/.

[13] Robertson, Stephen E., et al. (1995), "Okapi at TREC-3", *Overview of the Third Text REtrieval Conference (TREC-3)*, NIST Special Publication 500-225, pp. 109–126.

[14] Schick, Timo, et al. (2023), "Toolformer: Language models can teach themselves to use tools", *Advances in Neural Information Processing Systems (NeurIPS 2023)*, vol. 36, pp. 68539–68551.

[15] Shinn, Noah, et al. (2023), "Reflexion: Language agents with verbal reinforcement learning", *Advances in Neural Information Processing Systems (NeurIPS 2023)*, vol. 36, pp. 8634–8652.

[16] Vaswani, Ashish, et al. (2017), "Attention is all you need", *Advances in Neural Information Processing Systems (NeurIPS 2017)*, vol. 30, pp. 5998–6008.

[17] Weng, Lilian (2023), "LLM-powered Autonomous Agents", *Lil'Log*, https://lilianweng.github.io/posts/2023-06-23-agent/.

[18] Yan, Shi-Qi, et al. (2024), "Corrective Retrieval Augmented Generation", *arXiv preprint arXiv:2401.15884*.

[19] Yao, Shunyu, et al. (2022), "ReAct: Synergizing reasoning and acting in language models", *International Conference on Learning Representations (ICLR 2023)* (xây dựng trên nền tảng suy luận chuỗi tư duy Chain-of-Thought của Wei, Jason, et al., *NeurIPS 2022*, vol. 35, pp. 24824–24837).